Vietnam General Confederation of Labor

Ton Duc Thang University

Faculty of Information Technology



Dinh Quoc Cuong – 523H0008

Vo Huynh Minh Duc – 523H0014

Building a Video Sharing Platform with an Automated HLS Transcoding Pipeline on a Serverless Container and Event-Driven Architecture

INFORMATION TECHNOLOGY PROJECT

SOFTWARE ENGINEERING

Ho Chi Minh City, 2026

Vietnam General Confederation of Labor

Ton Duc Thang University

Faculty of Information Technology



Dinh Quoc Cuong – 523H0008

Vo Huynh Minh Duc – 523H0014

Building a Video Sharing Platform with an Automated HLS Transcoding Pipeline on a Serverless Container and Event-Driven Architecture

INFORMATION TECHNOLOGY PROJECT

SOFTWARE ENGINEERING

Advised by

MSc. Mai Van Manh

Ho Chi Minh City, 2026

Acknowledgment

First and foremost, we would like to express our sincere gratitude to our supervisor, **MSc. Mai Van Manh**, for his invaluable guidance, constructive feedback, and continuous encouragement throughout the duration of this project. His expertise, patience, and dedication have played a crucial role in shaping both the technical direction and the academic rigour of this work.

We are also deeply thankful to the lecturers and staff of the Faculty of Information Technology at Ton Duc Thang University, whose support and resources provided a strong foundation for our work. Their commitment to creating an inspiring and intellectually stimulating environment has greatly contributed to our growth.

Our gratitude further extends to our classmates and friends who offered helpful discussions, motivation, and meaningful insights during the development and evaluation of this project. Their companionship made the challenging moments far more manageable.

Finally, we are profoundly grateful to our families for their unconditional support, patience, and encouragement. Their belief in us has always been our greatest source of strength and determination.

To all of you, we extend our heartfelt thanks.

Ho Chi Minh City, 11 November 2026

Authors

(Signature and full name)

Dinh Quoc Cuong **Vo Huynh Minh Duc**

Certification

This project report was carried out at **Ton Duc Thang University**.

Advisor: MSc. Mai Van Manh

.....
(Title, Full Name and Signature)

This project report was defended at the Project Examination Committee held at Ton Duc Thang University on . . . . . / . . . . . / . . . . . .

Confirmation of the Chairman of the Project Examination Committee and the Dean of the Faculty after receiving the revised report (if any).

CHAIRMAN **DEAN OF FACULTY**

. . . . . .

Declaration of Authorship

We hereby declare that this is our own project, carried out at **Ton Duc Thang University** under the academic supervision of **MSc. Mai Van Manh**. The analyses, discussions and results presented in this report are truthful and original, and have not been published in any form before this submission.

The data used for analysis, evaluation and comparison were collected by the authors, and the sources they come from are cited in the references of this report. Evaluations, comments and figures quoted from other authors, organisations or research works are referenced and acknowledged where they are used.

If any form of academic dishonesty, plagiarism or copyright infringement is discovered, we accept full responsibility for the content of this report. Ton Duc Thang University bears no responsibility for any infringement of rights or copyrights committed by the authors during the project.

Ho Chi Minh City, 11 November 2026

Authors
(Signature and full name)

Dinh Quoc Cuong **Vo Huynh Minh Duc**

Abstract

Building a Video Sharing Platform with an Automated HLS Transcoding Pipeline on a Serverless Container and Event-Driven Architecture

A video sharing platform must turn every upload, whatever its format and resolution, into HTTP Live Streaming (HLS) renditions that a browser can play at adaptive quality. Transcoding is a bursty workload: a server kept running to wait for uploads is idle most of the time, yet the platform must still absorb many uploads that arrive at once.

This project designs, builds and evaluates a complete video sharing platform whose transcoding runs on serverless containers driven by events. The arrival of a file in Amazon S3 is queued by Amazon SQS and starts one AWS Step Functions execution per upload, which runs FFmpeg as AWS Batch jobs on AWS Fargate, Spot capacity first and On-Demand capacity as overflow, so no transcoding capacity exists while nothing is processed. FFmpeg decodes the source once and writes up to six renditions from 144p to 1080p, each with a capped constant rate factor and key frames aligned across rungs; videos longer than five minutes are cut into chunks that are encoded in parallel and stitched into ordinary playlists. The segments are encrypted with AES-128 and served by Amazon CloudFront only to requests that carry a policy signed for that video. Uploads are screened with Amazon Rekognition, and staff decisions are recorded so that the error rates of the screening can be measured. Eleven Terraform modules describe the infrastructure, and two GitHub Actions workflows lint, test and scan every change.

Measured on the production system, a four-hour video was ready 38.3 minutes after its upload, and of one hundred uploads started at once 92 reached the ready state while the other 8 were refused at creation with a status the client can retry. Once cached, the requests before the first frame took 435 ms at the 95th percentile. Processing three hundred five-minute videos a month is estimated at 31.87 USD, against 309.56 USD with an always-on server.

Contents

Acknowledgment i

Certification ii

Declaration of Authorship iii

Abstract iv

List of Figures ix

List of Tables xi

List of Abbreviations xiv

1 Introduction 1

1.1 Background and Rationale 1
1.2 Project Objectives 2
1.3 Scope and Deliverables 2
1.4 Research Methodology 3
1.5 Practical Significance 4
1.6 Main Contributions 4
1.7 Structure of the Report 5

2 Literature Review and Technologies 6

2.1 Related Platforms and Comparison 6
2.2 Core Models and Principles 8
2.2.1 Serverless Containers 8
2.2.2 Event-Driven Architecture and Workflow Orchestration 10
2.2.3 HTTP Live Streaming and Adaptive Bitrate 10
2.2.4 The Bitrate Ladder and Capped CRF 12

2.3 Technology Stack Justification 13

3 Requirements Analysis **16**
3.1 Stakeholders and User Requirements 16
3.2 Functional Requirements 18
3.3 Non-Functional Requirements 20
3.4 Use Case Model 22
3.4.1 Use Case Diagrams 22
3.4.2 Use Case Specifications 22

4 System Design **39**
4.1 High-Level Architecture 39
4.2 Transcoding Workflow 42
4.2.1 Ingest and Idempotency 42
4.2.2 State Machine 42
4.2.3 Compute Sizing and Queues 44
4.3 Media Design 45
4.3.1 HLS Output 45
4.3.2 FFmpeg Processing Graph 47
4.3.3 Bitrate Ladder 49
4.3.4 Long Videos: the Chunked Path 50
4.3.5 Encryption and Playback 52
4.3.6 Adaptive Bitrate in the Player 54
4.4 Database Design 55
4.5 API Design 59
4.6 Interface Design 59
4.7 Security Design 60
4.8 Content Moderation Design 63
4.8.1 Automatic Screening 63
4.8.2 Review, Appeals and Audit 65
4.8.3 Roles and Separation of Duties 66
4.9 Performance and Scalability 66

5 Development and Testing **68**
5.1 Repository Structure 68
5.2 Processing Pipeline 69
5.2.1 Workflow Functions 69

5.2.2 Transcoder Image . . . 70
5.2.3 Failure Handling Inside a Job . . . 70
5.3 API and Web Application . . . 71
5.4 Infrastructure as Code . . . 71
5.5 Continuous Integration and Deployment . . . 72
5.6 Challenges During Development . . . 74
5.6.1 One Policy for the Processed Bucket . . . 75
5.6.2 Index Creation at Start-up . . . 75
5.7 Testing . . . 75

6 Deployment and Evaluation **79**
6.1 Deployed Environment . . . 79
6.2 User Interface . . . 80
6.3 Measurement Method . . . 83
6.4 Playback . . . 83
6.4.1 Time to First Frame . . . 83
6.4.2 Playback under Limited Bandwidth . . . 84
6.4.3 Declared and Delivered Bit Rate . . . 85
6.5 Transcoding . . . 86
6.5.1 Time by File Size . . . 86
6.5.2 A Four-Hour Video . . . 87
6.5.3 Damaged Uploads . . . 88
6.6 Behaviour under Load . . . 89
6.6.1 One Hundred Simultaneous Uploads . . . 89
6.6.2 API . . . 90
6.7 Cost . . . 91
6.8 Evidence for the Non-Functional Requirements . . . 92

7 Conclusion and Future Work **93**
7.1 Summary of Achievements . . . 93
7.2 Strengths of the System . . . 94
7.3 Limitations . . . 95
7.4 Future Work . . . 96
7.5 Task Distribution . . . 97

References **98**

A Complete REST API Reference **101**

B Environment Variables Reference **106**

C Installation and Execution Guide **109**

C.1 Prerequisites 109

C.2 Installation Steps 109

C.3 Running Tests 109

C.4 Production Deployment 110

D Measurement Data **111**

D.1 Playback 112

D.2 Transcoding 114

D.3 Load 116

D.4 Prices 117

List of Figures

2.1 *Compute models for a bursty transcoding workload: an always-on instance compared with on-demand serverless containers* 9
2.2 *HLS artefacts produced from one source file and the order in which a player retrieves them* 12

3.1 *Use case diagram of viewers* 23
3.2 *Use case diagram of creators and the processing system* 24
3.3 *Use case diagram of moderation and operation* 25

4.1 *Architecture of the platform and its deployment boundaries; the numbered steps are listed in the key on the left* 40
4.2 *State machine of the transcoding workflow, with its Catch transitions and retry rules* 43
4.3 *HLS package of one video in the processed bucket, and where its key comes from* 46
4.4 *FFmpeg processing graph of a single-path encode: one decode, one chain per rung* 48
4.5 *Cap of each rung against the measured average bit rate of two production videos* 50
4.6 *The chunked path: plan on GOP boundaries, parallel encoding, checked join* 51
4.7 *Playback of an encrypted video: grant, signed CDN requests and key delivery* 53
4.8 *Quality selection loop of hls.js with the settings of the player* 55
4.9 *Collections of the product in MongoDB, with their keys and unique indexes* 56
4.10 *Collections of moderation and operation* 58
4.11 *User flow from an anonymous visit to a published video* 61
4.12 *Verifying the owner of an account: by the e-mailed link, or by Google sign-in with the same address* 62

4.13 *Moderation: automatic screening, a viewer's report, an operator's decision and the owner's appeal* 64

5.1 *The checks run by GitHub Actions on every push and pull request, and the deployment steps run after a merge* 73

6.1 *Watch page with the quality menu of the player open* 80
6.2 *Search results for a query typed without diacritics* 81
6.3 *Channel page* 81
6.4 *Upload page* 82
6.5 *Accuracy tab of the staff console* 82
6.6 *Requests before the first frame of a 720p rendition, by step* 84
6.7 *Time from the end of the upload to READY, by file size; the bar is the median run of three and the line below it the range* 86
6.8 *Timeline of a four-hour video on the chunked path* 88
6.9 *One hundred simultaneous uploads: time to READY of each video, split into waiting and processing* 89

List of Tables

2.1 Comparison of the platform with related products and reference architectures 7
2.2 Technology choices, alternatives and reasons 14

3.1 Stakeholders, their needs and the requirements that serve them 17
3.2 Functional requirements of accounts, uploading, processing and watching 18
3.3 Functional requirements of the community, the library and the channel . 19
3.4 Functional requirements of moderation and operation 20
3.5 Non-functional requirements of the system 21
3.6 Use case UC-01: register and verify the e-mail address 26
3.7 Use case UC-02: sign in with a password or with Google 27
3.8 Use case UC-03: reset or change the password 28
3.9 Use case UC-04: browse and search 28
3.10 Use case UC-05: watch a video 29
3.11 Use case UC-06: unlock a password-protected video 30
3.12 Use case UC-07: like, comment and reply 30
3.13 Use case UC-08: subscribe to a channel and read notifications 31
3.14 Use case UC-09: join a channel membership 31
3.15 Use case UC-10: keep a library 32
3.16 Use case UC-11: upload a video 33
3.17 Use case UC-12: edit and delete a video in the studio 34
3.18 Use case UC-13: schedule the publication 34
3.19 Use case UC-14: customise the channel and manage membership tiers . 35
3.20 Use case UC-15: transcode a video automatically 36
3.21 Use case UC-16: report a video or a comment 36
3.22 Use case UC-17: decide on a video or a comment 37
3.23 Use case UC-18: appeal a decision 37
3.24 Use case UC-19: audit a random sample and read the error rates 38

3.25 Use case UC-20: assign a role 38

4.1 Batch job roles, their size and their queue 44
4.2 The six-rung Capped-CRF ladder 49
4.3 Short-lived key-value data in DynamoDB 57
4.4 Route groups of the API 59
4.5 Permissions of the staff roles 63
4.6 Outcome classes of a human decision against the machine verdict 65
4.7 MongoDB connection budget of the production environment 66

5.1 Workspaces of the repository and where each one runs 69
5.2 Terraform modules and their responsibilities 72
5.3 Problems met during development, their causes and resolutions 74
5.4 Test levels, their size and where they run 76
5.5 Unit tests by workspace, 305 tests in 32 files 77

6.1 The production environment 79
6.2 Measurements, their tools and their samples 83
6.3 Playback of the watch page under three network profiles, five runs each . 84
6.4 Declared and delivered bit rate of a production recording, in kbit/s (47 segments per rung) 85
6.5 Transcoding time by file size, three runs each 86
6.6 Latency of read requests through the Worker, in milliseconds 90
6.7 Monthly cost of processing 300 five-minute videos, list prices in Singapore 91
6.8 Non-functional requirements and their evidence 92

7.1 Objectives of the project against their results 94
7.2 Task distribution between the members of the team 97

A.1 Routes of the API 101

B.1 Environment variables by component 106
B.2 Fields of the API secret in AWS Secrets Manager 108

D.1 Sources of the tables and figures of Chapter 6 111
D.2 Requests before the first frame of the 720p rendition, in milliseconds; the first request and 19 that followed it 112
D.3 Playback runs of 60 seconds under three network profiles 113

D.4 Playback runs of 90 seconds that start on Fast 3G and drop to Slow 3G after 20 seconds: the rung shown before the drop, and the seconds from the drop until a lower rung was asked for and shown 113
D.5 Declared (decl.) and delivered (deliv.) bit rate of three videos, in kbit/s; count of segments, and length of a segment without the last one 114
D.6 Transcoding runs by file size, in seconds from the end of the upload; intake ends with the probe, and start is when the first task starts; encoding is the single job or the chunk array 115
D.7 Steps of the four-hour video, 5.259 GiB, 48 chunks of 4 vCPUs 115
D.8 Uploads of damaged files on production; time in seconds from the end of the upload to the end of the workflow 115
D.9 One hundred simultaneous uploads: 92 ready (89 timed, 3 of the 3 with a refused completion call) and 8 refused at creation; times of the timed videos in seconds 116
D.10 Latency of read requests through the Worker by phase, in milliseconds . 116
D.11 Log of the API function during the API run and the burst of uploads; times in milliseconds 116
D.12 List prices in Singapore, in USD 117

List of Abbreviations

AAC	Advanced Audio Coding.
AES	Advanced Encryption Standard.
API	Application Programming Interface.
AWS	Amazon Web Services.
CDN	Content Delivery Network.
CI	Continuous Integration.
CRF	Constant Rate Factor.
CSS	Cascading Style Sheets.
DNS	Domain Name System.
DRM	Digital Rights Management.
FR	Functional Requirement.
GOP	Group of Pictures.
HLS	HTTP Live Streaming.
HTTP	Hypertext Transfer Protocol.
IV	Initialisation Vector.
JSON	JavaScript Object Notation.
NFR	Non-Functional Requirement.
REST	Representational State Transfer.
S3	Amazon Simple Storage Service.
SNS	Amazon Simple Notification Service.
SQS	Amazon Simple Queue Service.
TLS	Transport Layer Security.
UC	Use Case.
URL	Uniform Resource Locator.
VBV	Video Buffering Verifier.
vCPU	Virtual Central Processing Unit.
VOD	Video on Demand.

VPC Amazon Virtual Private Cloud.

Chapter 1

Introduction

1.1 Background and Rationale

Video has become the dominant form of content on the Internet, and platforms such as YouTube and Vimeo have made uploading and sharing video an everyday activity. Behind that apparent simplicity lies a demanding engineering problem. A source file may use any container, codec, resolution or frame rate, yet the platform must deliver a stream that starts quickly on every device and adapts to whatever bandwidth the viewer has at that moment.

The industry answer is HTTP Live Streaming (HLS) [1]. HLS cuts a video into short segments and encodes the same content at several quality levels, called renditions. A master playlist lists the renditions, each rendition has its own media playlist of segments, and the player chooses among them while it plays according to the throughput it measures. This mechanism, Adaptive Bitrate Streaming (ABR), lets playback lower its quality when a connection weakens instead of stalling.

Producing those renditions is expensive, and this is where the architectural difficulty lies. Transcoding is a bursty workload: it needs many processor cores while a video is being encoded and none while nobody uploads. A platform that keeps servers running to wait for uploads pays for idle capacity most of the time, while a platform sized for the average load cannot absorb a burst of uploads and makes them wait. This tension between idle cost and burst capacity motivates the project. Instead of keeping compute allocated, the system described here starts containers only when a video arrives and releases them as soon as the work is done, so that processing costs money only while it runs.

1.2 Project Objectives

The project builds a complete video sharing platform together with the automated transcoding pipeline behind it, and has five objectives. The first is a web platform on which users register, upload videos, watch them at several quality levels, and take part in a community through channels, subscriptions, comments, playlists and memberships, with the visibility of each video chosen by its owner. The second is a fully event-driven pipeline: the arrival of a source file in Amazon Simple Storage Service (S3) alone starts the processing, which is coordinated by AWS Step Functions, the workflow service of Amazon Web Services (AWS), and ends with an encrypted HLS package served through a content delivery network (CDN). The third is to run every FFmpeg job in a container on AWS Batch with AWS Fargate, created on demand and released when the queue is empty, using Spot capacity first and On-Demand capacity only as overflow. The fourth is to describe the whole environment as Infrastructure as Code and to check every change automatically with a continuous integration (CI) pipeline that also runs security scans. The fifth is to evaluate the result on the deployed system: playback start time and quality, transcoding time from a two-minute clip to a four-hour video, behaviour under one hundred simultaneous uploads, and operating cost.

The first objective defines what the platform offers its users, and the second and third how the processing is carried out; on-demand containers driven by events are the central hypothesis of the project. The fourth makes the system reproducible and safe to change, and the fifth turns the claims of the other four into measurements. Table 7.1 returns to each objective with its result.

1.3 Scope and Deliverables

The project delivers the full vertical slice of a video sharing service: a single-page web application with its own HLS player, a Representational State Transfer (REST) application programming interface (API), the transcoding workflow and its container image, the cloud infrastructure that hosts them, and the scripts used to deploy and measure them. Besides viewers and creators, the platform serves three staff roles, operator, auditor and administrator, who review flagged content, audit the automatic screening and manage accounts. The production system runs at the domain zelostech.site. Its interface carries the brand name VidShare, which the screenshots of Chapter 6 show; Zelos, the name in the domain, is the prefix of the cloud resources and of the code packages.

Several areas are deliberately left out. The project does not implement digital rights

management (DRM) such as Widevine or FairPlay: its 128-bit Advanced Encryption Standard (AES-128) encryption keeps segments unreadable to anyone without access, but cannot stop an authorised viewer from saving what they watch. It does not implement live streaming, since the pipeline is designed for video on demand. It does not optimise the bitrate ladder per title; every video receives the rungs of a fixed six-rung ladder from 144p to 1080p that its source resolution can fill, and each rung uses a quality-targeted encoding with a bitrate cap. Payments for channel memberships run in a sandbox, and the production database runs on the free tier of MongoDB Atlas, whose limits are measured and discussed in Chapter 6.

1.4 Research Methodology

The work combines a review of existing practice with an iterative build-and-measure cycle.

The theoretical part established how the problem is solved at industrial scale before the design was fixed. Engineering accounts from Netflix describe how a large platform splits video processing into independent steps and tunes its encoding per title [2, 3], and the AWS Video on Demand solution shows the managed-service equivalent, in which a workflow engine owns each job and every step reports its state [4]. The protocol was taken from its specification [1] and the encoding parameters from the FFmpeg documentation [5], rather than from secondary sources. Studies of serverless video processing supplied the motivation for the architecture [6, 7].

The practical part followed an incremental model. The infrastructure and the transcoding workflow were built first and run on a staging environment, then the API, then the web application, and only then was the production environment created from the same Terraform code. Every push and pull request runs linting, type checks, unit tests, a smoke test of the transcoder image and the security scans, so each change is reviewed before it is merged.

The evaluation was empirical. Every figure in Chapter 6 was produced by a measurement script kept in the repository and run against the production system in its final configuration, and the raw output of each run was stored as a JavaScript Object Notation (JSON) file; Table D.1 maps each table and figure to the run that produced it.

1.5 Practical Significance

The direct beneficiaries of this design are small teams and individual creators who need a video platform but cannot justify a transcoding cluster that runs all the time. Because compute exists only while a job runs, the cost of the platform follows the number of uploads rather than the number of hours in a month; Section 6.7 compares it with an always-on server sized for the same work. This makes video hosting affordable for a service whose uploads are intermittent, which describes most platforms before they grow large.

The design also addresses a requirement that is easy to state and easy to get wrong. Marking a video private means nothing if its segments remain reachable by anyone who knows their address. Here the processed files stay in a private bucket that only the content delivery network can read, the network accepts only requests signed for one video, and the segments are encrypted with a key that the API releases only after checking access. This scheme can be reused by any application that serves user-owned media from object storage.

1.6 Main Contributions

The main contribution of this work is an end-to-end demonstration that serverless containers, coordinated by a workflow engine, can serve as the transcoding backbone of a video platform, with a measured account of what that choice costs and what it delivers. Several parts can be reused on their own. An upload event travels from Amazon S3 through a queue to a Step Functions execution named after the video and its upload version, so a duplicated event never starts a second run. The FFmpeg graph decodes the source once and produces every rendition with aligned keyframes, so the player can switch quality at any segment boundary. Long videos are cut into chunks on keyframe boundaries, encoded in parallel as one Batch array job and stitched into ordinary playlists; a four-hour source becomes playable in about thirty-eight minutes within the capacity of one AWS account. Finally, the processing containers hold no database credentials: they report progress through events, and a single function owns every processing state written to the database.

1.7 Structure of the Report

The rest of this report is organised as follows. Chapter 2 reviews comparable platforms, explains the models behind the design and justifies the choice of technologies. Chapter 3 describes the users and stakeholders, states the functional and non-functional requirements, and specifies the use cases. Chapter 4 presents the architecture, analyses the transcoding workflow and the HLS output in detail, and describes the database, the API, the interface, security and content moderation. Chapter 5 covers the implementation of each part, the infrastructure code, the problems met during development and the automated tests. Chapter 6 presents the deployed system and measures playback, transcoding, behaviour under load and cost, then sets the requirements against this evidence. Chapter 7 compares the results with the objectives and states the strengths, limitations and future work.

Chapter 2

Literature Review and Technologies

This chapter first looks at how comparable platforms and reference architectures handle uploaded video, then explains the models on which the design rests, and finally justifies the technologies chosen for each part of the system.

2.1 Related Platforms and Comparison

YouTube, developed by Google, is the largest video sharing platform and the functional model for this project: channels, subscriptions, comments, playlists, paid memberships and a creator studio with analytics are all features its users expect. Its documentation asks creators to upload an MP4 file with H.264 video and Advanced Audio Coding (AAC) audio, about 8 Mbit/s for 1080p at 30 frames per second [8], which shows that the platform treats the upload as a high-quality master and produces its own delivery formats from it. Vimeo, developed by Vimeo, Inc., targets creators and businesses rather than a mass audience. It accepts H.264, H.265 or ProRes and recommends encoding the upload at a constant rate factor (CRF) of 18 or below [9], again a quality-targeted master from which the platform builds its own renditions. Neither platform documents its internal pipeline, and neither can be self-hosted, so they serve here as the reference for features and user expectations rather than for architecture.

Netflix offers the most thoroughly documented pipeline. Its video processing was rebuilt as independent services, for inspection, encoding and quality measurement, on a platform called Cosmos [2]. Cosmos is layered: an API tier receives requests, a workflow tier orchestrates the steps, and a serverless compute tier performs the media work, the

tiers being connected asynchronously [10]. Two ideas from this design are applied here at a much smaller scale. The first is the separation between a workflow that owns the job and the compute that performs each step. The second is chunk-based encoding, in which many workers encode parts of the same title in parallel instead of one worker encoding it from start to end.

Amazon Web Services publishes the Video on Demand (VOD) on AWS solution, a reference architecture built entirely from managed services [4]. An upload to Amazon S3 starts an AWS Step Functions workflow, AWS Elemental MediaConvert produces the renditions, and Amazon CloudFront delivers them. The architecture of this project follows the same overall shape, a workflow engine that owns each video, but replaces MediaConvert with FFmpeg in containers on AWS Batch and Fargate. This substitution is the central trade-off of the project. MediaConvert needs almost no operational effort but is billed per minute of output and fixes what the encoder can do, whereas a self-packaged container is cheaper per unit of work and fully controllable, at the price of maintaining the image and handling failures in the team's own code. Table 2.1 summarises the comparison.

Table 2.1: Comparison of the platform with related products and reference architectures

ASPECT	**YOUTUBE / VIMEO**	**NETFLIX (COSMOS)**	**AWS VOD SOLUTION**	**THIS PROJECT**
Kind	Public products, closed source	In-house platform	Reference architecture	Platform and pipeline, open repository
Users	Viewers and creators	Subscribers; titles from studios	Not a product	Viewers, creators and staff
Orchestration	Not documented	Workflow tier	AWS Step Functions	AWS Step Functions
Encoding	Not documented	Distributed, chunk-based	AWS Elemental MediaConvert	FFmpeg on Batch with Fargate; chunks for long videos
Bitrate ladder	Not documented	Per title and per shot	Job template	Fixed six-rung Capped-CRF ladder bounded by the source
Protection	Not documented	DRM	Signed URLs or cookies	AES-128 with signed CDN requests
Cost model	Free to creators	Private	Per output minute	Per virtual CPU (vCPU) second of real work

The comparison shows where the project stands. It offers the community features of YouTube and Vimeo, which neither the Netflix platform nor the AWS solution addresses, and it takes its processing architecture from the latter two. Its limits are equally visible: a fixed ladder instead of per-title optimisation, and AES-128 encryption instead of DRM, both discussed again in Chapter 7.

2.2 Core Models and Principles

2.2.1 Serverless Containers

A serverless container runs a container image without the developer provisioning or managing the virtual machines underneath, and is billed for the time it runs rather than for capacity kept available. AWS Fargate provides this model, and AWS Batch can schedule jobs onto it through compute environments of type FARGATE or FARGATE_SPOT [11]. Spot capacity uses spare AWS capacity at a lower price and may be reclaimed; this project therefore places a Spot environment first and an On-Demand environment second behind the same job queue, so that jobs run on Spot whenever it is available and overflow to On-Demand otherwise.

Figure 2.1 contrasts this model with an always-on server for a workload of three hundred five-minute videos per month. Measured on the final configuration, a 1080p video costs about 5.5 vCPU-seconds of encoding work per second of source, so the month needs about 138 vCPU-hours of real work. An always-on instance with eight vCPUs, the size of one encoding job, is billed for all 5,840 vCPU-hours of the month and is busy for about 2.4 percent of them. Its capacity is also a hard ceiling: a burst larger than one instance can handle must wait. With serverless containers, no compute exists between uploads, a container starts for each job and stops when the job ends, and a burst is absorbed by the queue up to the configured maximum number of vCPUs. Priced at the On-Demand rate, the same month costs about 8.47 USD instead of 286.16 USD; Section 6.7 gives the full calculation.

A function platform such as AWS Lambda is the more common serverless choice, so its rejection for encoding needs a reason. A Lambda invocation can run for at most fifteen minutes [12], while encoding a long video takes far longer, and a measurement study of serverless video processing found that the cost-efficient memory setting depends on the task and even on the content, with differences of up to 498 percent between settings [7]. Fargate lets vCPU and memory be chosen explicitly for each job. Lambda remains the right tool for the short steps of the pipeline, such as reading the source metadata, starting

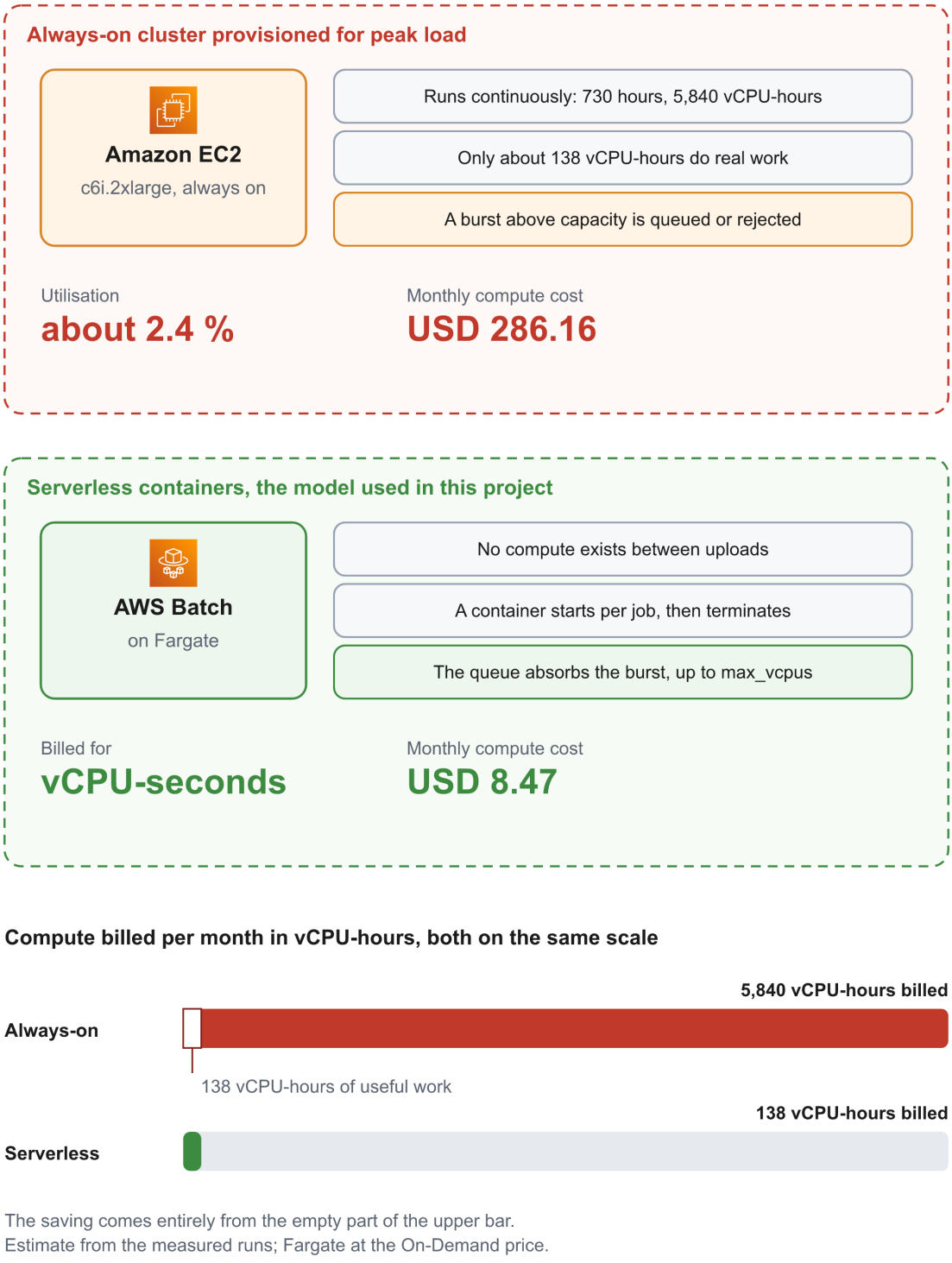


Figure 2.1: *Compute models for a bursty transcoding workload: an always-on instance compared with on-demand serverless containers*

the workflow or writing the final playlists, and the project uses it there. The known weakness of containers is start-up latency; a systematic review reports that pulling the image dominates the cold start of container-based functions [13], which motivates the small multi-stage image described in Chapter 5.

2.2.2 Event-Driven Architecture and Workflow Orchestration

In an event-driven architecture, work moves through the emission and consumption of events rather than through direct calls. The AWS Serverless Applications Lens describes such systems as event sources, routers and destinations connected by durable queues and explicit data contracts [14]. In this project, the event source is Amazon S3: when an upload completes, S3 emits a notification that is placed on an Amazon Simple Queue Service (SQS) queue. The queue absorbs bursts, because a hundred simultaneous uploads become a hundred messages that are consumed as fast as the downstream steps allow, and a dead-letter queue isolates a message that keeps failing instead of retrying it forever. Because a queue delivers each message at least once, never exactly once, every consumer must make a repeated delivery harmless.

A queue alone does not say what happens to a video after the first step. AWS Step Functions fills this gap: it runs state machines whose states are tasks, choices, parallel branches and maps, and a Standard workflow can run for up to one year while keeping the history of every step [15]. Its integration with AWS Batch can submit a job and wait until the job finishes [16], so the workflow, not the container, decides what runs next, how often a failed step is retried and what happens when retries are exhausted. AWS Batch array jobs complete the picture for long videos: one submission creates between 2 and 10,000 child jobs that share a definition and differ only by their index [17], which is how one video is split into many chunks without one call per chunk. Chapter 4 draws the resulting state machine.

2.2.3 HTTP Live Streaming and Adaptive Bitrate

HLS delivers a video as a master playlist that refers to several media playlists, each listing the segments of one rendition [1]. For each rendition the master playlist declares a `BANDWIDTH`, the peak bit rate the client must sustain, and optionally an `AVERAGE-BANDWIDTH`. Apple's authoring specification, which most deployments follow, asks for segments of nominally six seconds, for every video segment to start with an IDR frame, and for `BANDWIDTH` to be the largest peak segment bit rate [18].

Figure 2.2 shows the artefacts produced from one source and the order in which a

player retrieves them. The player first fetches the master playlist, a few hundred bytes, chooses a rendition from the declared bandwidths and its own throughput estimate, and only then requests that rendition's playlist and segments. Because every rendition is cut on the same boundaries and each segment starts with a key frame, the player can change rendition at any segment boundary without interrupting playback. The player used here, hls.js, estimates throughput as an exponentially weighted moving average of segment downloads and scales the estimate down by a safety factor before choosing [19]. For the transcoding side, the consequence is that the work grows with the number of renditions, since each one is encoded separately even though the source is decoded only once.

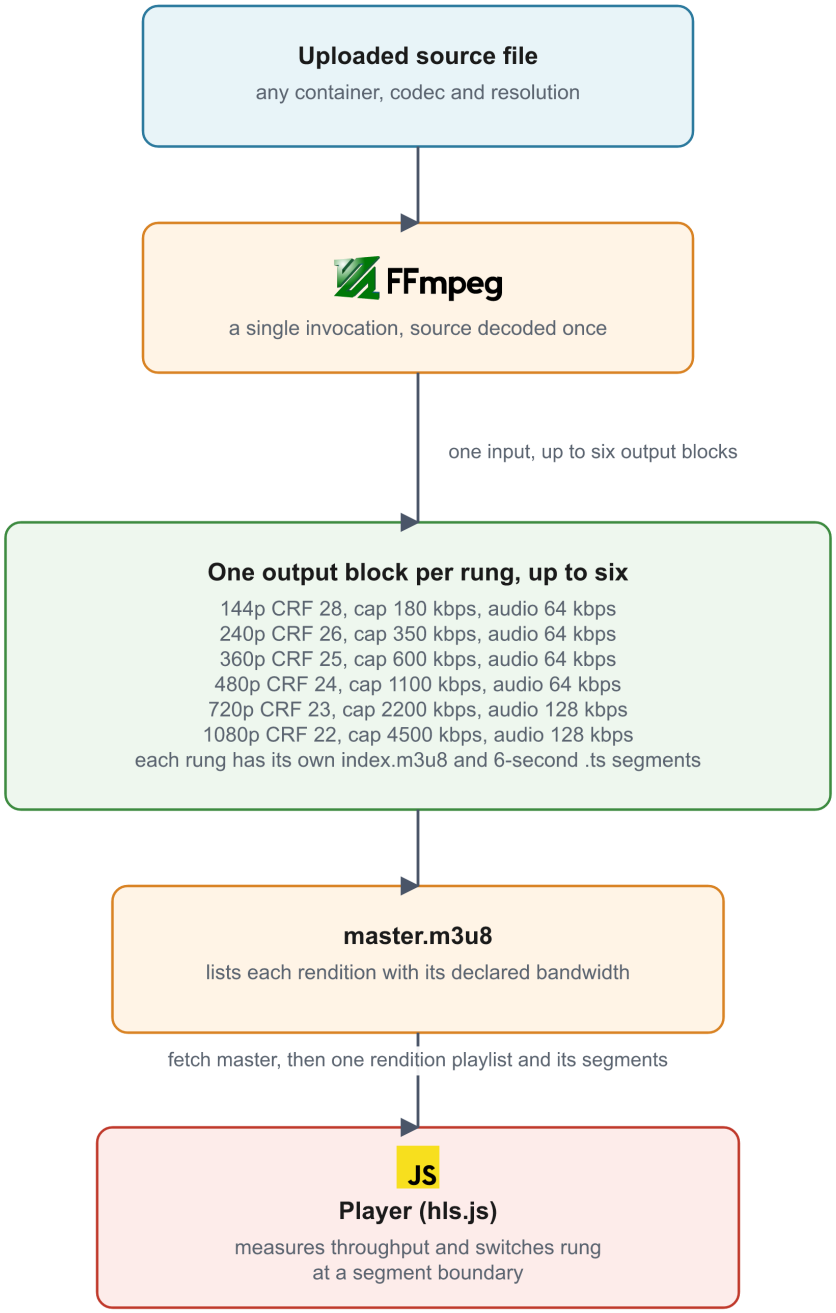


Figure 2.2: *HLS artefacts produced from one source file and the order in which a player retrieves them*

2.2.4 The Bitrate Ladder and Capped CRF

The set of resolution and bit rate pairs offered is called the bitrate ladder. Netflix showed that the best ladder depends on the content and that the ideal operating points lie on the convex hull of the quality against bit rate curve [3]; encoding each shot separately under

a perceptual quality metric reduced bit rates by 28 to 38 percent at the same quality in its measurements [20]. Such optimisation requires many trial encodes per title, which is beyond the budget of this project.

This project instead combines a fixed ladder with quality-targeted encoding. The x264 encoder used through FFmpeg [5] can work at a constant rate factor (CRF), which keeps perceptual quality roughly constant and lets the bit rate rise and fall with the complexity of each scene; Vimeo's recommendation to upload at CRF 18 or below rests on the same idea [9]. Pure CRF has no upper bound, however, which would make the `BANDWIDTH` of a rendition unpredictable. Adding a maximum rate and a decoder buffer size, the video buffering verifier (VBV), caps the peak while CRF still controls quality below the cap. This combination, called Capped CRF, gives static scenes fewer bits and complex scenes more, up to a known limit per rung. Each video receives only the rungs its source resolution can fill, from 144p to 1080p, so nothing is upscaled. Chapter 4 lists the rungs and their parameters.

2.3 Technology Stack Justification

Table 2.2 lists the technology chosen for each part of the system, the main alternative considered and the reason for the choice; the paragraphs after it explain the less obvious decisions.

Table 2.2: Technology choices, alternatives and reasons

PART	CHOICE	ALTERNATIVE	REASON
Web client	React 19, Vite, TypeScript	Server-rendered pages	Rich interactive pages (player, studio, admin) as one static bundle on the CDN
Player	hls.js	Native HLS	Works in every browser with Media Source Extensions and can add signed parameters to each request
API	Express 5 on one AWS Lambda function	API Gateway with one function per route	One cold-start profile, no API Gateway cost; scales per request
API edge	Cloudflare Worker	Amazon API Gateway	The domain's Domain Name System (DNS) and Transport Layer Security (TLS) are already on Cloudflare; the Worker rewrites the host and adds an origin secret
Orchestration	AWS Step Functions	Containers that chain jobs themselves	One place for state, retries and failure handling, with a visible history
Encoding	FFmpeg on AWS Batch with Fargate	AWS Elemental MediaConvert	Full control of the encoder, billed per vCPU-second
Documents	MongoDB Atlas	Relational database	Document-shaped data whose schema grows with features; managed service
Search	Atlas Search	OpenSearch cluster	Full-text search inside the same database, no cluster to run
Hot counters	Amazon DynamoDB	MongoDB	Pay per request, built-in expiry; keeps frequent writes off the free database tier
Delivery	Amazon CloudFront	Serving from S3	Edge caching and signed access to a private bucket
Infrastructure	Terraform	AWS CloudFormation	Modules with explicit inputs and a plan step that shows each change; covers both AWS and Cloudflare
Checks	GitHub Actions	Separate CI server	Source already on GitHub

The API is written in TypeScript on Node.js 22. Its work is almost entirely input and output: checking a token, reading a document, signing a uniform resource locator (URL). The heavy computation is delegated to the containers, so the event loop of Node.js suits it, and one language across the API, the Lambda steps, the transcoder and the web client lets them share a single package of schemas that both the producer and the consumer of each event validate. Running the API as one Express application inside one Lambda function keeps the code of an ordinary web server while removing the server: AWS starts as many copies as there are concurrent requests and none when there are none.

MongoDB Atlas holds the documents: users, channels, videos, comments, playlists and moderation records. Its search feature provides full-text search embedded in the same database [21], which avoids running a separate search cluster. The project uses the free tier of Atlas, whose limit of one hundred operations per second would be reached by data written on every few seconds of playback, such as watch progress, view marks and rate-limit counters. These are therefore kept in Amazon DynamoDB, billed per request and able to expire entries on its own.

Playback uses hls.js rather than the browser's native HLS support even in browsers that have it. Access to the processed files is granted by a signed policy that must be appended to every playlist and segment request, and only hls.js lets the application change each request before it is sent. Chapter 4 explains this access scheme in detail.

Chapter 3

Requirements Analysis

This chapter describes who uses the platform and what each group needs, states the functional requirements (FR) and non-functional requirements (NFR) that follow from those needs, and models them as use cases (UC), each specified step by step and checked against the routes of the API that implement it.

3.1 Stakeholders and User Requirements

The platform serves people who watch videos, people who publish them, and staff who keep the content within the rules. Table 3.1 lists each group, what it needs and the requirements of Sections 3.2 and 3.3 that answer those needs.

Table 3.1: Stakeholders, their needs and the requirements that serve them

STAKEHOLDER	NEEDS	REQUIREMENTS
Anonymous visitor	Find and watch public videos at once, without an account, on any connection, in their language and a light or dark theme	FR-05, FR-06, FR-09, FR-17, NFR-01
Viewer (registered user)	Take part: react, comment, subscribe, join memberships, keep a library, receive notifications	FR-01, FR-02, FR-10 to FR-14
Creator (registered user)	Upload large files without waiting, decide who sees each video, edit and schedule it, earn members, read analytics	FR-03, FR-04, FR-06 to FR-08, FR-13, FR-15, FR-16, NFR-02, NFR-03
Operator	A queue of what the screening held back or viewers reported, and a way to decide with a reason and resolve appeals	FR-18 to FR-21
Auditor	The error rates of the screening with their uncertainty, and a decision log that cannot be altered	FR-22
Administrator	Assign staff roles and draw random audit samples	FR-22, FR-23
Development team	A system that costs almost nothing while idle, is reproducible from code, observable and safe to change	NFR-04 to NFR-10

The anonymous visitor matters because asking for registration before any content can be seen would deter new viewers, so the platform asks for an identity only where an action needs one: reacting, commenting, subscribing, uploading. The viewer and the creator are the same kind of account seen from two sides. Every registered user can watch and take part, and becomes a creator by uploading; there is no separate creator role, as on YouTube. The creator's central need is that a large upload neither blocks the browser nor waits on a server, and that the video becomes playable soon after the upload ends.

The three staff roles are kept apart on purpose. An operator decides on individual videos; an auditor reads how often the automatic screening was wrong but cannot change a decision, so the person who reads the error rates is not the person whose decisions produce them; an administrator manages roles and draws the random samples that measure what the screening missed. An administrator may also act as an operator or an auditor. The development team is listed as a stakeholder because several requirements, such as idle cost and reproducibility, serve the people who run the platform rather than its users.

3.2 Functional Requirements

Table 3.2 lists the functional requirements of the platform. Each has an identifier so that it can be traced through the use cases, the design and the evaluation.

Table 3.2: Functional requirements of accounts, uploading, processing and watching

ID	REQUIREMENT	DESCRIPTION
FR-01	Accounts	Register with a display name, an e-mail address and a password, or sign in with Google, which also links to an existing account. A password must have at least eight characters and is refused when it is common, contains the user's own details or appears in a known breach. A new account must confirm its address before it may upload. Signing out ends every session of the account.
FR-02	Password recovery and change	A reset link by e-mail, valid for one hour and usable once; a change from the settings page that requires the current password.
FR-03	Direct upload	The browser sends the file straight to Amazon S3: in one request up to 5 GiB, and in parts of 32 MiB up to 20 GiB, with progress, speed, remaining time and cancel.
FR-04	Automatic processing	Without any manual step, the source becomes an encrypted HLS package with up to six renditions from 144p to 1080p bounded by the source, a thumbnail and seek-preview images; long videos are split into chunks encoded in parallel, and the owner sees the progress of each step.
FR-05	Playback and sharing	An adaptive player with manual quality, speed, picture-in-picture, full screen, captions, chapters, seek preview and resume from the last position. A link can be shared with a start time, and public and unlisted videos can be embedded in another site.
FR-06	Visibility and audience	Each video is public, unlisted, private or password-protected, and may be limited to channel members from a chosen tier upward. A stranger learns nothing about a private video, not even that it exists.
FR-07	Creator studio	The owner edits the title, description, tags, category and thumbnail, uploads WebVTT captions, defines chapters and deletes a video with all its files.
FR-08	Scheduled publishing	The owner chooses a publication time between five minutes and one year ahead; the video becomes public at that time.
FR-09	Discovery	A home feed of the newest videos, full-text search over videos and channels that tolerates typing errors and missing diacritics, suggestions while typing, categories, related videos, and a page per channel that lists its videos.

Table 3.3 continues with the requirements of the community, the library and the creator's channel.

Table 3.3: Functional requirements of the community, the library and the channel

ID	REQUIREMENT	DESCRIPTION
FR-10	Community	Likes and dislikes, comments of up to 1000 characters with one level of replies; the creator pins one comment; authors and the creator delete comments.
FR-11	View counting	A view counts after thirty seconds of playback, or the whole of a shorter video, at most once per viewer and video in thirty minutes, and never for the owner.
FR-12	Subscriptions	Subscribe to a channel, see a feed of subscribed channels, and receive an in-app notification, and an e-mail unless turned off, when a subscribed channel publishes.
FR-13	Memberships	A channel offers up to five tiers; a viewer joins one for thirty days through a sandbox payment and may cancel; members get a badge and the videos limited to their tier.
FR-14	Library	Watch history that can be cleared, a watch-later list, and ordered playlists that are public, unlisted or private.
FR-15	Analytics	Views per day, watch time and audience retention per video and per channel.
FR-16	Channel profile	A unique handle, a description, an avatar and a banner, both screened before they are shown.
FR-17	Interface	Light and dark themes, Vietnamese and English, and dedicated pages for errors such as not found or forbidden.

Table 3.4 continues the list with the requirements of moderation and operation.

Table 3.4: Functional requirements of moderation and operation

ID	REQUIREMENT	DESCRIPTION
FR-18	Automatic screening	Before a video is published, sampled frames are classified; clearly violating content is removed, doubtful content is held for review, and violent content that may stay is shown behind a warning. When screening cannot run, the video is held, never published.
FR-19	Reports	A signed-in viewer reports a video or a comment for one of eight reasons, with at most one open report per item.
FR-20	Staff review	Operators approve or remove held and reported videos, and dismiss or remove reported comments; every video decision carries a reason code from a closed list.
FR-21	Appeals	The owner of a removed or held video appeals with a message; an operator upholds or rejects the appeal with a note.
FR-22	Accountability	Each decision is logged with the verdict the machine had reached, so that the error rates of the screening can be estimated: the share of automatic removals reversed on review, and the share of machine approvals that fail a random audit, each with a confidence interval; an auditor reads them but cannot decide.
FR-23	Role management	An administrator assigns the roles user, operator, auditor and administrator; the last administrator cannot be demoted, and every change is logged.

Requirement FR-03 shapes the architecture more than any other. Routing a multi-gigabyte upload through the API would make the API pay in memory, bandwidth and execution time for every byte, and a serverless function cannot even run long enough to receive a large file. Instead the API returns a short-lived grant, and the browser sends the file directly to Amazon S3; the API handles only small JSON requests whatever the size of the file. FR-04 follows from it: because the file reaches S3 without passing through the API, it is the arrival of the file in S3, not a call from the browser, that starts the processing.

3.3 Non-Functional Requirements

The non-functional requirements are listed in Table 3.5. Where a quality can be measured, the requirement states a target, and Section 6.8 sets each target against the measurements of Chapter 6.

Table 3.5: Non-functional requirements of the system

ID	ATTRIBUTE	REQUIREMENT
NFR-01	Playback start	Through the CDN, the first frame appears within one second at the 95th percentile once the files are cached, and playback starts within three seconds on an unthrottled connection without stalling.
NFR-02	Processing time	A video of up to ten minutes is ready within three minutes after its upload completes, and a four-hour video within one hour.
NFR-03	Burst load	One hundred simultaneous uploads lose no work: every upload the API accepts reaches READY, and an upload refused under overload receives a status the client can retry.
NFR-04	Idle cost	No transcoding container runs and no compute capacity is reserved or billed while nothing is being processed; functions that run on a schedule are billed per invocation.
NFR-05	API latency	A warm API answers within 300 ms at the median and serves 80 concurrent requests without errors.
NFR-06	Content security	Segments, playlists and keys of a video are unreachable without a grant for that video, even to someone who knows their addresses.
NFR-07	API security	Sign-in and other sensitive routes are rate-limited, only the web origins of the platform may call the API from a browser, and the function behind the API answers only requests that came through its gateway.
NFR-08	Reliability	A failed processing step is retried when a retry can succeed; otherwise the video is marked failed with a reason and an alert is sent. No video stays in processing for more than a day.
NFR-09	Reproducibility	Every environment is created from the same infrastructure code, with no manual changes.
NFR-10	Maintainability	A change reaches the main branch only after linting, type checks, tests and security scans pass.

NFR-06 goes beyond ordinary access control. Restricting what the API returns is straightforward, but the API does not deliver the video: the content delivery network delivers it as hundreds of small files, and a single leaked address would otherwise be enough to download them. The design of Chapter 4 therefore protects the files themselves, not only the API. NFR-03 is phrased around accepted work because a burst can exceed what a free database tier and a fixed concurrency limit absorb at once; what must never happen is that an accepted upload silently disappears.

3.4 Use Case Model

3.4.1 Use Case Diagrams

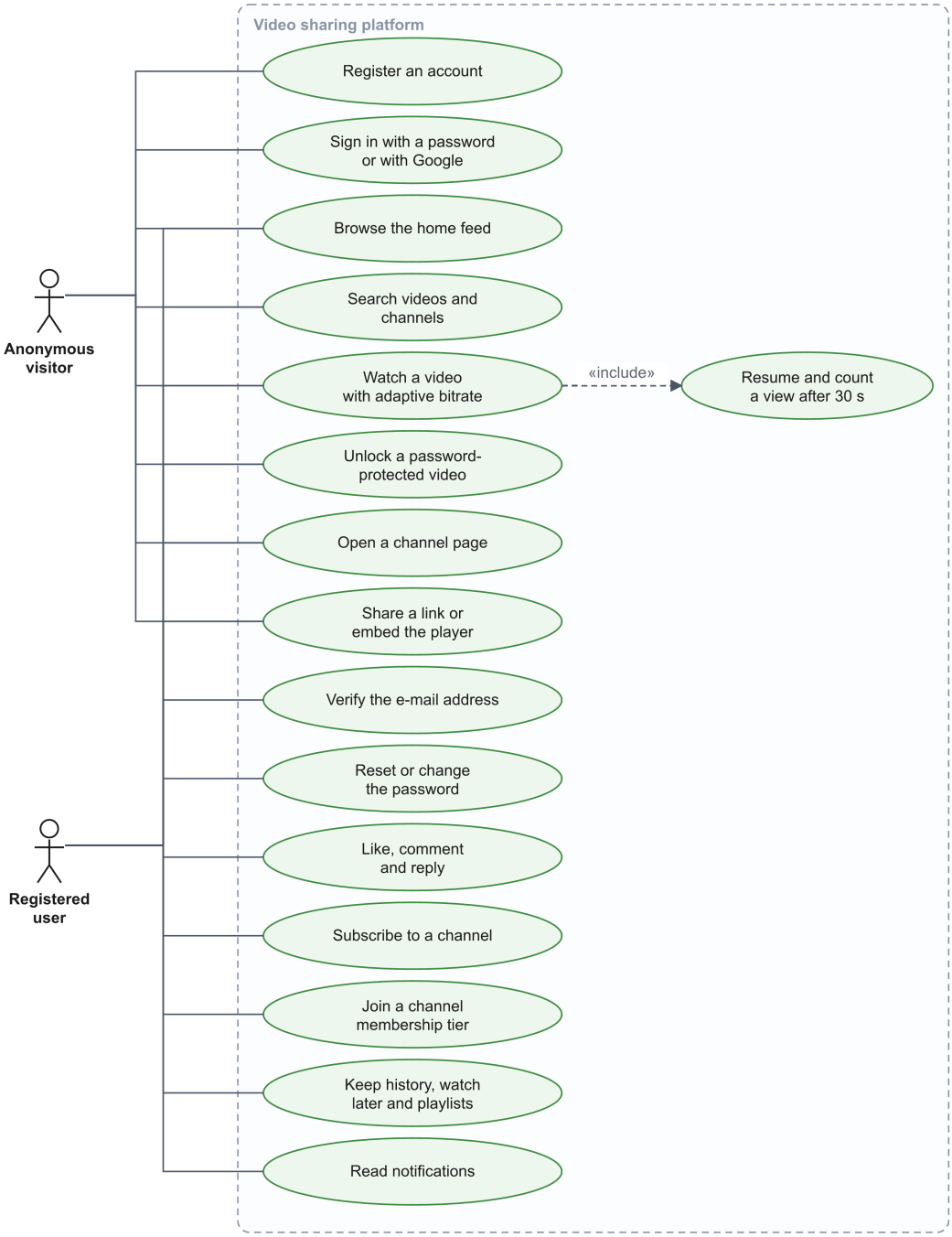
The use cases are drawn in three diagrams, because one diagram with all of them would be taller than a page. Figure 3.1 shows those of viewers. The anonymous visitor can register, sign in, browse, search, watch, unlock a password-protected video, open a channel and share; a registered user can do all of this and also take part in the community. Watching includes resuming from the last position and counting a view, which the player does without any action from the viewer.

Figure 3.2 shows the creator, who is the same registered user acting on their own channel, and the processing system. The processing system is drawn as an actor because events start its use cases, not a person: the upload reaching storage starts transcoding, the scheduled time starts publication, and a published video starts the notifications to subscribers.

Figure 3.3 adds moderation and operation. Three things lead to a decision that only staff can take: what the screening holds back, a report from a viewer, and an appeal from the owner of a removed video. Each staff use case is drawn once, at the role that owns it.

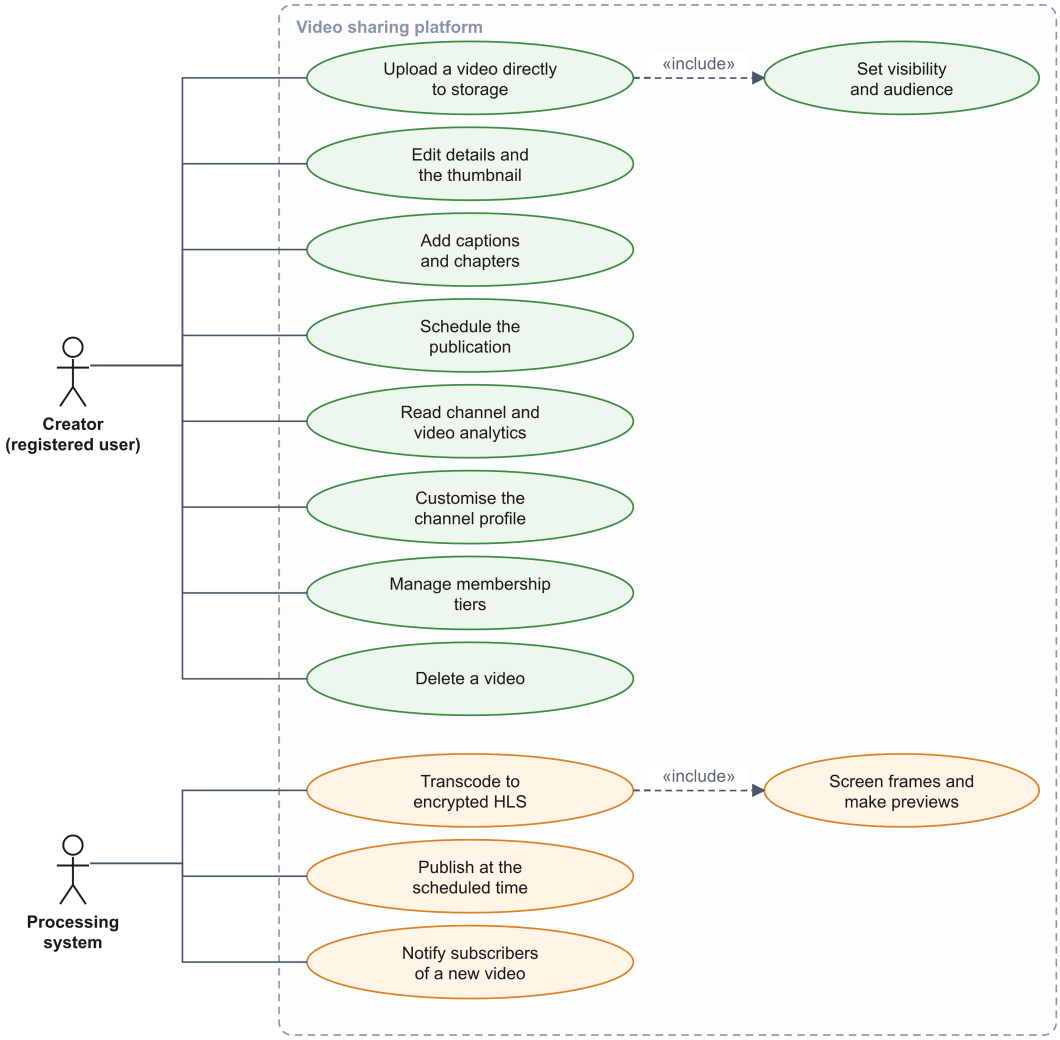
3.4.2 Use Case Specifications

The use cases are specified in Tables 3.6 to 3.25. Closely related use cases of the diagrams are specified together, for example liking, commenting and replying, so that twenty specifications cover the diagrams. Each lists its actor, the requirements it serves, the main flow and the alternative flows, numbered after the step they branch from.



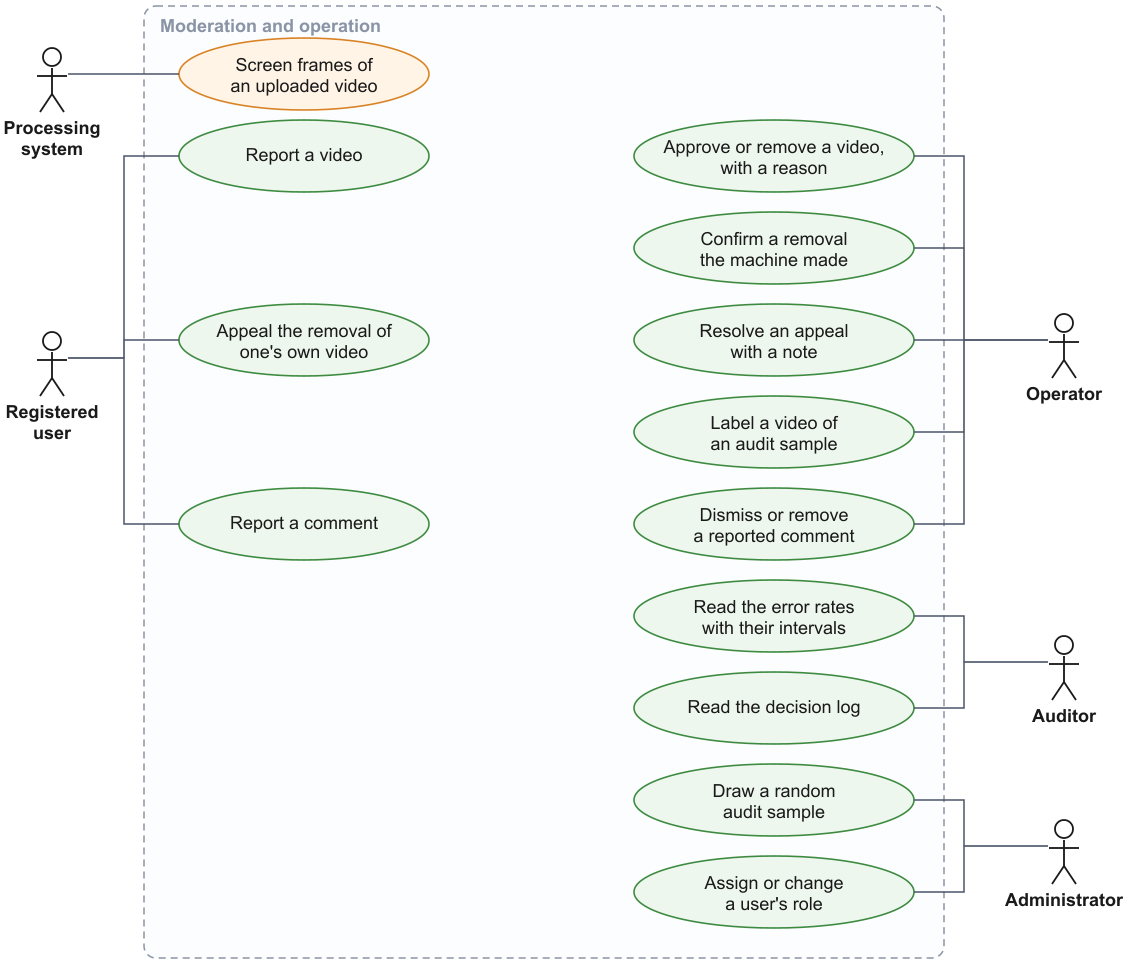
Association lines carry no arrowhead, as UML specifies; only «include» is directed. A signed-in user can do everything a visitor can. Uploading and the creator studio are in Figure 3.2, moderation in Figure 3.3.

Figure 3.1: *Use case diagram of viewers*



The processing use cases attach to a system actor because events start them: the upload reaching storage, the scheduled time, or a video being published. The creator is the same registered user as in Figure 3.1.

Figure 3.2: *Use case diagram of creators and the processing system*



Reporting and appealing are open to every signed-in user. An administrator can also do everything an operator or an auditor can, so each association is drawn once, at the role that owns the use case. What screening holds back, what viewers report and what owners appeal reaches an operator's queue.

Figure 3.3: *Use case diagram of moderation and operation*

Table 3.6: Use case UC-01: register and verify the e-mail address

FIELD	**CONTENT**
Actor	Anonymous visitor
Requirements	FR-01
Precondition	No account uses the chosen e-mail address.
Main flow	1. The visitor enters a display name, an e-mail address, a password and optionally a handle. 2. The server checks the password against the length rule, a list of common passwords, the visitor's own details and a breach service. 3. The server creates an unverified account, generating a handle from the address when none was given, and returns a session token. 4. The server e-mails a link carrying a token valid for 24 hours, of which only the hash is stored. 5. The visitor opens the link, and the server marks the address verified and spends the token.
Alternative flows	1a. Five registrations from the same network address within an hour: the server answers 429. 2a. The password is refused: the server answers 400 with the reason, which the form shows. 3a. The address or the requested handle is taken: the server answers 409. 4a. Three links already went to the address this hour: the account is created and the mail is skipped. 5a. The link is unknown, used or expired: the server answers 400, and the user can ask for a new link.
Postcondition	The account exists; once the address is verified, it may upload.

Signing in, with a password or with Google, is specified in Table 3.7.

Table 3.7: Use case UC-02: sign in with a password or with Google

FIELD	**CONTENT**
Actor	Anonymous visitor who has, or will get, an account
Requirements	FR-01, NFR-07
Precondition	None.
Main flow	1. The visitor enters the e-mail address and the password. 2. The server compares the password with the stored hash. 3. The server returns a session token, which the web client sends in the `Authorization` header of later requests.
Alternative flows	1a. The visitor signs in with Google: the server verifies the Google identity token; a known Google account signs in, an unknown address creates a verified account, an unverified local account with the same address is taken over and its password removed, and a verified local account is refused with 409, so that its owner links Google from the settings instead. 1b. More than ten attempts for one address, or twenty from one network address, in fifteen minutes: the server answers 429. 2a. The address or the password is wrong: the server answers 401 with the same message for both. 2b. The account is disabled: the server answers 403.
Postcondition	The visitor holds a session token.

Table 3.8 covers a forgotten password and a change of password; the reply to a reset request never reveals whether an address has an account.

Table 3.8: Use case UC-03: reset or change the password

FIELD	CONTENT
Actor	Registered user
Requirements	FR-02
Precondition	None for a reset; a session for a change.
Main flow	1. The user who forgot the password enters the e-mail address. 2. The server gives the same answer whether or not an account exists; if one does, it e-mails a link valid for one hour. 3. The user opens the link and enters a new password. 4. The server checks the password before spending the token, stores the new hash, marks the address verified, ends every older session and returns a new session token.
Alternative flows	1a. A signed-in user changes the password from the settings page: the server requires the current password, unless the account only signs in with Google, and then continues at step 4. 4a. The token is unknown, used or expired: the server answers 400 and the password is unchanged. 4b. The new password is refused: the server answers 400 and the link remains usable.
Postcondition	The password is changed, and tokens issued before the change are no longer accepted.

Browsing and searching (Table 3.9) and watching (Table 3.10) are open to every visitor.

Table 3.9: Use case UC-04: browse and search

FIELD	CONTENT
Actor	Anonymous visitor or registered user
Requirements	FR-09, FR-16
Precondition	None.
Main flow	1. The visitor opens the home page, which lists, newest first, videos that are ready, public and approved by moderation. 2. As the visitor scrolls, the page loads the next page through a cursor, so the list never repeats or skips a video. 3. The visitor types in the search box and receives suggestions after two characters. 4. The visitor submits the query, optionally within a category; the server returns matching videos by relevance and, on the first page, up to three matching channels. 5. The visitor opens a channel page, which shows the profile, the subscriber count and the channel's videos, newest first.
Alternative flows	4a. Nothing matches: the page shows an empty result. 4b. More than sixty searches in a minute: the server answers 429.
Postcondition	None; nothing is changed.

Table 3.10: Use case UC-05: watch a video

FIELD	CONTENT
Actor	Viewer, signed in or not
Requirements	FR-05, FR-06, FR-11, NFR-01, NFR-06
Precondition	The viewer has the link of the video.
Main flow	1. The viewer opens the watch page, and the server returns the video after applying the access rule. 2. The page asks for playback and receives the master playlist address, a signed CDN policy for this video and a playback token, both valid for two hours. 3. The player loads the playlists and segments from the CDN with the policy, and fetches the decryption key from the API with the playback token. 4. The player switches renditions as the bandwidth changes; a signed-in viewer resumes from the last position, which the player saves every minute and on pause. 5. After thirty seconds of playback, or the whole of a shorter video, the page reports a view with its playback token, which the server counts only if that much time has passed since the token was issued, and not if the same viewer was counted within thirty minutes or the viewer owns the video. 6. The viewer copies the link, optionally starting at the current time, or the code that embeds the player in another site.
Alternative flows	1a. The video is private or not ready, and the viewer is not its owner: the server answers 404. 1b. The video was removed or is under review: the server answers 403 with a distinct code, and the page explains it. 1c. The video is password-protected: the page shows only a password form (UC-06). 2a. The video is for members and the viewer is not one of a high enough tier: the page shows, instead of the player, a card that leads to the channel's tiers. 3a. The grant expires during playback: the player asks for a new one and continues. 6a. In an embedded player, a private, password-protected or members-only video shows a card that leads to the site instead of playing.
Postcondition	The view count may have grown by one.

Table 3.11 specifies how a password-protected video is opened.

Table 3.11: Use case UC-06: unlock a password-protected video

FIELD	CONTENT
Actor	Viewer, signed in or not
Requirements	FR-06
Precondition	The video is ready and password-protected.
Main flow	1. The page shows a password form, without the title or any other detail of the video. 2. The viewer enters the password, and the server compares it with the stored hash. 3. The server returns an unlock token for this video, and the page continues with UC-05.
Alternative flows	2a. The password is wrong: the server answers 403, still without revealing the title. 2b. More than ten attempts from one network address in fifteen minutes: the server answers 429.
Postcondition	The viewer may watch the video while the unlock token is valid.

The interactions of a signed-in viewer are specified in Tables 3.12 to 3.15: reacting and commenting, subscribing (Table 3.13), joining a membership (Table 3.14) and keeping a library.

Table 3.12: Use case UC-07: like, comment and reply

FIELD	CONTENT
Actor	Registered user
Requirements	FR-10
Precondition	The user is signed in and may play the video.
Main flow	1. The user presses like or dislike; the server stores the one reaction of this user and recounts both totals from the stored reactions. 2. The user writes a comment of at most 1000 characters, or a reply to a comment. 3. The server stores it, a reply always under the top-level comment, and recounts the comments of the video and the replies of the thread. 4. The creator of the video pins one top-level comment, which then appears first.
Alternative flows	1a. Pressing the same reaction again removes it. 2a. More than ten comments in a minute: the server answers 429. 3a. The comment replied to was deleted: the server answers 400. 4a. Someone other than the creator tries to pin: the server answers 403. 4b. The author, the creator or an operator deletes a comment, and its replies go with it; anyone else receives 403.
Postcondition	Reactions, comments and their counts are up to date.

Table 3.13: Use case UC-08: subscribe to a channel and read notifications

FIELD	CONTENT
Actor	Registered user
Requirements	FR-12
Precondition	The user is signed in.
Main flow	1. The user presses Subscribe on a channel page or below a video. 2. The server records the subscription and recounts the subscribers of the channel. 3. The subscriptions page lists the newest videos of every subscribed channel. 4. When a subscribed channel publishes a video, the user receives an in-app notification, and also an e-mail unless the user has turned e-mails off. 5. The user opens the notifications and marks them read.
Alternative flows	1a. The channel is the user's own: the server answers 400. 1b. Pressing again unsubscribes, and the count is recomputed.
Postcondition	The subscription and the subscriber count agree.

Table 3.14: Use case UC-09: join a channel membership

FIELD	CONTENT
Actor	Registered user
Requirements	FR-13
Precondition	The user is signed in and the channel offers at least one tier.
Main flow	1. The user opens the channel's tiers, each with a name, price, level and perks. 2. The user chooses a tier and confirms the sandbox payment. 3. The server records a membership at that tier's level for thirty days. 4. The user gets the member badge and the videos limited to that level or below.
Alternative flows	2a. The channel is the user's own: the server answers 400. 3a. The user is already a member: the tier changes and the end date stays. 4a. The user cancels: the membership stays active until its end date and is not renewed.
Postcondition	The membership is active until its end date.

Table 3.15: Use case UC-10: keep a library

FIELD	**CONTENT**
Actor	Registered user
Requirements	FR-14
Precondition	The user is signed in.
Main flow	1. Watching a video adds it to the user's history with the last position. 2. The user adds a video to Watch later, a playlist that every account has. 3. The user creates a playlist with a title, a description and a visibility, adds videos and changes their order. 4. The user opens the library to see the history, Watch later and the playlists.
Alternative flows	1a. The user removes one entry or clears the whole history. 3a. A video that becomes private or removed disappears from what others see of a playlist. 4a. Another person opens a private playlist: the server answers 404.
Postcondition	The library reflects the user's changes.

Uploading, the use case that shapes the architecture most, is specified in Table 3.16, and the other work of a creator in Tables 3.17 to 3.19, among them scheduled publication (Table 3.18).

Table 3.16: Use case UC-11: upload a video

FIELD	CONTENT
Actor	Creator
Requirements	FR-03, FR-04, FR-06
Precondition	The creator is signed in with a verified address.
Main flow	1. The creator selects a file and chooses the title, description and visibility. 2. The client sends the name, size and type of the file; the server creates the video in state UPLOADING and returns a grant valid for fifteen minutes that fixes the storage key, the size and the metadata. 3. The browser sends the file directly to Amazon S3 and shows progress, speed and remaining time. 4. The arrival of the file in S3 starts processing (UC-15); the client's completion call only records when the upload ended. 5. The studio shows the video with the current step and progress until it is ready.
Alternative flows	2a. The address is not verified: the server answers 403, and the page offers to send the link again. 2b. The file is larger than 20 GiB or not a supported video type: the server answers 400 before creating anything. 2c. More than twenty uploads in an hour: the server answers 429. 3a. The file is larger than 5 GiB: the server opens a multipart upload, the browser asks for grants for parts of 32 MiB and sends them, and the completion call assembles the parts from the list that S3 itself keeps, refusing missing or wrong-sized parts. 3b. The creator cancels: the multipart upload, the stored data and the draft are removed.
Postcondition	The video is being processed.

Table 3.17: Use case UC-12: edit and delete a video in the studio

FIELD	CONTENT
Actor	Creator
Requirements	FR-06, FR-07, FR-15
Precondition	The creator is signed in and owns the video.
Main flow	1. The studio lists the creator's videos with status, views and visibility, including those still processing. 2. The creator edits the title, description, tags, category, visibility, audience and minimum tier, and uploads a custom thumbnail. 3. The creator uploads WebVTT captions per language and defines chapters. 4. The creator reads the analytics: views per day, watch time and retention. 5. The creator deletes a video; the server removes the record first, then its comments, reactions and playlist entries, and finally the source, the renditions and the key.
Alternative flows	2a. Switching to password protection without a password: the server answers 400. 2b. Limiting to members without any tier: the server answers 400. 2c. The custom thumbnail is not a JPEG or PNG image of at most 2 MiB, or fails screening: the server refuses it. 3a. The chapters break the rules, which ask for at least three, the first at 0:00 and each at least ten seconds long: the server answers 400. 5a. The video is still uploading: the upload is aborted instead.
Postcondition	The watch page and the listings reflect the change.

Table 3.18: Use case UC-13: schedule the publication

FIELD	CONTENT
Actors	Creator; processing system
Requirements	FR-08
Precondition	The creator owns the video, and it is not yet public.
Main flow	1. The creator chooses a date and time. 2. The server creates a one-time schedule for that moment; a video already processed becomes SCHEDULED. 3. At that moment the schedule fires, and the video becomes public and ready, which notifies the subscribers.
Alternative flows	1a. The time is less than five minutes or more than a year ahead: the server answers 400. 2a. The video failed processing or is already public: the server answers 409. 3a. The schedule fires before processing ends: nothing happens, and the video is published as soon as it is ready. 3b. The creator cancels: the schedule is deleted and the video stays private.
Postcondition	The video is published at the chosen time, or stays private.

Table 3.19: Use case UC-14: customise the channel and manage membership tiers

FIELD	**CONTENT**
Actor	Creator
Requirements	FR-13, FR-16
Precondition	The creator is signed in.
Main flow	1. The creator sets the handle and the description of the channel. 2. The creator uploads an avatar or a banner, a JPEG or PNG image of up to 2 MiB, which the server screens with the policy of the videos before it replaces the current one. 3. The creator creates a tier with a name, price, level, perks and badge colour. 4. The creator edits or retires a tier. 5. The creator reads the list of current members with their tier and end date.
Alternative flows	1a. The handle is reserved or already taken: the server answers 409. 2a. The image is too large or of another type: the server answers 400. 2b. The image breaks the content rules: the server answers 403 and the current image stays. 3a. The channel already has five tiers: the server answers 409.
Postcondition	The channel page shows the profile and offers the tiers.

Table 3.20 specifies the use case of the processing system, which no person starts.

Table 3.20: Use case UC-15: transcode a video automatically

FIELD	CONTENT
Actor	Processing system
Requirements	FR-04, FR-18, NFR-02, NFR-03, NFR-08
Precondition	A source file has arrived in the raw bucket.
Main flow	1. Amazon S3 emits an event, which Amazon SQS buffers. 2. A function reads the metadata of the object and starts a workflow execution named after the video and its upload version. 3. The workflow marks the video as processing, creates its encryption key and probes the source. 4. In parallel, one job screens sampled frames and makes the thumbnail and seek-preview images, while the encoding runs as one job for a video of up to five minutes, or as an audio job and parallel chunk jobs followed by a stitching step for a longer one. 5. The workflow publishes the video as ready, or as scheduled, with the result of the screening, and the subscribers are notified.
Alternative flows	2a. The same event arrives twice: the second execution is refused by name, and nothing more happens. 4a. A job fails for a reason that may pass, such as a reclaimed Spot task: it is retried, up to three attempts. 4b. A job fails for a reason that another attempt cannot fix, such as an unreadable source: the video becomes ERROR with the reason, the owner is told once, and an alert is sent. 4c. The screening cannot run: the video is held for review. 5a. The screening removes the video or holds it: it is not listed, and its owner may appeal.
Postcondition	The video is READY or SCHEDULED with a moderation result, or ERROR.

The remaining use cases belong to moderation. A viewer's report (Table 3.21) and the screening both lead to an operator's decision (Table 3.22).

Table 3.21: Use case UC-16: report a video or a comment

FIELD	CONTENT
Actor	Registered user
Requirements	FR-19
Precondition	The user is signed in and may see the video.
Main flow	1. The user chooses Report on a video or a comment and one of eight reasons, such as sexual, violent, hateful or spam, with optional details. 2. The server records the report and recounts the open reports of the item, which places it in the review queue.
Alternative flows	1a. The item is the user's own: the server answers 400. 1b. More than twenty reports in an hour: the server answers 429. 2a. The user already has an open report on the item: no second report is created.
Postcondition	An open report waits for an operator.

Table 3.22: Use case UC-17: decide on a video or a comment

FIELD	CONTENT
Actor	Operator or administrator
Requirements	FR-20, FR-22
Precondition	The person holds the permission to review, and a video was held by the screening, reported by viewers or removed by the machine.
Main flow	1. The operator opens the review queue of held and reported videos, plays a video and reads its reports grouped by reason. 2. The operator approves or removes the video with a reason code from the closed list of that decision and an optional note. 3. The server keeps the machine's verdict if it was not kept before, applies the decision, closes the open reports and logs the decision with its outcome class. 4. For a reported comment, the operator dismisses the reports or removes the comment and its replies. 5. In the tab of removed videos, the operator confirms a removal the machine made or restores the video, and the decision is logged as a confirmation or a restoration.
Alternative flows	2a. The reason code does not belong to the decision: the server answers 400. 2b. An operator tries to change a decision taken by an administrator: the server answers 403.
Postcondition	The video or comment is shown or hidden accordingly, and the decision is logged.

The owner of a removed or held video can then appeal (Table 3.23).

Table 3.23: Use case UC-18: appeal a decision

FIELD	CONTENT
Actors	Owner of the video; operator
Requirements	FR-21
Precondition	The video is removed or held for review.
Main flow	1. The owner writes an appeal of 10 to 500 characters on the video. 2. The server records it; the owner sees its state on the video. 3. An operator reads the appeal, watches the video, and upholds or rejects it with a note. 4. Upholding approves the video with the reason "appeal upheld"; rejecting keeps the decision.
Alternative flows	1a. The requester does not own the video: the server answers 404. 1b. The video is neither removed nor held: the server answers 409. 2a. An appeal on the video is already open: the server answers 409.
Postcondition	The appeal is resolved, and the owner sees the outcome.

Table 3.24 specifies the random audit that estimates what the screening misses, and Table 3.25 the assignment of roles.

Table 3.24: Use case UC-19: audit a random sample and read the error rates

FIELD	CONTENT
Actors	Administrator; operator; auditor
Requirements	FR-22
Precondition	Each person holds the permission for their step.
Main flow	1. The administrator draws a random sample of 1 to 200 ready videos that the machine alone approved. 2. An operator watches each sampled video and labels it pass or fail. 3. A video labelled fail is removed with the reason “audit failed”. 4. The auditor reads the rates of the screening, each with its 95% Wilson interval and sample size, and the decision log filtered by outcome.
Alternative flows	1a. A video already sampled is not added twice. 2a. The sample was already labelled: the server answers 404.
Postcondition	The share of approvals that failed the audit, the false omission rate of the screening, includes the new labels.

Table 3.25: Use case UC-20: assign a role

FIELD	CONTENT
Actor	Administrator
Requirements	FR-23
Precondition	The person is signed in as an administrator.
Main flow	1. The administrator searches users by the start of their e-mail address or handle. 2. The administrator chooses user, operator, auditor or administrator. 3. The server changes the role, logs who changed whom from which role to which, and forgets the cached session of that user. 4. The change takes effect within 30 seconds, when every instance of the API has read the account again.
Alternative flows	2a. The change would demote the last administrator: the server answers 409.
Postcondition	The role is changed, and the change is logged.

Chapter 4

System Design

This chapter describes the design of the system. It starts with the architecture as a whole, then analyses in detail the two parts that make the platform a video platform: the event-driven workflow that turns an upload into a stream, and the media design of that stream, from the FFmpeg processing graph to the way the player chooses a quality. The remaining sections describe the database, the API, the interface, security and content moderation.

4.1 High-Level Architecture

Figure 4.1 shows the deployed architecture with its real boundaries. Two regions appear. Outside AWS are the viewer's browser, Cloudflare, which holds the domain's DNS and runs the gateway in front of the API, and MongoDB Atlas, which holds the documents and the search indexes. One AWS account holds everything else: compute and storage in the Singapore region, and the two CloudFront distributions that deliver the site and the media from the edge, which read the buckets through Origin Access Control.

The account is divided into four tiers along the path of a video, with delivery beside them. The first tier serves the API. A request from the browser reaches the Cloudflare Worker, which forwards it to the function URL of a single AWS Lambda function running an Express application and adds a secret header, so that the function refuses requests that bypassed the gateway (steps 1 and 2). The function reads and writes documents in MongoDB Atlas (step 3) and keeps data that is written every few seconds, such as rate limits, view marks, watch progress and notifications, in Amazon DynamoDB (step 4), for the reason given in Section 4.9. It also reads the encryption key of a video when a player asks for it (step 5).

The second tier receives uploads and starts the processing. The browser sends the

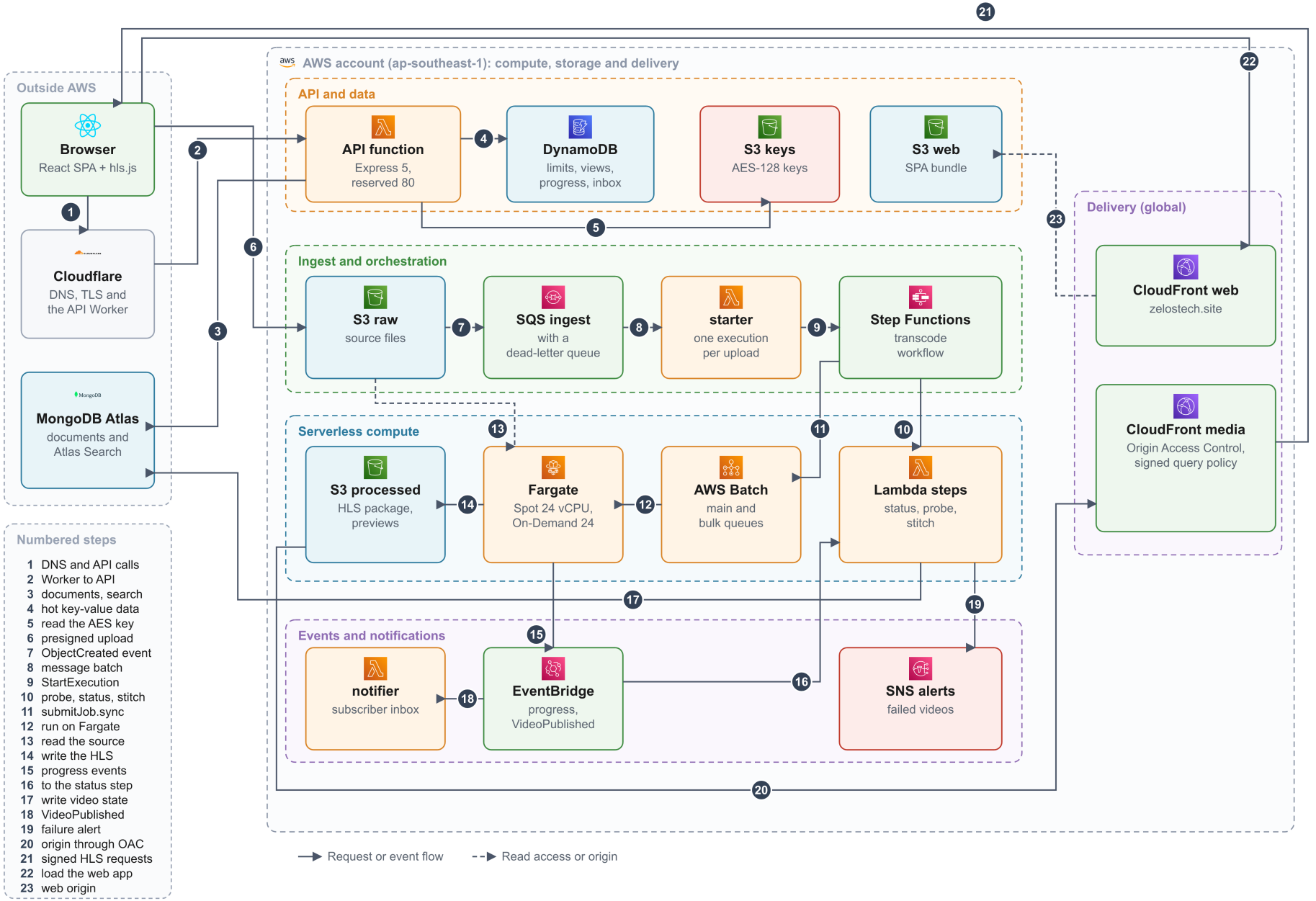


Figure 4.1: *Architecture of the platform and its deployment boundaries; the numbered steps are listed in the key on the left*

file straight to the raw bucket with a grant from the API (step 6). The arrival of the object makes Amazon S3 emit an event into an Amazon SQS queue (step 7), and a small function called the starter consumes the queue (step 8) and starts one AWS Step Functions execution for the upload (step 9). From then on the workflow owns the video: it calls short Lambda steps (step 10) and submits jobs to AWS Batch, waiting for each one to finish (step 11).

The third tier is the serverless compute. AWS Batch runs every job as a container on AWS Fargate (step 12), which reads the source (step 13) and writes the HLS package to the processed bucket (step 14). The containers hold no database credentials. They report progress as events on an Amazon EventBridge bus (step 15), and a single function, *status*, turns workflow states and progress events into the processing fields of the video (steps 16 and 17). The fourth tier handles what follows publication: when a video is published, the *notifier* function writes a notification for each subscriber (step 18), and a failed video raises an alert on Amazon Simple Notification Service (SNS) (step 19).

Delivery closes the loop. The media distribution reads the processed bucket through Origin Access Control (step 20) and serves playlists and segments only to requests that carry a valid signed policy (step 21). The web distribution serves the single-page application from its own bucket (steps 22 and 23).

The architecture scales in each tier through a different mechanism, and each mechanism has a ceiling that is set on purpose. The API scales by running more copies of the function, up to 80 reserved instances in production; with a pool of three database connections each, this stays well under the 500 connections that the database tier allows. Uploads scale without any server, since the browser writes to S3 directly. A burst of uploads becomes a burst of messages that the queue holds, and every upload gets its own workflow execution. Compute scales by starting containers: the Spot environment may run up to 24 vCPUs, and once it is full or Spot capacity is short, jobs overflow to an On-Demand environment of another 24 vCPUs. Both environments scale to zero when the queues are empty, which is what makes the processing tier cost nothing while idle. Delivery scales at the edge, since the CDN caches segments and every viewer shares the same cached copies.

4.2 Transcoding Workflow

4.2.1 Ingest and Idempotency

`raw/{videoId}/{uploadVersion}/source` is the object key that the upload grant fixes for the source; the grant also attaches three metadata fields: the video, the upload version and the owner. A single POST request carries files up to 5 GiB; larger files use a multipart upload that the API opened with the same key and metadata, so the browser only sends parts and the API completes the upload from the part list that S3 itself keeps. Either way, the event that starts processing is the creation of the object, so a lost browser request cannot leave an uploaded file unprocessed.

The starter never parses the key. It reads the metadata of the object, validates it against a schema shared with the API, and starts an execution named after the video and its upload version. The name, `{videoId}-{uploadVersion}`, is what makes a duplicate harmless: Step Functions refuses a second execution with the same name [15], so when SQS delivers the same event twice, as an at-least-once queue may, the second delivery finds the execution already started and does nothing. This makes the whole pipeline idempotent without a table of processed events. A message that keeps failing moves to a dead-letter queue after three receives.

4.2.2 State Machine

Figure 4.2 shows the state machine with the names of its states as defined in the infrastructure code. *MarkProcessing* sets the video to PROCESSING, creates its encryption key and confirms that the video still exists; if the owner deleted it in the meantime, the execution ends in *Gone* without doing any work. *Probe* runs ffprobe in a Lambda function and returns the plan: the streams, the display size after rotation, the frame rate, the rungs of the ladder, the path and, for a long video, the chunk plan.

Process is a parallel state with two branches that run at the same time. The first branch is *Analyse*, a Batch job that samples frames for moderation and makes the thumbnail and the seek-preview images. The second branch encodes. A video shorter than five minutes, or one that cannot be split safely, goes to *Encode*, one job that writes every rung. A longer video goes to the chunked path: *Audio*, when the source has audio, then *Chunks*, an array job, then *Stitch*. When both branches have finished, *Publish* checks that the master playlist exists and sets the video to READY, or SCHEDULED when its publication time is still ahead, together with the result of the screening.

Failures are handled in two layers. First, each task retries the errors that may pass

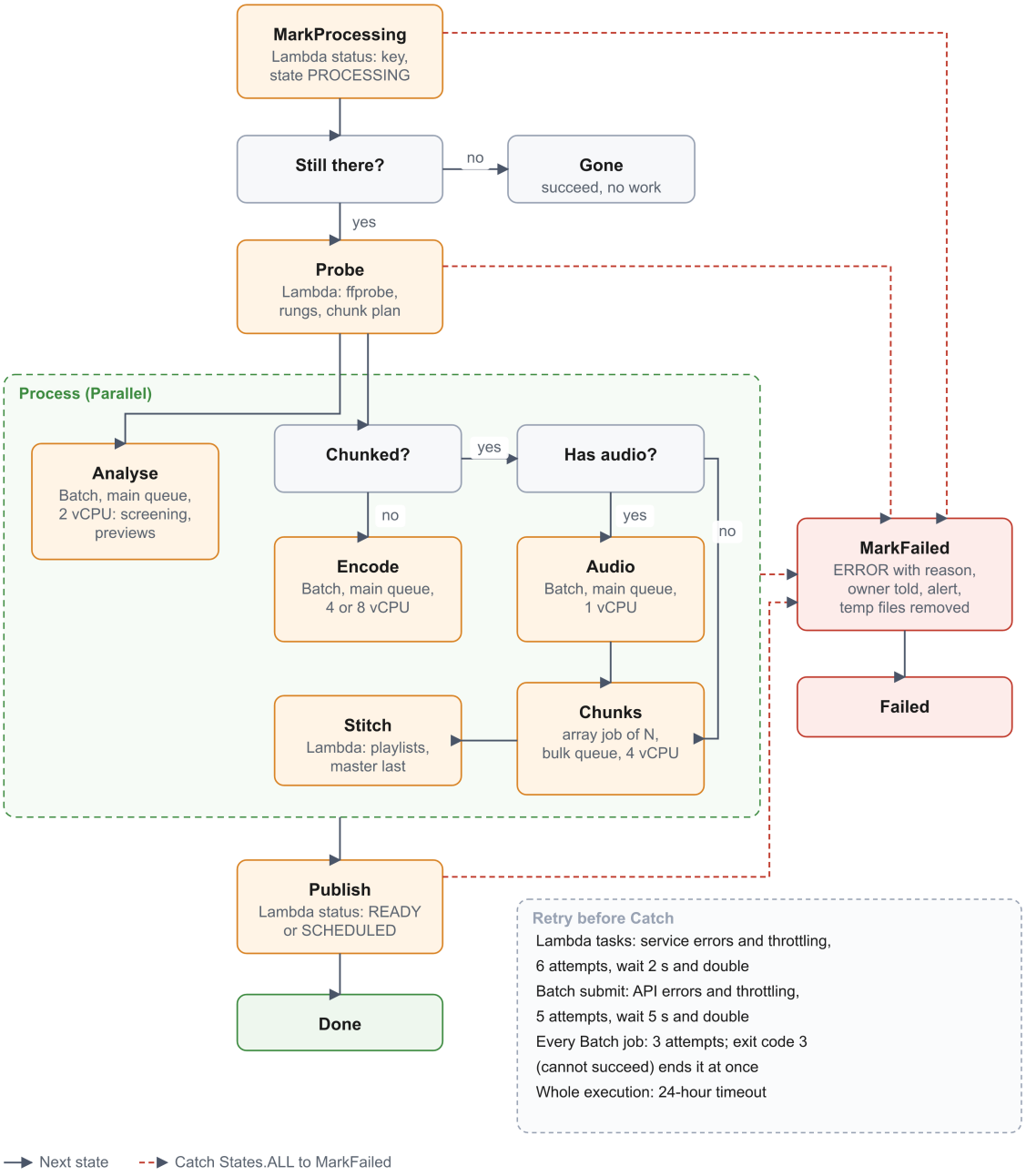


Figure 4.2: *State machine of the transcoding workflow, with its Catch transitions and retry rules*

on their own: a Lambda task retries service errors and throttling up to six times, waiting two seconds and doubling the wait, and a Batch task retries a rejected submission up to five times. Every Batch job itself has three attempts, so a Spot task reclaimed by AWS is simply run again. An error that another attempt cannot fix, such as an unreadable source, makes the container exit with code 3, which Batch treats as final. Second, whatever is left after the retries is caught: every task state, and the parallel state as a whole, has a *Catch* that leads to *MarkFailed*, which sets the video to ERROR with the reason, tells the owner once, sends an alert and removes the temporary files. An execution also has a timeout of 24 hours, queue time included, so no video can stay in processing forever.

4.2.3 Compute Sizing and Queues

All jobs use one container image built from a slim Node.js base and a pinned static build of FFmpeg, with one job definition per role. Table 4.1 lists the roles. Two queues share the Spot and On-Demand environments: *main*, with priority 10, for analysis, single encodes and audio, and *bulk*, with priority 1, for chunks, so that a short video uploaded while a long one is being processed does not wait behind dozens of chunks.

Table 4.1: Batch job roles, their size and their queue

ROLE	SIZE	QUEUE	WORK
analyse	2 vCPU, 4 GiB	main	Moderation frames, thumbnail, seek-preview sprites
encode	4 vCPU, 8 GiB up to 60 s; 8 vCPU, 16 GiB above	main	Every rung of a video up to five minutes, in one ffmpeg run
audio	1 vCPU, 2 GiB	main	The AAC tracks of a long video, once per bitrate
chunk	4 vCPU, 8 GiB per child	bulk	Every rung of one chunk of a long video

The sizes follow from measurements on the final configuration. A clip of up to one minute finishes encoding in about thirty seconds on four vCPUs, while a longer single encode benefits from eight, because one ffmpeg run feeds up to six x264 encoders from a single decoder. The threshold of five minutes for the chunked path is kept because, below it, one eight-vCPU job avoids starting the extra audio, chunk and stitch tasks. Chunks use four vCPUs each, so the 24 vCPUs of the Spot environment run six chunks at once and the On-Demand environment six more. The planner sizes the chunks for the six Spot slots, as Section 4.3.4 explains; when the overflow is free as well, twelve chunks run at once.

4.3 Media Design

4.3.1 HLS Output

Every upload version has its own folder in the processed bucket, so a retry never mixes the files of two runs and one signed policy for `videos/{videoId}/*` covers every version of a video. Figure 4.3 shows what the folder contains and which tags live in which playlist.

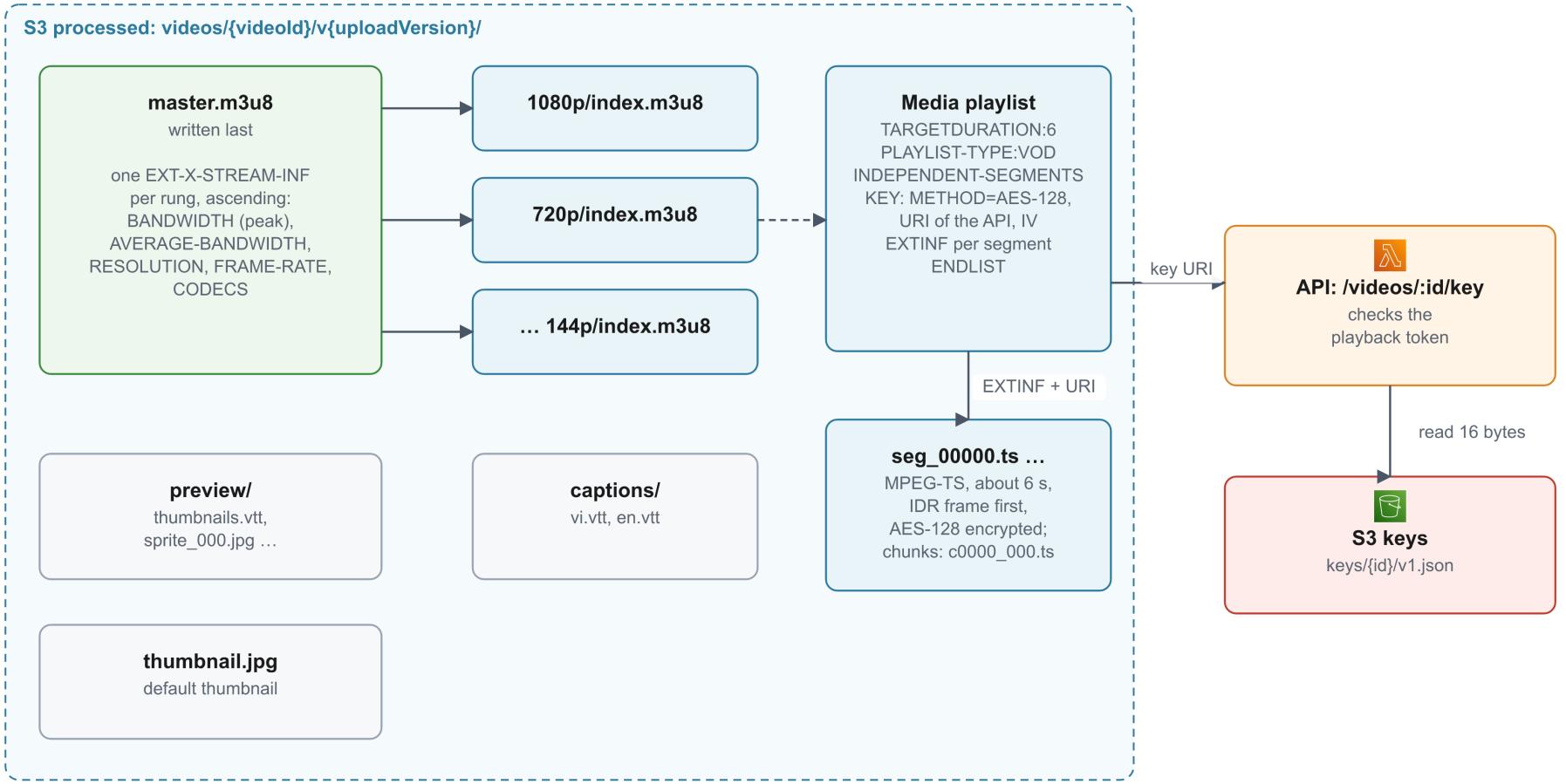
Listing 4.1 is the master playlist of a real video in production, a 1080p recording with audio. Each `EXT-X-STREAM-INF` line describes one rendition, in ascending order of bandwidth. `BANDWIDTH` is the highest bit rate of any segment of that rendition, measured from the real segment sizes after encoding and rounded up, as RFC 8216 requires of this attribute [1, 18]; `AVERAGE-BANDWIDTH` is the total size over the total duration. `CODECS` is read from the parameters of the first segment rather than guessed, and is left out when it cannot be read. The resolutions are not exactly 16:9 because this source is 1912 by 1080 pixels; each rung keeps the aspect ratio of the source with even dimensions.

```
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-INDEPENDENT-SEGMENTS
#EXT-X-STREAM-INF:BANDWIDTH=200589,AVERAGE-BANDWIDTH=153783,
  RESOLUTION=256x144,FRAME-RATE=28.725,CODECS="avc1.4d400c,mp4a.40.2"
144p/index.m3u8
...
#EXT-X-STREAM-INF:BANDWIDTH=4932815,AVERAGE-BANDWIDTH=3561550,
  RESOLUTION=1912x1080,FRAME-RATE=28.725,CODECS="avc1.4d4028,mp4a.40.2"
1080p/index.m3u8
```

Listing 4.1: Master playlist of a production video (first and last variants)

Listing 4.2 is the start and the end of the 720p media playlist of the same video. `EXT-X-KEY` names the encryption method, the address of the key on the API and an explicit initialisation vector, and applies to every segment that follows. Each `EXTINF` gives the duration of one segment: 6.006 seconds, which is the first frame boundary after six seconds at this frame rate, because segments are cut only at key frames. `EXT-X-ENDLIST` marks a complete video on demand, so the player knows the whole timeline from the start.

```
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-TARGETDURATION:6
#EXT-X-MEDIA-SEQUENCE:0
```



Side files (preview, thumbnail, captions) are not in the master playlist; the player loads them separately, so adding a caption never rewrites a published playlist.

Figure 4.3: *HLS package of one video in the processed bucket, and where its key comes from*

```
#EXT-X-PLAYLIST-TYPE:VOD
#EXT-X-INDEPENDENT-SEGMENTS
#EXT-X-KEY:METHOD=AES-128,
  URI="https://api.zelostech.site/v1/videos/6aea3d0094989c6980c6c03c/
  key?v=1",
  IV=0xda792782b937795de9f65f15773327ca
#EXTINF:6.006000,
seg_00000.ts
...
#EXTINF:3.970633,
seg_00046.ts
#EXT-X-ENDLIST
```

Listing 4.2: Start and end of the 720p media playlist of the same video

The order of the uploads matters as much as the content. Segments go first, then the media playlists, and the master playlist last, so a player can never open a master playlist whose renditions are still incomplete. Captions, the seek-preview sprites and the thumbnail are kept out of the playlists and loaded by the player separately, so adding a caption never rewrites a published playlist.

4.3.2 FFmpeg Processing Graph

Figure 4.4 shows the processing graph of one single-path encode. The job first copies the source to its local disk, because a long encode could outlive a presigned URL. One ffmpeg process then reads the copy once and decodes it once; the decoder hands every frame to each output, and each output has its own filter chain, encoder and HLS muxer. Decoding once matters because decoding a 1080p source is a large share of the cost, and producing six renditions with six separate processes would decode the same frames six times.

Each video chain scales the frame to the rung and sets square pixels, then encodes with x264 in the main profile at the rung's CRF, with a maximum rate and a buffer size that cap the peak. The remaining rules exist so that all rungs can be switched between, and they are the same on every output. A key frame is forced at every absolute multiple of six seconds, whatever the frame rate. The group of pictures (GOP) is six seconds of real frames, with the minimum interval set to the same value and scene-cut key frames disabled, so the encoder never inserts an extra key frame that would shift the next segment boundary. The pixel format is 8-bit 4:2:0, which every hardware decoder supports, and no H.264 level is forced, so x264 chooses it from the size and rate. Each output also encodes AAC-LC stereo audio at 128 kbit/s for 720p and above, and 64 kbit/s

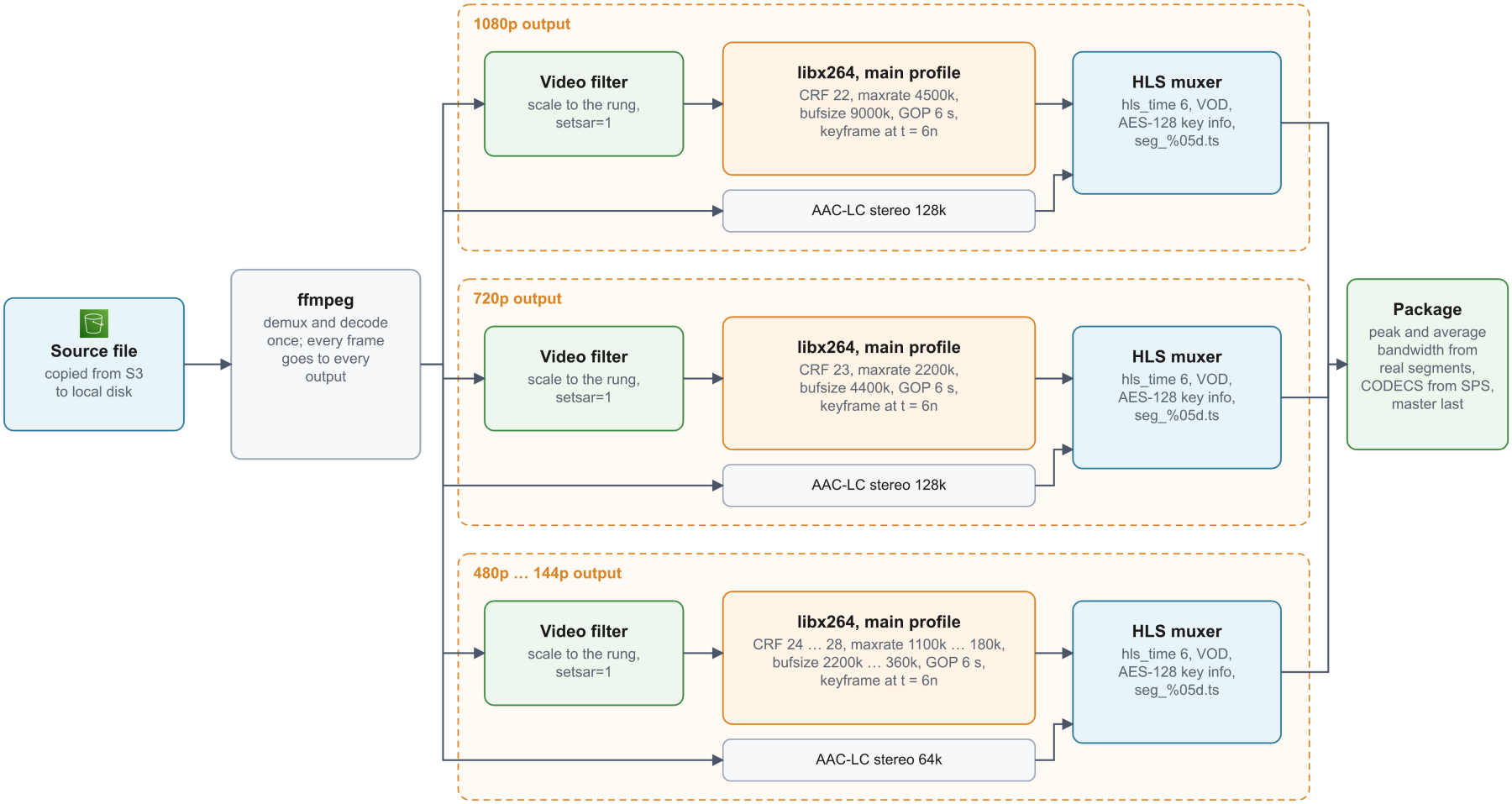


Figure 4.4: *FFmpeg processing graph of a single-path encode: one decode, one chain per rung*

below. Each HLS muxer cuts six-second segments, writes a VOD playlist and encrypts the segments with the key and IV of the video.

After ffmpeg ends, a packaging step measures the size of every segment to compute the two bandwidth values, reads the codec string, rewrites the media playlists in a canonical form and writes the master playlist, which is uploaded last.

4.3.3 Bitrate Ladder

Table 4.2 lists the six rungs. Each rung combines a CRF value with a VBV cap, the combination called Capped CRF in Section 2.2.4. CRF falls by one or two steps per rung as the resolution grows, because the same CRF looks worse on a larger screen. The maximum rates follow the spacing of common HLS ladders, roughly doubling from rung to rung, and the buffer holds two seconds at the maximum rate, which lets the rate vary within a segment while keeping the peak of each segment close to the cap [5, 18]. The cap is not a hard ceiling for a segment. VBV lets any six-second window carry six seconds at the maximum rate plus one full buffer, up to a third more, and a segment file also holds the audio and the MPEG-TS packaging of 188-byte packets. The measured peaks of Section 6.4.3 therefore lie up to 6.6% above the video cap plus the audio rate.

Table 4.2: The six-rung Capped-CRF ladder

RUNG	CRF	MAXRATE (KBIT/S)	BUFSIZE (KBIT)	AUDIO (KBIT/S)
144p	28	180	360	64
240p	26	350	700	64
360p	25	600	1200	64
480p	24	1100	2200	64
720p	23	2200	4400	128
1080p	22	4500	9000	128

The ladder is bounded by the source. A rung is named by the short side of its frame, so a portrait video gets portrait frames without padding. Rungs smaller than the source are kept, the first rung at or above the source is kept at the source size, and nothing larger is made, so a 360p source gets 144p, 240p and 360p and is never upscaled. A video longer than twenty minutes that falls back to the single path keeps only 360p, 720p and 1080p, to bound the time of one job; the chunked path always gets the full ladder.

Figure 4.5 shows what Capped CRF does with two real videos of production that went through the same ladder. For the recording, the average bit rate stays below the cap of every rung, reaching about three quarters of it at 1080p. For a synthetic test pattern, whose frames are easy to predict, the encoder needs only 274 kbit/s at 1080p, a sixteenth

of the cap. A fixed-bitrate ladder would have spent the full rate on both.

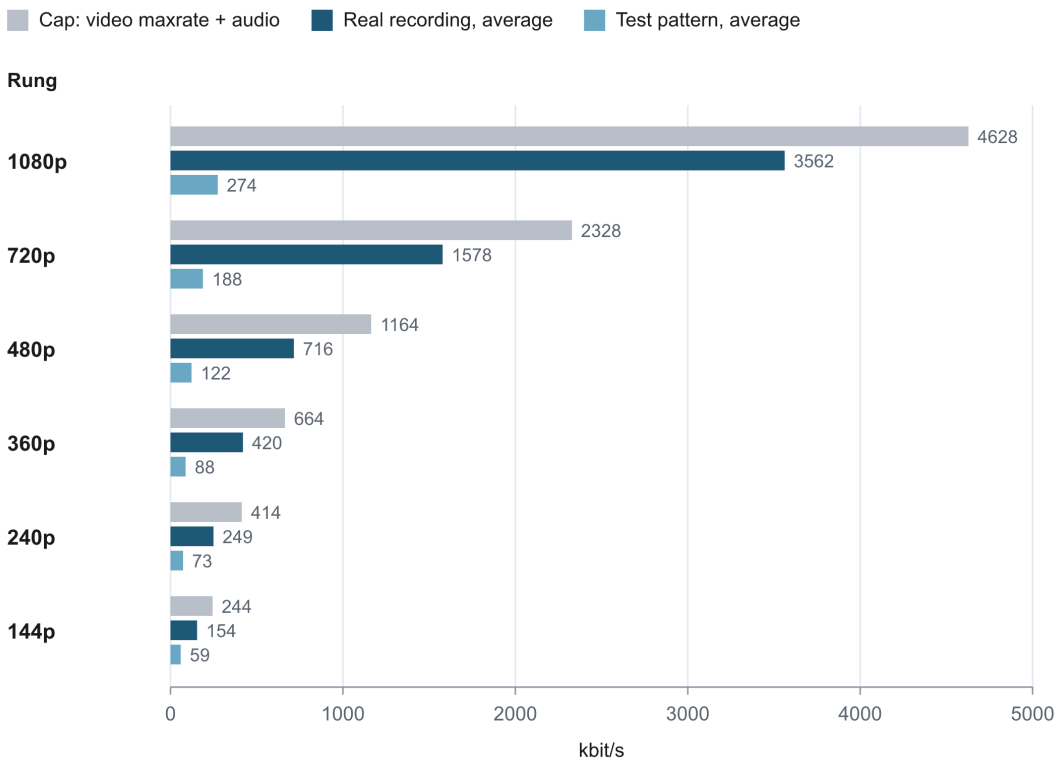


Figure 4.5: *Cap of each rung against the measured average bit rate of two production videos*

4.3.4 Long Videos: the Chunked Path

One encode of a four-hour video, even on eight vCPUs, would take hours. The chunked path in Figure 4.6 splits such a video into chunks that are encoded at the same time by separate containers, then joins the results into ordinary playlists.

The plan is made by the probe step and respects one rule above all: a chunk is a whole number of six-second GOPs. Since every chunk starts at a key frame and every segment inside it is six seconds, all rungs cut their segments at the same frames, and the joined playlists can be switched between exactly like those of a single encode. The planner aims at about three chunks per available slot, between 10 and 50 GOPs per chunk, so a ten-minute video becomes 10 chunks of one minute and a four-hour video 48 chunks of five minutes. A remainder shorter than one GOP joins the previous chunk, and the last chunk is open-ended, so an inaccurate frame count can never drop the end of the video. Each window starts half a frame before its first frame, because a seek that lands exactly on a frame boundary can return the previous frame, and every window ends exactly where the next one begins.

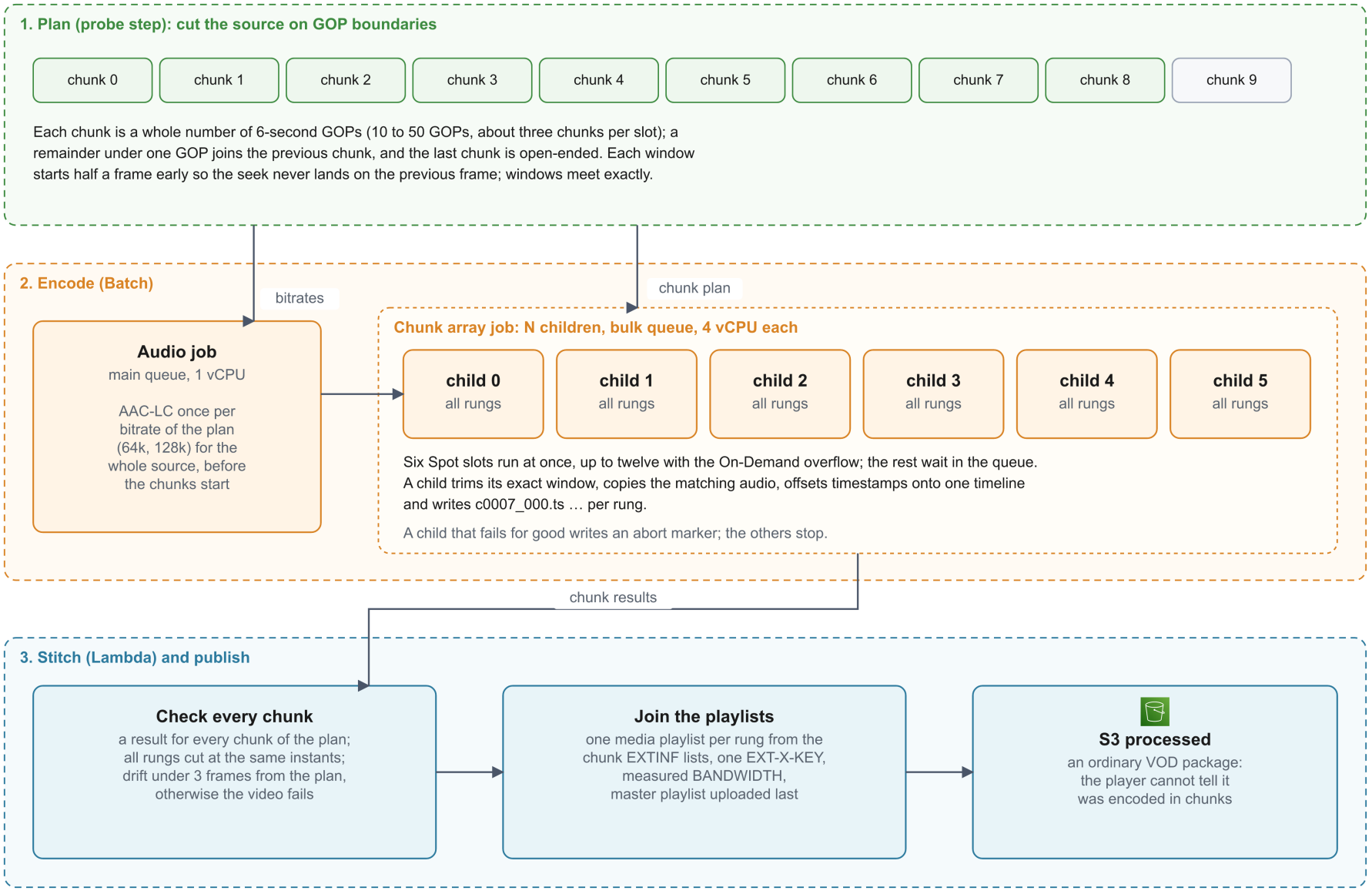


Figure 4.6: *The chunked path: plan on GOP boundaries, parallel encoding, checked join*

Audio is encoded once for the whole source, by a separate job, and each chunk copies the window of audio that matches its video. Encoding audio per chunk would leave small gaps or overlaps at every boundary, since audio frames do not align with video frames. Each chunk child then encodes every rung of its window with the same arguments as the single path, shifts its timestamps onto the shared timeline of the video, and writes its segments under names that carry the chunk index. A child that fails for good writes an abort marker, and the others stop instead of finishing useless work.

The stitch step refuses to publish anything it cannot trust. It checks that every chunk of the plan has a result, that all rungs of a chunk cut their segments at the same instants, and that no chunk is three or more frames longer or shorter than planned; any of these errors fails the video. Only then does it concatenate the segment lists into one media playlist per rung, with a single key tag, measures the bandwidth and uploads the master playlist last. The player receives an ordinary package and cannot tell that it was encoded in pieces.

4.3.5 Encryption and Playback

Protection of the content has two layers. The first controls who can fetch the files: the processed bucket is private and only the media distribution can read it, and the distribution serves a file only to a request that carries a policy signed for that video. The second makes the files useless without the key: every segment is encrypted with AES-128, and the key is released only by the API after it checks access. Figure 4.7 shows how a player passes both.

MarkProcessing creates a random 16-byte key and an explicit initialisation vector (IV) for each upload version and stores the key in a separate, private, encrypted bucket that only the transcoder and the API can read; the workflow passes only a reference, so the key never appears in the execution history or the logs. When a player asks to play a video, the API applies one access rule, the same function that also decides what the watch page shows: the owner and staff may always play, strangers learn nothing about private or unready videos, removed and held videos return 403, a password-protected video needs an unlock token, and a members-only video needs a membership of a high enough tier. If the rule allows playback, the API returns the master playlist address, a CloudFront policy for `videos/{videoId}/*` valid for two hours, and a playback token, a short token scoped to that video.

The policy travels as query parameters that the player appends to every request for the CDN, and never to the API. Cookies were the alternative [22], but browsers block

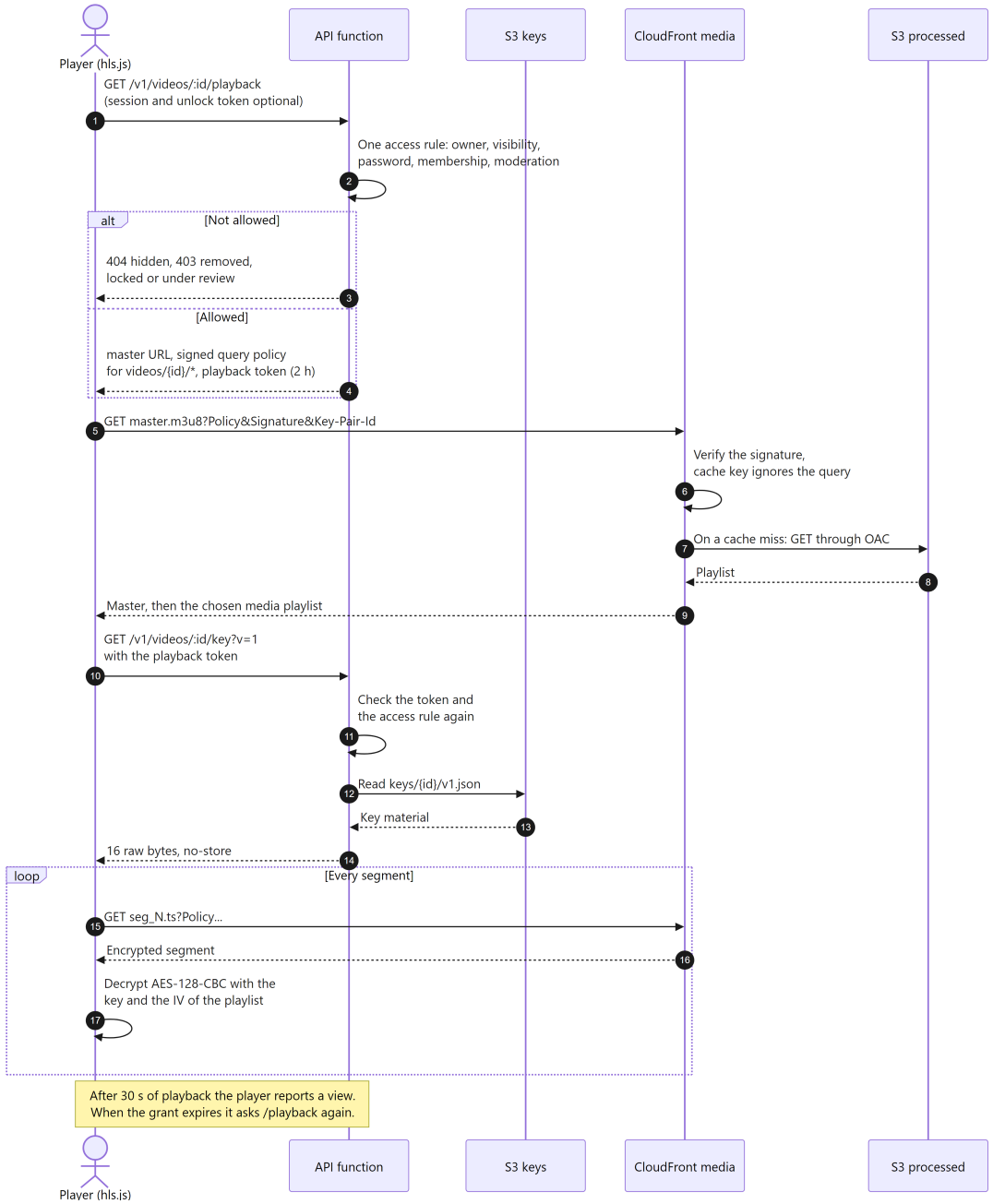


Figure 4.7: *Playback of an encrypted video: grant, signed CDN requests and key delivery*

third-party cookies inside an embedded player, and query parameters work everywhere. The cache key of the distribution ignores the query, so all viewers share the same cached segments even though each viewer's policy is different. The key endpoint accepts only the playback token, checks the access rule again for the viewer the token was issued to, and returns sixteen raw bytes that no cache may store. The player then decrypts each segment with AES-128 in cipher block chaining (CBC) mode, using the key and the IV of the playlist [1].

This design is not digital rights management. A viewer who may play a video can also save the decrypted stream, and Widevine or FairPlay would be needed to prevent that. What it guarantees is that no one without a grant for the video can obtain either the files or the key, even with the exact addresses.

4.3.6 Adaptive Bitrate in the Player

The player is built on hls.js, and Figure 4.8 shows the loop by which it chooses a quality. Before any segment is loaded, it has no measurement and starts from a default estimate of 500 kbit/s. Since the start level is left to it, hls.js first downloads one segment of the lowest rung only to measure the throughput, without decoding or showing it, so the first picture comes from a rung chosen on a real measurement. After each segment it measures the throughput of the download, folds it into moving averages, scales the estimate down by a safety factor of 0.95 and chooses the highest rendition whose declared `BANDWIDTH` fits under it [19]. The switch takes effect at the next segment boundary. If the buffer runs low while a segment is still downloading slowly, the player abandons it and loads a lower rendition at once rather than stalling.

Two settings of the player shape the result. *capLevelToPlayerSize* caps the choice at the smallest rendition whose width or height covers the larger side of the video element on the screen, multiplied by the pixel ratio of the display, which saves bandwidth on small windows; Chapter 6 shows its effect in the measurements. A choice in the quality menu fixes the rendition and turns automatic selection off until the viewer chooses Auto again. The loop depends on the encoding side in two ways: segments that start with a key frame at the same instant in every rung make a switch invisible, and a `BANDWIDTH` measured as a real peak makes the choice safe.

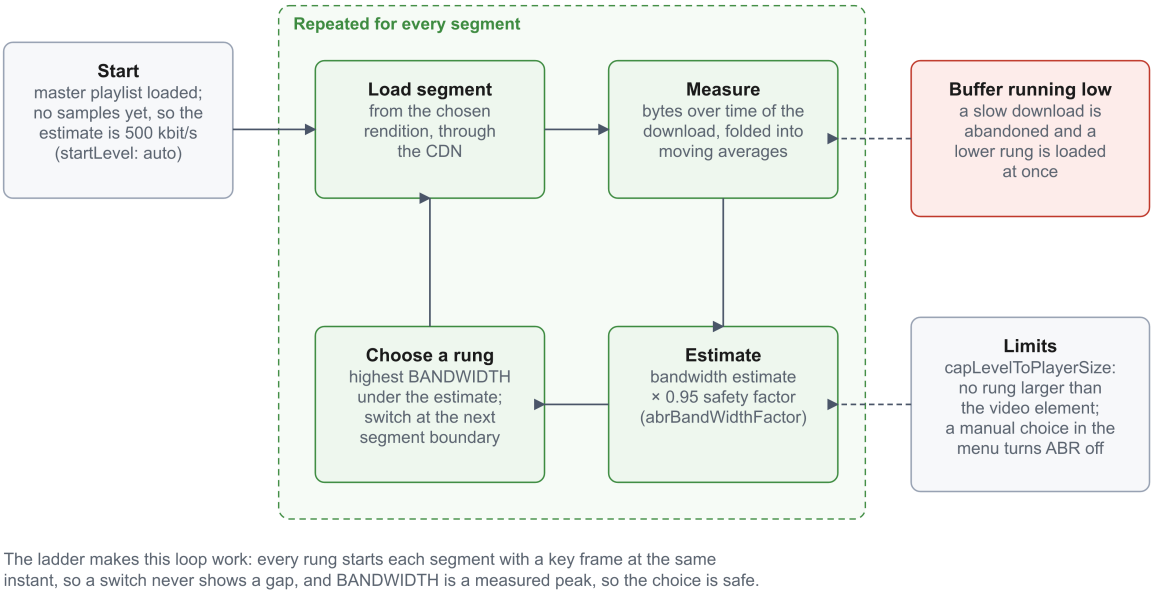


Figure 4.8: *Quality selection loop of hls.js with the settings of the player*

4.4 Database Design

The documents of the platform live in MongoDB Atlas. A document store suits this data for two reasons. A video is naturally one document whose shape grows with features: captions, chapters, the moderation result and the counters are nested inside it and read together on every watch page, and adding a field never needs a migration. And Atlas includes a full-text search engine in the same cluster, so search needs no second system [21]. The cost of this choice is that the database does not enforce references between collections; the design compensates with unique indexes for every rule that must hold, and with counters that are recomputed rather than trusted.

Figure 4.9 shows the nine collections of the product. A channel is not a separate entity: it is the user, identified by a unique handle. Each foreign key names the collection it refers to.

The rules that must never break are enforced by unique indexes, so they hold even when two requests race. An e-mail address and a handle belong to one account; a Google identity links to at most one account, through a partial index that ignores accounts without one. A viewer has at most one reaction per video and one subscription per channel, a channel has at most one active tier per level, a viewer has one membership per channel, and every account has exactly one Watch later playlist. Where a rule applies only to open items, such as one open report per person and video, the index is partial on the status, so closed items do not block new ones.

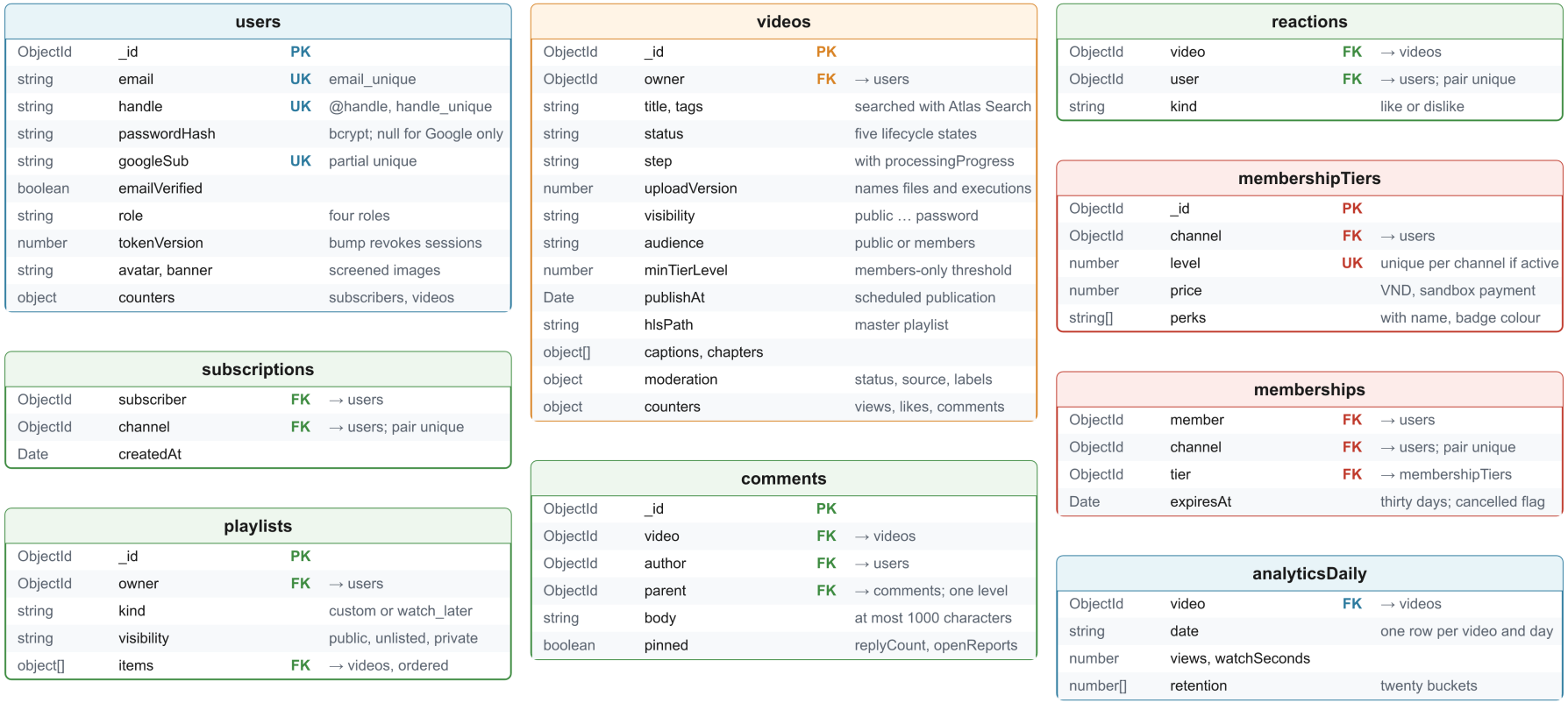


Figure 4.9: *Collections of the product in MongoDB, with their keys and unique indexes*

A video moves through five states. UPLOADING is set by the API when the upload is created; PROCESSING, READY, SCHEDULED and ERROR are set only by the *status* function of the workflow. The API changes only the fields an owner edits, so the two writers never touch the same field. Moderation is a separate dimension with its own states, PENDING, APPROVED, REVIEW and REMOVED, so a video can be ready and still held for review.

Counters follow two rules. Counters of relationships, such as subscribers, likes, comments and replies, are recomputed with *countDocuments* after each change, so they can never drift from the documents they count. View and watch-time totals are sums of counted events and are increased in place, because recounting them would itself cost database operations.

Data that is written every few seconds is kept out of MongoDB, as Table 4.3 shows. The free tier of Atlas throttles above one hundred operations per second, and a single playing video reports progress every minute, so writing these records to MongoDB would spend the budget of the whole platform on bookkeeping. Amazon DynamoDB on demand is billed per request and expires entries by itself.

Table 4.3: Short-lived key-value data in DynamoDB

TABLE	CONTENT	EXPIRY
rateLimits	One counter per caller, scope and window	End of the window
viewMarks	One mark per viewer and video once a view is counted	Thirty minutes
watchProgress	Resume position per user and video, which also gives the history	None
heartbeats	Watch time and retention per video and hour, rolled up hourly into *analyticsDaily*	Seven days
notifications	Inbox of each user	Ninety days

Search uses three Atlas Search indexes, the most the free tier allows: videos, channels and playlists. The video index covers the title, description, tags and channel name with an analyser that lower-cases and folds diacritics, after first mapping the letter đ to *d*, which Unicode folding leaves alone, so that a query typed without Vietnamese marks still finds the video. Queries tolerate one or two typing errors and filter on visibility, status and moderation inside the index, so private, unready or removed videos never appear in results.

Figure 4.10 shows the six collections of moderation. Two of them are append-only logs, the moderation events and the role changes: the API inserts into them and never updates them, which is what lets an auditor trust them.

reports

ObjectId	video	FK	→ videos
ObjectId	reporter	UK	→ users; one open each
string	reason		eight reasons
string	details		at most 500 characters
string	status		open, dismissed, actioned
ObjectId	resolvedBy	FK	→ users

commentReports

ObjectId	comment	FK	→ comments
ObjectId	reporter	UK	→ users; one open each
string	reason		same eight reasons
string	status		open, dismissed, actioned
string	commentText		kept when removed

moderationEvents

ObjectId	video	FK	→ videos
ObjectId	actor	FK	→ users, with actorRole
string	decision		approve or block
string	action		four actions
string	fromStatus		with toStatus
string	machineStatus		verdict before people
string	outcome		six outcome classes
string	reasonCode		closed list
string[]	labels		top screening labels
Date	createdAt		append only

roleChanges

ObjectId	user	FK	→ users
string	from, to		roles before and after
ObjectId	by	FK	→ users; append only

appeals

ObjectId	video	UK	→ videos; one open each
ObjectId	owner	FK	→ users
string	message		10 to 500 characters
string	status		open, upheld, rejected
string	resolution		note of the operator

auditSamples

ObjectId	video	UK	→ videos; one pending
string	status		pending, pass, fail
ObjectId	createdBy	FK	→ users (administrator)
ObjectId	labelledBy	FK	→ users (operator)

UK unique index, partial where the note says so · FK reference; the note names the collection

Figure 4.10: *Collections of moderation and operation*

4.5 API Design

The API is a REST interface under the base path `/v1`. Requests and responses are JSON, and every error has the same shape, an object with a machine-readable code such as EMAIL_NOT_VERIFIED and a message, so the interface can react to the code and show the message. Listings are paged with an opaque cursor rather than page numbers, so a list never repeats or skips an item while new videos are published. A signed-in request carries a bearer token in the `Authorization` header. Request bodies are limited to 64 kB, because no video data passes through the API. Table 4.4 lists the route groups.

Table 4.4: Route groups of the API

GROUP	ROUTES	ACCESS
Accounts	`/auth`: register, login, google, link-google, verify-email, resend-verification, forgot-password, reset-password, change-password, logout, me	Public or signed in; rate-limited
Uploads	`/uploads`: create, parts, complete, abort	Signed in, verified address
Videos	`/videos`: list, get, update, delete, unlock, playback, key, view, heartbeat, related	Public under the access rule; edits by the owner
Discovery	`/search`, `/search/suggest`	Public, rate-limited
Community	reactions, comments, replies, pin, delete, report a comment	Signed in
Channels	`/channels/:ref`: profile, videos, subscription, membership	Public; actions signed in
Studio	thumbnail, captions, chapters, schedule, analytics, avatar, banner, tiers, members	The owner
Library	`/me`: history, progress, feed, playlists, notifications, subscriptions	Signed in
Moderation	report and appeal a video; `/admin`: queue, decisions, comment reports, appeals, audit, metrics, events, users, roles	Viewers; staff by permission

Two conventions follow from the design. Every route that touches a video applies the same access rule as playback, so a private video that the requester may not see returns not-found rather than forbidden, which does not reveal that it exists. And no route carries video bytes: an upload is a grant around a direct transfer to Amazon S3, and playback is one call that returns a signed policy for the CDN.

4.6 Interface Design

Figure 4.11 sets out the main paths through the application. Two of its properties constrain the design. The first is that the home feed and the watch page are reachable without an

account, and authentication is demanded only where an action needs an identity; a visitor who chooses to upload is sent through sign-in or registration and returned to the upload page. The second is that the upload path does not end when the user's part ends. Once the browser has sent the file, the interface returns control and shows the video in the creator studio with its current step and progress, and only when the workflow marks it ready does it join the feed. The interface therefore has to show a video that exists but cannot yet be played, which is why its states are explicit.

The web application is a single-page application of React components. Its pages are the home feed, search results, the watch page, channel pages, the subscriptions feed, the library with history, Watch later and playlists, the creator studio with upload, editing, captions, chapters, scheduling, tiers and analytics, the settings, the staff console, the embedded player and the pages for errors. It is available in Vietnamese and English and in light and dark themes, and its styles are plain Cascading Style Sheets (CSS) with design tokens, without a utility framework.

The video player is a custom component built on hls.js rather than the browser's native controls, because native controls cannot be extended: the browser has its own menus for speed and picture-in-picture, which cannot be merged with a quality selector. The component therefore implements play and pause, seeking with the sprite preview, volume, full screen, speed, picture-in-picture, captions, chapters and quality in one settings menu, and resumes from the saved position.

The staff console is part of the same application, and a person sees only the tabs their role's permissions allow: the review queue, removed videos, reported comments, appeals, audit samples, the rates, the event log and the users. An operator never sees the rates, and an auditor never sees a decision button. Thumbnails of videos under review stay blurred until a reviewer reveals them, so that reading the queue does not by itself expose staff to the content.

4.7 Security Design

Authentication uses passwords hashed with bcrypt, or Google sign-in. A new password must have at least eight characters and is refused when it is common, contains the user's own e-mail address or handle, or appears in a known breach [23]. Sessions are bearer tokens that carry a version number of the account; signing out, changing or resetting the password, or a Google claim of an unverified account increments the version, which revokes every older token at once. Tokens sent by e-mail are stored only as hashes, are valid for 24 hours for verification and one hour for a reset, and are spent on use.

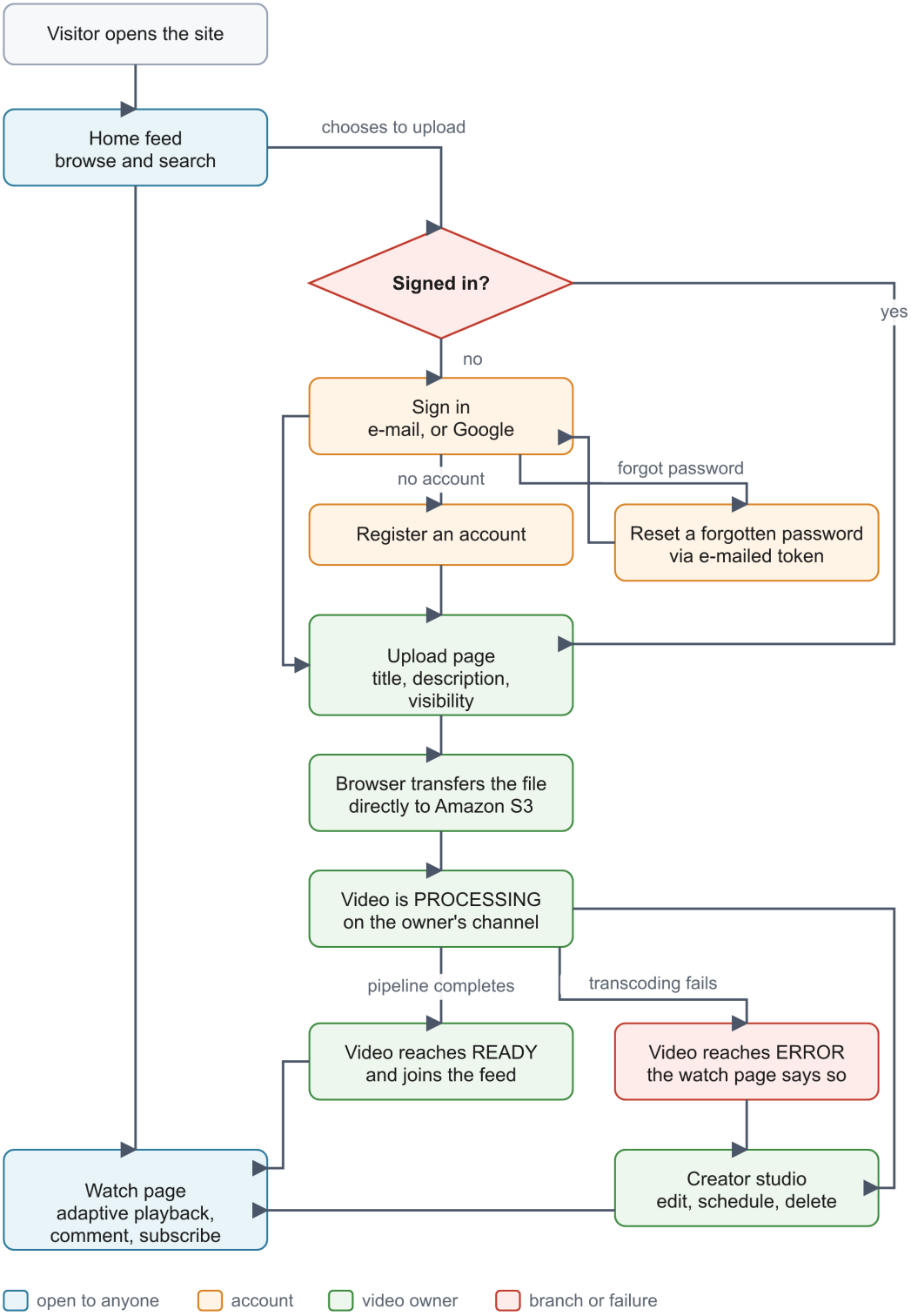


Figure 4.11: *User flow from an anonymous visit to a published video*

Figure 4.12 shows how an address is verified, by the link or by Google, and why a verified local account is never taken over by a Google sign-in.

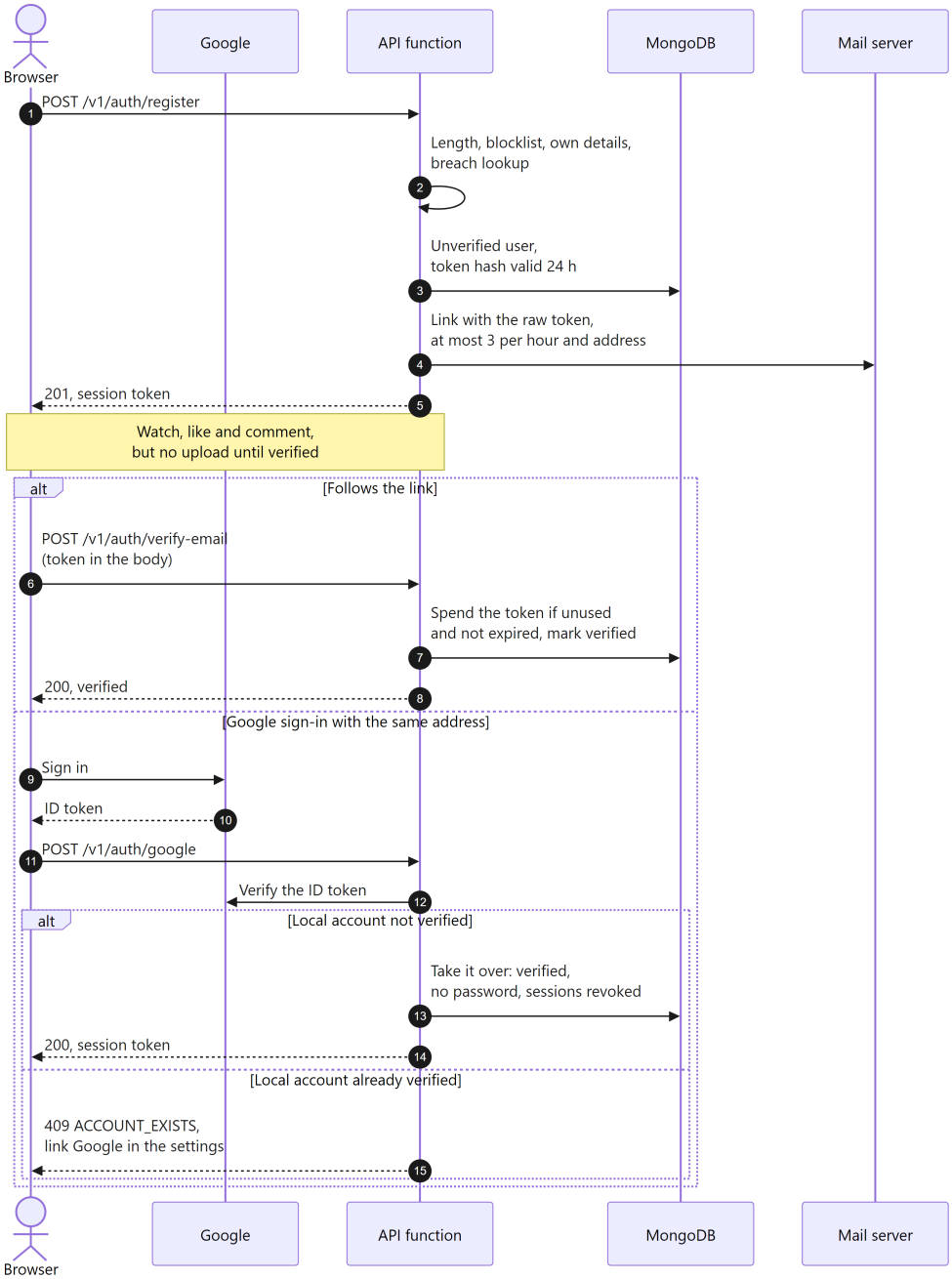


Figure 4.12: *Verifying the owner of an account: by the e-mailed link, or by Google sign-in with the same address*

Authorisation is a table of permissions held by roles, which Table 4.5 lists. The API checks a permission, never a role name, so adding a role changes one table. The role is read from the account on each request through a cache of 30 seconds in each instance of the function. The instance that changes a role clears its own entry at once; the other running instances see the change when their entry expires, so a role change reaches every

instance within 30 seconds.

Table 4.5: Permissions of the staff roles

PERMISSION	ALLOWS	ROLES
moderation:review	Review queue, decisions, reported comments	administrator, operator
moderation:appeals	Resolve appeals	administrator, operator
moderation:audit	Label audit samples	administrator, operator
moderation:sample	Draw audit samples	administrator
metrics:read, events:read	Error rates, decision log, role changes	administrator, auditor
users:manage	Users and roles	administrator
content:viewRemoved	Open removed and held videos	administrator, operator

The API is protected in depth. The function behind it refuses any request without the secret header that only the Cloudflare Worker adds, so its own address is useless to an attacker. Browsers may call it only from the platform's own origins, and the key endpoint additionally refuses other origins. Every request counts against a limit of 300 per minute per caller, and sensitive routes have tighter ones, such as ten sign-in attempts per address in fifteen minutes; the counters live in DynamoDB so that every copy of the function shares them. Security headers are set on every response.

Least privilege governs the infrastructure. The processing containers have no database credentials: their role may read the raw and keys buckets, write the processed and work buckets, publish events and call Rekognition, and nothing more. Only the *status* function writes processing state. Secrets, such as the database address and the signing keys, are held in AWS Secrets Manager and read by the functions at start-up. The protection of the content itself, private storage, signed CDN requests and encryption, is described in Section 4.3.5.

4.8 Content Moderation Design

Moderation combines automatic screening, which acts on every video before it is published, with human decisions that correct it and are recorded so that its errors can be measured. Figure 4.13 shows the whole flow.

4.8.1 Automatic Screening

The *analyse* job takes one frame every five seconds, at most 120 frames, so the interval widens for long videos and the cost of a video is bounded. Each frame goes to Amazon

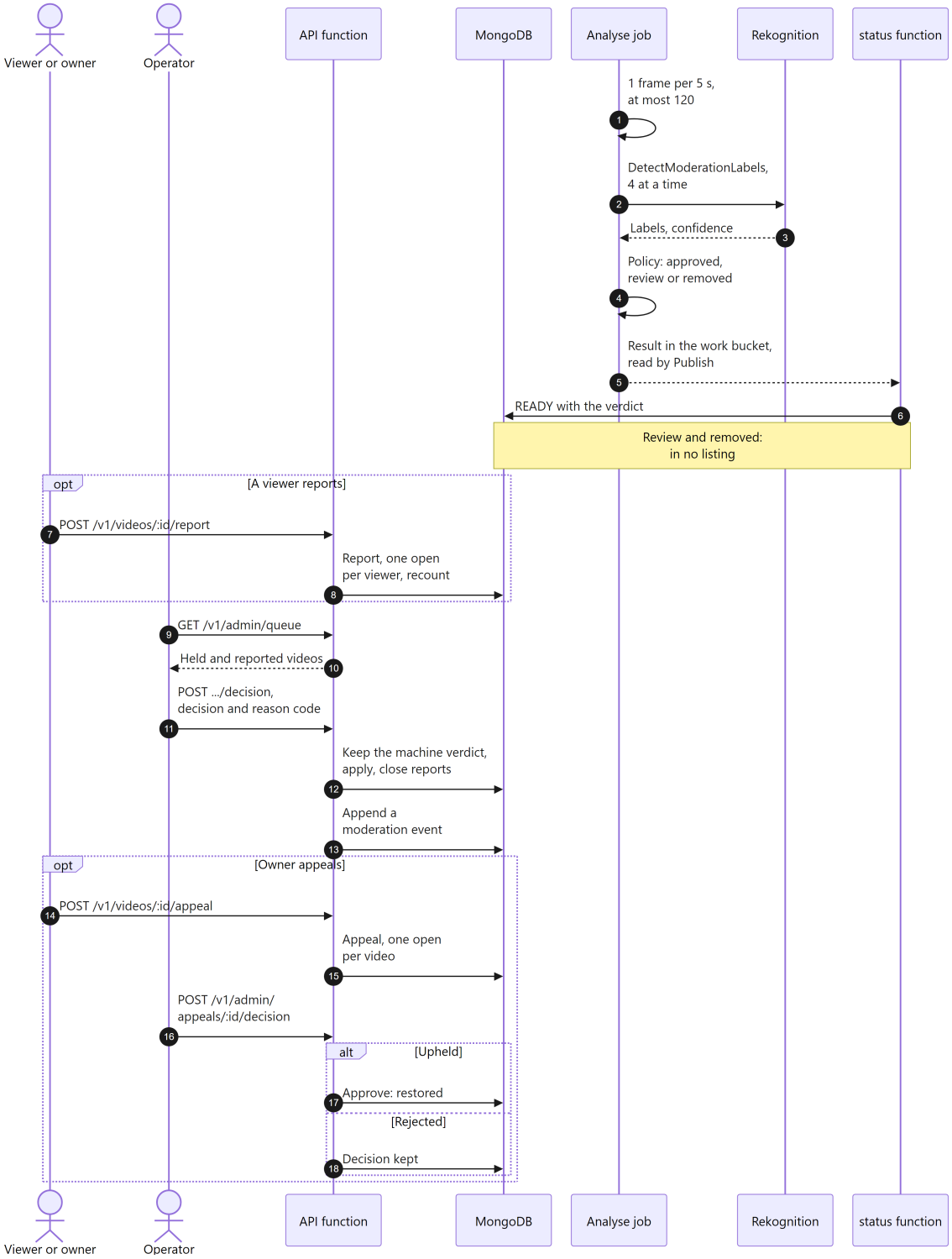


Figure 4.13: *Moderation: automatic screening, a viewer's report, an operator's decision and the owner's appeal*

Rekognition, four at a time, and the labels are mapped by a shared policy to block, review or ignore, following the label taxonomy of the service [24]. Labels below 60% confidence are treated as noise. A block label at 90% confidence or more removes the video; any other relevant label holds it for review. A throttled call to Rekognition is retried by the client with adaptive back-off, up to five attempts, so a burst of uploads slows the screening rather than skipping frames. Labels such as graphic violence in animation or illustration are lowered to review, since fiction is not the same as a recording. A video with violence that may stay is published behind a warning that the viewer dismisses. Screening fails closed: when no frame, or less than 80% of the planned frames, could be analysed, the video is held for review rather than published. Avatars, banners and custom thumbnails pass through the same policy and are refused on any relevant label.

4.8.2 Review, Appeals and Audit

An operator decides on every held or reported video with a decision, approve or remove, and a reason code from a closed list for that decision, such as a model error, fiction, a confirmed violation or a report upheld. Before the first human decision overrides it, the verdict of the machine is kept, and each decision is appended to the event log with an outcome class that compares the two, as Table 4.6 shows. An operator cannot change a decision taken by an administrator. The owner of a removed or held video can appeal while no other appeal on it is open, and an upheld appeal approves the video with the reason “appeal upheld”.

Table 4.6: Outcome classes of a human decision against the machine verdict

MACHINE VERDICT	HUMAN DECISION	OUTCOME
removed	approve	false positive
review	approve	flag noise
approved	approve	true negative
approved	remove	false negative
review or removed	remove	true positive
none recorded	either	unclassified

Decisions alone cannot measure what the screening misses, because nobody reviews a video that the machine approved and no viewer reported. The random audit fills this gap: an administrator draws a sample of videos that the machine alone approved, an operator labels each one pass or fail, and a failed video is removed. From these records the console computes three rates, each with a 95% Wilson interval, which stays meaningful for small samples and rates near zero, and the sample size beside it. The first

is the share of reviewed automatic removals that a person reversed. Its denominator is the removals, not every acceptable video, so it estimates the false discovery rate of removals (one minus their precision) rather than the false positive rate. The second is the precision of holds. The third is the share of audited approvals that failed, which estimates the false omission rate of approvals rather than the false negative rate, since its denominator is the approved videos and not every violating one. Removals reach a person when an operator opens them or the owner appeals, not at random, so the first rate can be biased toward contested cases; only the third comes from a random sample.

4.8.3 Roles and Separation of Duties

The roles are kept apart so that no person checks their own work. Operators decide and label; auditors read the rates and the log but hold no permission that changes anything; administrators manage roles and draw samples, and can act as either. The last administrator cannot be demoted, and every change of role is logged with who made it.

4.9 Performance and Scalability

Section 4.1 described how each tier scales; this section sets out the limits that bound the scaling and how the design stays within them. The tightest are those of the free database tier: 500 connections and one hundred operations per second.

Connections are budgeted per function, as Table 4.7 shows. Each function that uses MongoDB has a fixed reserved concurrency and a small connection pool per instance, so the worst case is known in advance and stays near half of the limit. The functions of the workflow that need no database, the starter, the probe and the stitch, open no connection at all, and idle instances release theirs after a minute.

Table 4.7: MongoDB connection budget of the production environment

FUNCTION	RESERVED INSTANCES	POOL	MAXIMUM CONNECTIONS
api	80	3	240
status	5	2	10
notifier	2	2	4
rollup, reconciler	1 each	2	4
Total			**258 of 500**

Operations are protected by moving every frequent write to DynamoDB (Table 4.3) and by keeping the processing containers away from the database, which reduces the

writes of a video in processing to a few per minute. What remains is the work of real requests. A burst of one hundred uploads arriving within seconds still needs several operations per upload; the free tier then slows the requests rather than failing them, and a request refused under overload receives a status that the client retries. Chapter 6 measures the effect.

On the compute side, the ceilings are those of Batch and Fargate: 24 vCPUs of Spot and 24 of On-Demand in production, set by Terraform below the account quota of 64 of each kind so that staging can run beside production. The two queues keep short videos ahead of chunks: the scheduler starts a waiting job of *main* before any waiting chunk, so a new upload waits at most for one running chunk to end, a few minutes, and is never queued behind all the chunks of a long video. On the delivery side, the CDN caches segments for every viewer, because the signed policy is not part of the cache key, so the number of viewers adds load at the edge rather than at the origin.

Chapter 5

Development and Testing

This chapter describes how the design of Chapter 4 was built. It presents the structure of the code, the implementation of the processing pipeline, the API and the web application, the infrastructure code and the delivery pipeline. It then records the problems met while building the system and how each was solved, and ends with the automated tests.

5.1 Repository Structure

The whole system lives in one repository organised as npm workspaces, so that a change to a contract shared by two parts is made, type-checked and tested in one commit. Table 5.1 lists the workspaces and where each one runs. Every workspace except *infra* is written in TypeScript and runs on Node.js 22; each deployable part is bundled into a single file with esbuild, so a Lambda package or the container image carries no *node_modules* folder.

Table 5.1: Workspaces of the repository and where each one runs

WORKSPACE	CONTENT	RUNS ON
shared	Contracts between producer and consumer (zod schemas of events, job inputs and object metadata), video states and their allowed transitions, the bitrate ladder, the chunk planner, playlist writing and stitching, the moderation label policy	Imported by every other part; never deployed alone
transcoder	The four container roles *analyse*, *encode*, *audio* and *chunk*, the FFmpeg argument builder, the process runner and the encode guard	AWS Batch on Fargate
functions	*starter*, *probe*, *status*, *stitch*, *notifier*, *rollup* and *reconciler*	AWS Lambda
api	The REST API: Express 5 application, features, services and models	AWS Lambda behind a Function URL
web	React 19 single-page application built with Vite, the hls.js player, vanilla CSS	S3 and CloudFront
e2e	Playwright tests of the web application against a running environment	The developer's machine
infra	Terraform modules and environments, the Cloudflare Worker	AWS and Cloudflare

The *shared* package is what keeps the event-driven parts consistent. The starter function validates an S3 event against the same schema that the API used when it wrote the object metadata, and the status function accepts a progress event only if it matches the schema that the container used to emit it. A field renamed on one side therefore fails the type check of the other side before it can fail in production.

5.2 Processing Pipeline

5.2.1 Workflow Functions

The seven functions are small and each has its own role with only the permissions it needs. The starter turns each record of an SQS batch into one Step Functions execution named after the video and its upload version, and reports only the failed records back to SQS, so one bad message does not make the whole batch return to the queue. The probe function runs *ffprobe* from a Lambda layer that holds a static build of the tool, plans the job with the functions of *shared* and stores the plan in the work bucket; the state machine receives only a summary, which keeps the execution history small. The status function is the only writer of processing state: it moves a video through PROCESSING, READY or ERROR, folds progress events from the two parallel branches into one percentage that never goes

backwards, and announces a published video exactly once. The stitch function joins the chunk playlists and writes every media playlist before the master playlist, as Section 4.3.1 requires. The last three functions serve the API: the notifier writes a notification for each subscriber and mails those who opted in, the roll-up adds the viewing heartbeats of the previous hours into daily analytics, and the reconciler removes drafts whose upload never finished.

5.2.2 Transcoder Image

The four roles share one container image built in stages. The build stage bundles the entry points; the runtime stage holds only Node.js, a static FFmpeg build and the bundle, runs as a non-root user and measures 136 MB compressed. Both base images are pinned by digest, so two builds of the same commit produce the same image. *ffprobe* is left out of the runtime image because probing runs in Lambda. Every Fargate task pulls the image before it starts, and pulling dominates the start-up time of a container [13], so its size bears directly on how quickly a job begins. A separate *smoke* stage adds *ffprobe* and a test program on top of the runtime stage; it is used only by the delivery pipeline of Section 5.5, and because the runtime stage is the last one in the file, a build without an explicit target can never ship the test tools.

The encode and audio roles download the source to the task's disk before they start FFmpeg. Reading it through a pre-signed URL would avoid the copy, but such a URL is signed with the task's temporary credentials and stops working when they expire, which can happen during an encode of several hours. The chunk role reads only its own time range, which is short, so it does read through a pre-signed URL.

5.2.3 Failure Handling Inside a Job

A failed job ends with one of two exit codes, because the retry rule of Section 4.2.3 depends on it. A failure that would repeat with the same input, such as a file FFmpeg cannot decode or a plan that fails validation, ends with exit code 3, which Batch does not retry; every other failure, including a Spot reclaim or a network error, ends with a code that Batch retries. The FFmpeg runner keeps the last lines of the error output in the failure reason and reports a process killed by a signal as such, so an operator can tell a reclaimed Spot task from a broken file.

The encode guard stops a run that cannot finish in time as soon as that is clear, instead of letting it use the Fargate task until the timeout. During the run it projects the total time as the video duration multiplied by the ratio of elapsed time to encoded position. It

gives no verdict during the first 120 seconds or before 30 seconds of video are encoded, and it stops the run only when the projection has stayed above 90% of the job budget for 180 seconds, so one slow scene never ends a healthy job.

5.3 API and Web Application

The API is one Express 5 application adapted to Lambda with *serverless-express*. It is organised by feature: authentication, uploads, videos, community, channels, library, studio, discovery and moderation each register their own routes, and reach storage, the CDN signer, mail, the scheduler, image screening and search through service interfaces. In production these interfaces are backed by AWS and Atlas; in the tests they are backed by in-memory implementations, which lets a test run a whole route without the network. Every request body is validated with a zod schema, and every error leaves the API in the shape `{error:{code,message}}` of Section 4.5. The rule that decides whether a user may see or play a video is one function, called by every route that exposes a video, its comments, its playlist or its key.

A new instance does as little as possible before its first request: it reads its secret from Secrets Manager and opens the database connection, and both are kept for the life of the instance. Indexes are created by a separate maintenance action, run once after each deployment (Section 5.6). The Cloudflare Worker in front of the Function URL replaces the host header with that of the function, removes any `X-Origin-Verify` header sent by a client and adds the secret one, so the API refuses every request that did not pass through Cloudflare, and trusts the client address in `CF-Connecting-IP` only on such requests.

The web application is written in React 19 with React Router and i18next for its two languages. Styling uses vanilla CSS with custom properties for the light and dark themes, without a utility framework. Two parts of its client code carry most of the robustness. The HTTP client retries a request that receives 429 or 503 up to three times, after about 0.5, 1 and 2 seconds with random jitter, and follows a `Retry-After` header capped at ten seconds; a retry is safe even for a POST, because both answers are given before any handler runs. The upload client sends the parts of a multipart upload, retries a failed part with a newly signed URL, and stops at once when the viewer cancels.

5.4 Infrastructure as Code

All cloud resources are described with Terraform in eleven modules, listed in Table 5.2, and three environments. The *shared* environment holds what every environment uses

once: the virtual private cloud (VPC) and the image registry. The *staging* and *prod* environments compose the modules with their own names (prefix *zelos-staging* or *zelos-prod*), capacity ceilings and hosts. Each environment keeps its state in its own key of an S3 bucket, with a DynamoDB table as lock, so two applies can never write the same state at once. Every resource carries the tags *Project*, *Stack*, *Environment* and *ManagedBy*.

Table 5.2: Terraform modules and their responsibilities

MODULE	RESPONSIBILITY
network	One VPC with public subnets and a free S3 gateway endpoint, so no network address translation (NAT) gateway is billed while idle
ecr	Registry of the transcoder image, with scanning on push
storage	The raw, processed, work and keys buckets of one environment, their encryption, lifecycle rules and access blocks
ingest	The S3 event notification, the ingest queue and its dead letter queue with an alarm on the first message
events	The EventBridge bus for pipeline events and the SNS topic for operator alerts
kv	The five DynamoDB on-demand tables of Table 4.3, with time-to-live
batch	The Fargate Spot and On-Demand compute environments, the *main* and *bulk* queues and the four job definitions
workflow	The state machine, the starter, probe, status and stitch functions, the ffprobe layer and one role per function
app	The API function and its Function URL, the notifier, roll-up and reconciler, and the scheduler group for scheduled publishing
cdn	The media distribution, its Origin Access Control, signing key group, cache and header policies, certificate and DNS record
web	The private bucket and distribution that host the web application, with the rewrite that serves the single-page routes

An environment declares three providers: AWS in the Singapore region, which also manages CloudFront because CloudFront is a global service, AWS in *us-east-1*, the only region where CloudFront accepts certificates, and Cloudflare for DNS. The *cdn* and *web* modules receive the first two through aliases, so the account and the regions they use are chosen by the environment and not written into the modules. The transcoder image is selected with a variable that holds the commit of the image and has no default value, so a production apply cannot fall back to an untested image.

5.5 Continuous Integration and Deployment

Two GitHub Actions workflows check every push and pull request, as shown in the upper part of Figure 5.1. The first lints all workspaces, type-checks them, runs the unit tests and builds every deployable part; it then builds the *smoke* stage of the image and runs

real FFmpeg in it, and checks the formatting and validity of the Terraform code without access to any account. The second is the security gate: Gitleaks scans the whole history for secrets, Trivy scans the dependencies and *eslint-plugin-security* analyses the code statically. The security jobs are required checks of the rule that protects the main branch, so a pull request cannot be merged while one of them fails.

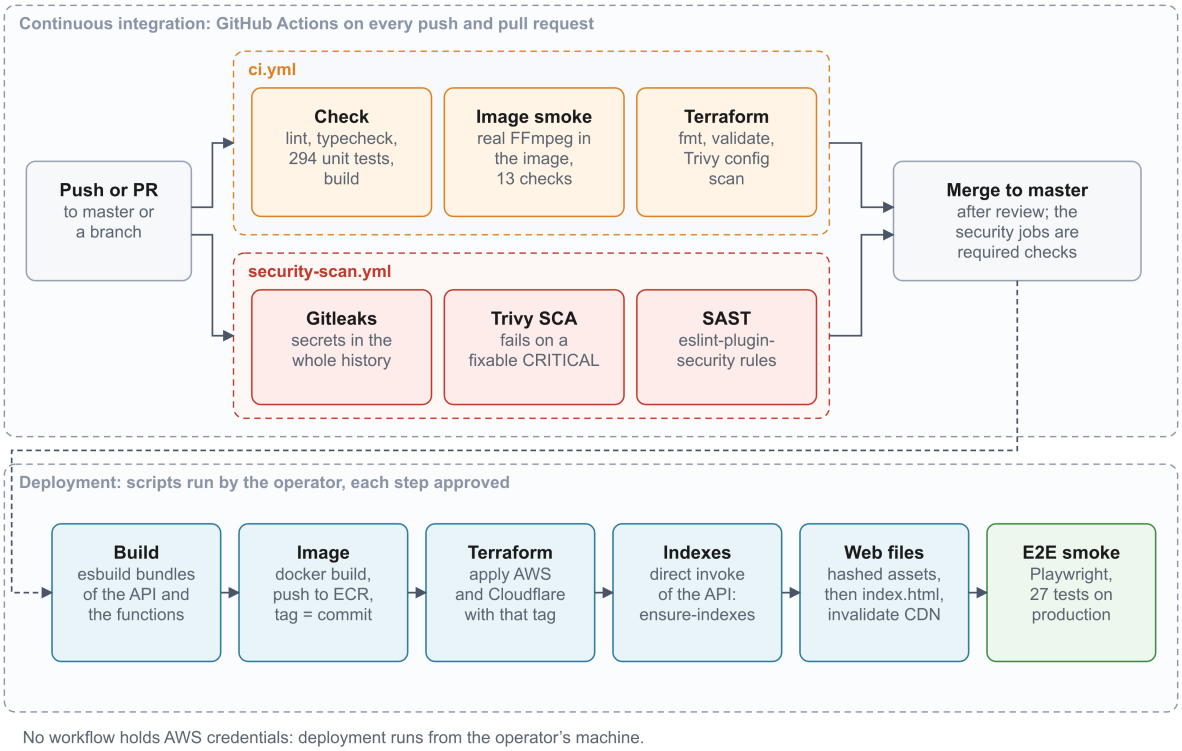


Figure 5.1: *The checks run by GitHub Actions on every push and pull request, and the deployment steps run after a merge*

The dependency scan has two tiers. It reports every critical and high finding without failing, and fails only on a critical finding that already has a fix. Most high findings come from transitive dependencies with no patch yet, so a gate that failed on all of them would fail for reasons the team cannot act on and would soon be ignored. The scan of the Terraform code is stricter and fails on any high or critical misconfiguration, because every such finding can be fixed in the project's own code. Each action that runs Trivy is pinned to a release commit rather than a branch, so a change in another repository cannot alter what runs in the pipeline.

The smoke test runs the encoding commands of the deployed image on generated sources and checks the result as a player would. It encodes a 40-second source on the single path and a 75-second source on the chunked path, decrypts and decodes every rung, and checks the codec strings, the segment durations, the rising bandwidth values and the absence of repeated audio packets at chunk boundaries; a third case checks the

preview sprites. In thirteen checks it exercises what the unit tests cannot, namely the behaviour of FFmpeg itself.

Deployment is deliberately not automated. No workflow holds AWS credentials; the steps in the lower part of Figure 5.1 run from the operator's machine, and each is approved before it runs. The API and the functions are bundled; the image is built and pushed to the registry with the commit as its tag; Terraform applies the environment with that tag; the maintenance action creates the indexes; and the web files are uploaded, hashed assets first and the entry page last, so the new entry page never refers to a file that is not there yet, after which the distribution is invalidated. The end-to-end tests of Section 5.7 then run against the deployed environment. A change is applied to *staging* and checked there before it is applied to *prod*.

5.6 Challenges During Development

Several problems appeared between a design that was correct on paper and a system that behaved correctly once deployed. Table 5.3 lists them with their causes and resolutions; the two with the widest effect are described after it.

Table 5.3: Problems met during development, their causes and resolutions

PROBLEM	CAUSE	RESOLUTION
Player cannot decrypt any segment	The Lambda response format carries text, so the 16-byte key was altered on its way out	The key endpoint's body is returned base64-encoded with the flag that tells Lambda to decode it; a test sends the key through the adapter and compares the raw bytes
502 responses during a burst of uploads	Each new API instance created the indexes before its first request; dozens starting at once overloaded the free cluster and passed the start-up timeout	Indexes are created by a maintenance action invoked once per deployment; a start-up only reads the secret and connects
Function URL refused calls from the Worker	The permission must carry a condition that the Terraform provider in use cannot express	A Terraform resource adds the statement with the AWS command-line interface (CLI), and skips it when it already exists
ffprobe layer rejected by Lambda	The archive exceeds the size that can be uploaded directly	The layer is published from an object in an artifacts bucket, named by its checksum so each version can be traced to its build
Repeated audio at chunk joins	Each chunk kept audio packets from before its seek point, so the packet at each boundary appeared twice	FFmpeg drops packets before the seek point (`-copypriorss 0`); the smoke test counts repeated packets
Plan replaced the CloudFront signing key	Git on Windows checked the public key out with CRLF line ends, which Terraform saw as a new key	`.gitattributes` keeps LF for key files, shell scripts and the CloudFront function

5.6.1 One Policy for the Processed Bucket

A bucket has exactly one policy, but two modules need a statement in the policy of the processed bucket: *storage*, which refuses every request without TLS, and *cdn*, which lets its distribution read the segments. If both wrote the policy, each apply of one would remove the statement of the other, and the bucket would alternate between refusing CloudFront and accepting plain HTTP. The *cdn* module therefore does not write the policy: it returns the statement that grants read access to its distribution, and the *storage* module merges it with its own and attaches the result. The statement names the CloudFront service with a condition on the Amazon Resource Name (ARN) of the distribution, so the processed bucket accepts no other reader and a segment can be fetched only through the signed distribution.

5.6.2 Index Creation at Start-up

The free Atlas cluster limits concurrent requests and operations per second (Section 4.9). Creating indexes is idempotent and cheap for one instance, so it first ran before each instance served its first request. Under a burst of uploads, however, the reserved concurrency of the API let dozens of instances start together, each sending its index commands to the cluster at the same moment; the cluster refused them as too many concurrent requests, start-up passed its timeout and the uploads that reached those instances received 502. Indexes change only when the API code changes, so they are now created by a maintenance action that deployment invokes directly. The action cannot be reached through the Function URL: the API accepts it only from a direct invocation, which needs an AWS Identity and Access Management (IAM) permission that only the operator holds.

5.7 Testing

The tests are organised in three levels, summarised in Table 5.4. Unit tests check the logic of each workspace without the cloud; the smoke test checks FFmpeg inside the image; the end-to-end tests check the deployed system through a browser. The acceptance checklist of the design lists the edge cases each part must handle, and each line of it is covered by at least one test.

Table 5.4: Test levels, their size and where they run

LEVEL	SIZE	WHAT IT CHECKS AND WHERE
Unit	305 tests in 32 files	Logic of every workspace with Vitest, on every push; the API runs against a real MongoDB server started in memory, with AWS services replaced by in-memory implementations
Image smoke	13 checks	Real FFmpeg inside the built image: both encoding paths encrypted, packaged, decrypted and decoded, on every push
End-to-end	27 tests in 6 files	The deployed web application and API through Chromium with Playwright, against staging and then production after each deployment

Table 5.5 groups the unit tests by workspace. The *shared* workspace holds the largest share because the decisions that affect every video are made there: which rungs a source receives, how a long video is cut into chunks, how playlists are written and when chunks may be joined.

Table 5.5: Unit tests by workspace, 305 tests in 32 files

WORKSPACE	TESTS	FOCUS
shared	104	The ladder bounded by the source, with portrait and very small sources, and the reduced ladder for long single jobs; frame rates, rotation and sample aspect ratio read from probe data, and the conditions that keep a video on the single path; chunk plans whose windows meet exactly, including 48 chunks for a 4-hour video at 30 fps and a 48-hour video under the array limit; playlists with the peak bandwidth rounded up and codec strings read from the stream; the stitch checks of Section 4.3.4; the check that an output covers the probed duration; the label policy; progress folding; the event contracts
transcoder	27	FFmpeg arguments: one decode, the same key frame rules on every rung, capped CRF with the rung's buffer, chunk seeks half a frame early and audio without repeated packets; the exit codes that Batch retries or not; the encode guard; preview sprites
functions	37	One execution per upload version and none for a duplicate event; state changes of the status function, including publishing once, scheduled publishing and failure alerts whose reason holds no presigned query; the order of playlist writes; notifications within the daily mail cap; the hourly roll-up and the draft clean-up
api	109	Registration, sign-in, password rules and the Google account rule; the origin secret, client addresses and rate limits; uploads by pre-signed POST and multipart, with exact part sizes; the access rule for every visibility and moderation state; the key returned as 16 raw bytes; views after 30 seconds; comments, reactions, subscriptions and memberships with recounted counters; playlists and history; captions, chapters and schedules; search folding Vietnamese letters; reports, decisions, appeals, audit samples and roles
web	28	Retries with back-off and `Retry-After`; multipart upload with part retries and cancel; the 30-second view rule on real playback only; the signed policy added to CDN addresses and never to API calls; chapters, timestamps and preview thumbnails

The tests of the access rule matter most for security. They assert that a private, unready or scheduled video is answered with not-found rather than forbidden to anyone but its owner, because a forbidden answer would confirm that a video with that identifier exists; that a members-only video returns no playlist address to a non-member; and that the key endpoint refuses another origin, another video, another upload version or a missing token, and stops serving the key once the video is removed.

The end-to-end tests sign in through the API in a set-up step, so no password appears in a test. The pipeline test is the longest: it uploads a 36-second clip through the interface, waits until the video is ready, plays it through the CDN with its key, edits it in the studio and deletes it. The other files cover the pages of a guest, a viewer, a creator and an administrator, and the embedded player inside a page of another origin. All 27 tests pass against production; the measured behaviour of the deployed system is presented in

Chapter 6.

Chapter 6

Deployment and Evaluation

This chapter presents the system as deployed and measures it. It describes the production environment and its interface, the method of the measurements, and then the results for playback, transcoding, behaviour under load and cost. It ends by setting each non-functional requirement of Chapter 3 against the evidence.

6.1 Deployed Environment

The platform runs in production at the domain of the project, in the Singapore region of AWS. Table 6.1 lists what the production environment consists of and the setting that bounds each part. A second environment, *staging*, is built from the same code with lower ceilings, and changes are checked there before they reach production (Section 5.5).

Table 6.1: The production environment

PART	SERVICE	SETTING
Web application	S3 bucket behind CloudFront	Hashed assets cached for a year
API	One Lambda function, Node.js 22, 512 MiB, behind a Cloudflare Worker	80 reserved instances
Workflow	S3 events, SQS, the starter, Step Functions and four Lambda steps	One execution per upload version
Compute	AWS Batch on Fargate Spot and Fargate On-Demand, queues *main* and *bulk*	24 + 24 vCPUs, six + six chunk slots
Media delivery	CloudFront, Origin Access Control, signed query policy	Edges of price class 200, which include Asia
Documents and search	MongoDB Atlas, free M0 cluster of its own	500 connections, 100 operations per second
Short-lived data	Five DynamoDB tables on demand	Billed per request

6.2 User Interface

Figure 6.1 shows the watch page with the quality menu of the player open: the menu lists the rungs that the source produced, here every rung from 240p to 1080p above the *Auto* entry, which leaves the choice to the loop of Section 4.3.6.

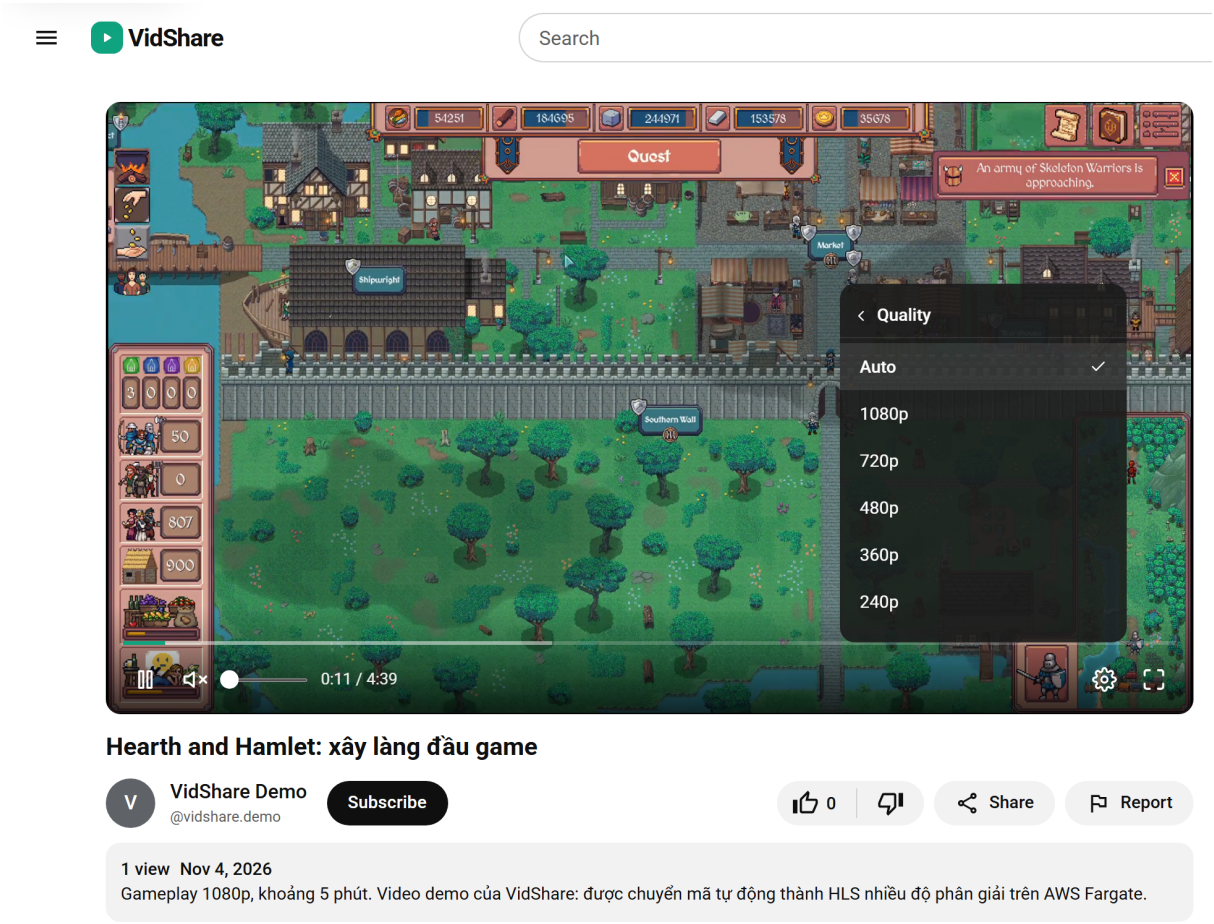


Figure 6.1: *Watch page with the quality menu of the player open*

Figure 6.2 shows a search typed without Vietnamese diacritics. The query *xay lang* finds the title *xây làng*, because the search index folds diacritics on both sides (Section 4.4). Figure 6.3 shows the page of a channel, with its handle, its subscriber count and its videos.

Figure 6.4 shows the upload page before a file is chosen. It states the accepted formats and the ceiling of 20 GiB, which the interface writes as GB, and tells the creator that processing starts when the transfer ends. Figure 6.5 shows the accuracy tab of the staff console. Every rate is shown with the number of decisions behind it, and the console says that there is not enough data rather than print a rate computed from no decision at all.

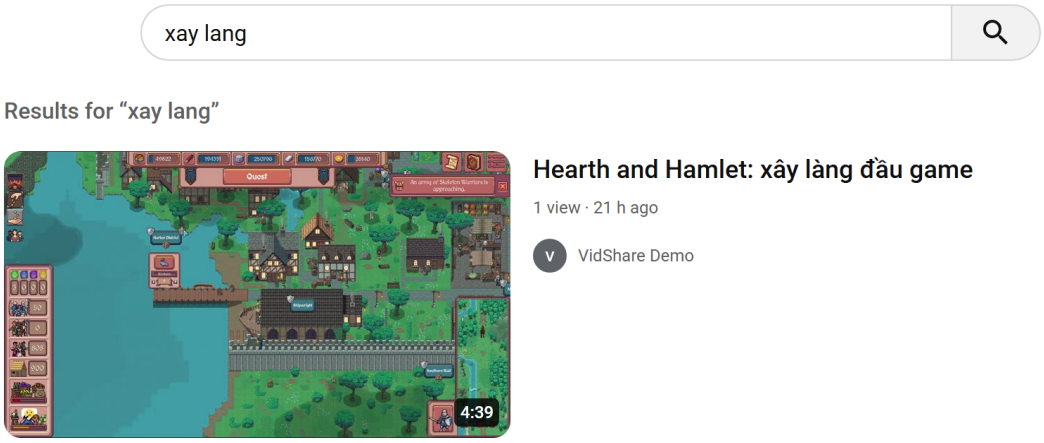


Figure 6.2: *Search results for a query typed without diacritics*

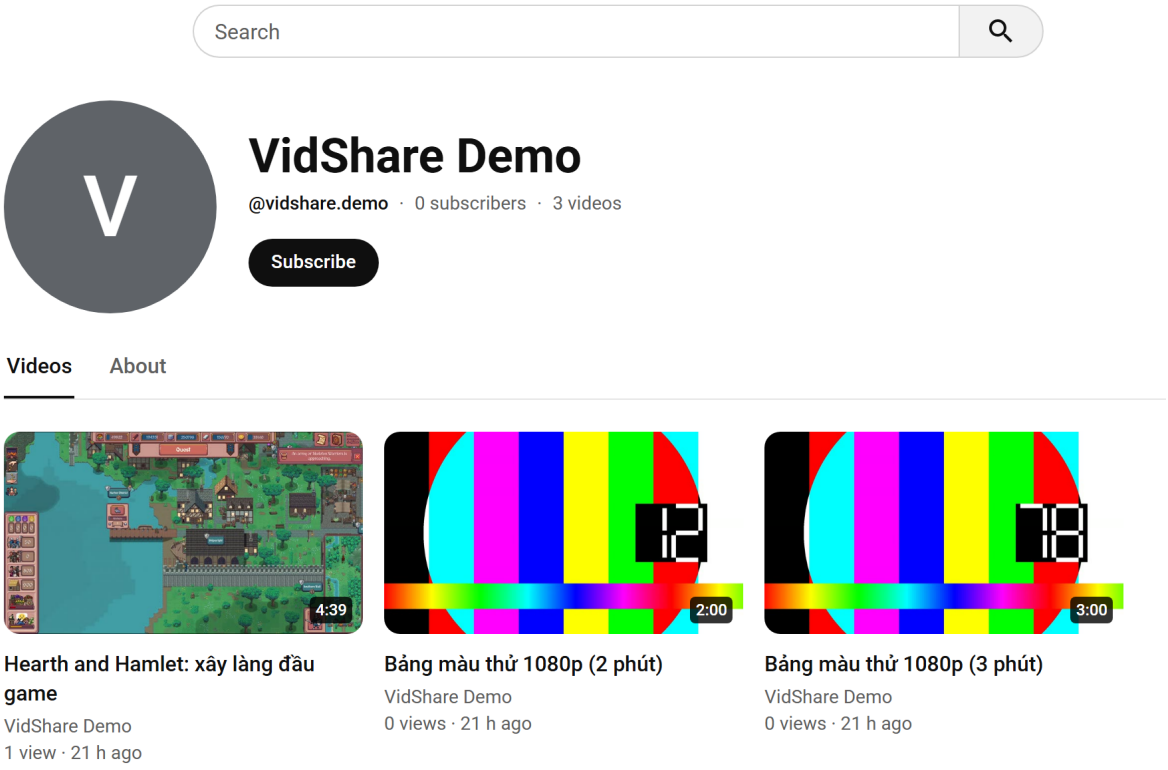


Figure 6.3: *Channel page*

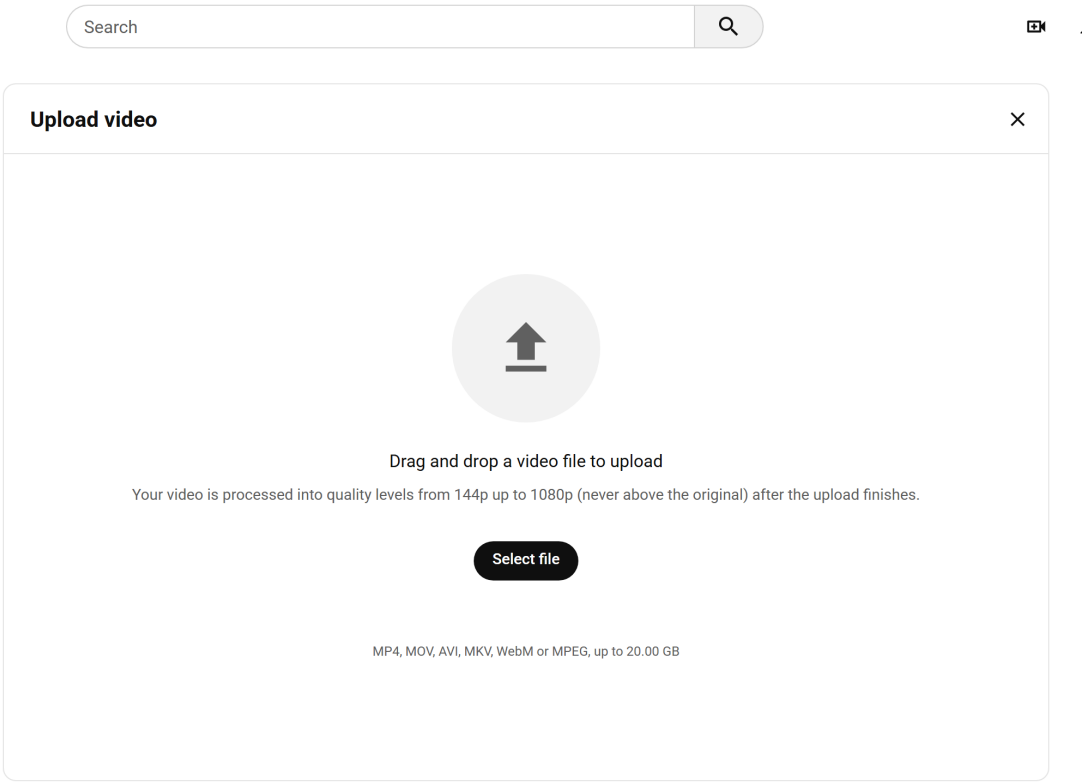


Figure 6.4: *Upload page*

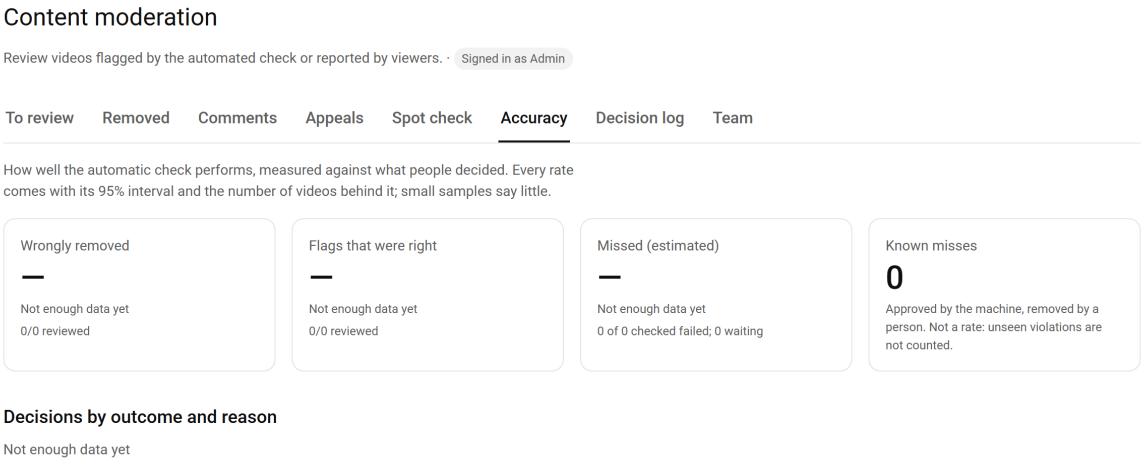


Figure 6.5: *Accuracy tab of the staff console*

6.3 Measurement Method

Every measurement was made on the production environment, with the configuration described in Chapters 4 and 5, and written by its script to a result file. Table 6.2 lists the measurements. The raw values are given in Appendix D, with a map from each table and figure of this chapter to its result file.

Table 6.2: Measurements, their tools and their samples

MEASUREMENT	TOOL AND METHOD	SAMPLE
Time to first frame	Script that performs the requests of a player, in order, and times each	20 requests from Vietnam, 720p
Playback quality	Chromium driven by Playwright plays the real page under a network profile	5 runs of 60 s per profile, after 2 warm-up runs
Bandwidth drop	The same, switching from Fast 3G to Slow 3G after 20 s	5 runs of 90 s, after 2 warm-up runs
Declared and delivered bit rate	Script that reads the playlists and the segment sizes in the bucket	Every segment of three videos
Transcoding time	Script that uploads through the API and follows the video until READY	3 runs per size, one upload at a time
Four-hour video	The same script	1 run
Burst of uploads	The same script, 100 uploads at once over five accounts	1 run of 100 uploads
Damaged uploads	Script that uploads a damaged file and reads where the workflow stopped	2 files, 4 runs
API load	Script that sends read requests in bursts through the Worker, with the function log	40 sequential, bursts of 10 to 80
Prices	AWS Price List Query API [25]	List prices, Singapore

Two choices of method affect how the numbers read. The transcoding times are measured from the moment the upload completes, so they do not depend on the connection of the uploader. The playback runs keep start-up delay and stalling apart, as ITU-T P.1203 does [26]: a stall counts only after the first frame has played, because waiting for a video to start and having it stop are different experiences for a viewer.

6.4 Playback

6.4.1 Time to First Frame

Before a player can show a frame it makes five requests: the playback grant from the API, the master playlist, a rendition playlist, the key from the API and the first segment. Figure 6.6 shows how long each took. The first request after a pause took 1425 ms,

because every cache was cold: the CDN had to fetch the playlists and the segment from the bucket, and the two calls to the API were slower too. The next 19 requests averaged 320 ms with a 95th percentile of 435 ms, which with 19 samples is also the slowest of them (Appendix D). Once the files are cached at the edge, the three CDN requests together take under 50 ms, and the two calls to the API, the grant and the key, make up more than four fifths of the time.

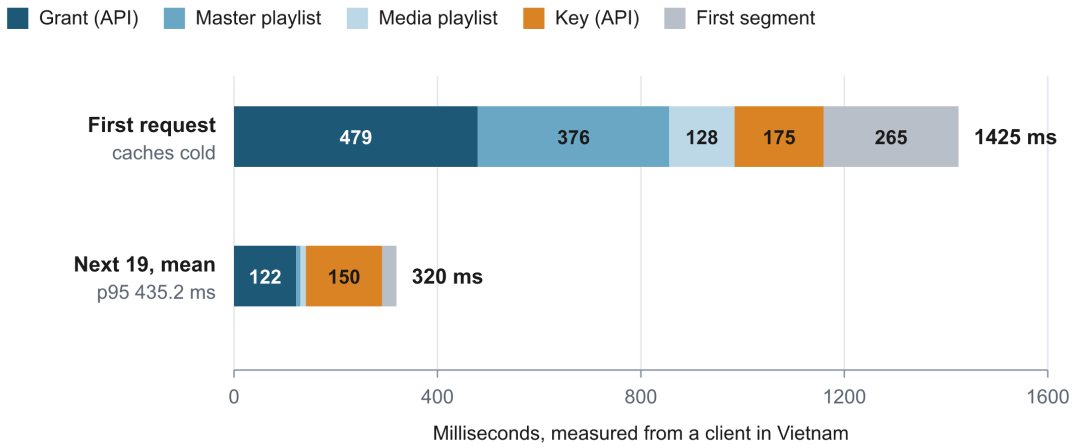


Figure 6.6: *Requests before the first frame of a 720p rendition, by step*

6.4.2 Playback under Limited Bandwidth

Table 6.3 gives the result of playing a 4 minute 40 second recording, encoded in all six rungs, for 60 seconds under three network profiles. The start-up delay is measured from loading the page to the first frame, so it includes loading the web application as well as the five requests above. No run stalled in any profile.

Table 6.3: Playback of the watch page under three network profiles, five runs each

PROFILE	LINK	START-UP, MEDIAN	WORST	STALLS
Unthrottled	Connection of the test machine	2.27 s	3.23 s	0
Fast 3G	1.6 Mbit/s, 150 ms latency	5.87 s	8.89 s	0
Slow 3G	500 kbit/s, 400 ms latency	9.01 s	9.29 s	0

The rung the player settled on shows the two limits of Section 4.3.6 at work. In every run the player first downloaded one 144p segment only to measure the throughput; it neither decoded nor showed that segment, so the first picture already came from the rung

it then kept, and no run changed rung while it played. Unthrottled and on Fast 3G that rung was 480p, not 1080p: in the browser window of the test, 1280 by 720 pixels, the video element is 825 pixels wide, and the player caps its choice at the smallest rung that covers the element, here 480p at 850 pixels. On Slow 3G it was 240p, whose peak of 408 kbit/s fits under the link of 500 kbit/s, and still played without a stall.

A profile that stays the same for a whole run does not show a switch, so five more runs started on Fast 3G and cut the link to Slow 3G twenty seconds into playback, for 90 seconds in all (Table D.4). Every run was showing 480p when the link dropped. The player asked for a lower rung 8.5 to 8.9 seconds later, 240p in four runs and 360p in one, and that rung reached the screen 20.2 to 20.6 seconds after the drop, once the 480p segments already in the buffer had played. Each run changed rung once, and none stalled: the buffer filled on the faster link covered the time the player needed to react.

6.4.3 Declared and Delivered Bit Rate

The player can only choose safely if the playlist is truthful. Table 6.4 sets what the master playlist of the recording declares against what its segments contain. The peak segment bit rate equals the declared `BANDWIDTH` in every rung, and the average equals `AVERAGE-BANDWIDTH`, because both values are measured from the segments when the playlist is written. From 360p up, the peak lies above the video cap plus the audio rate of Table 4.2, by at most 6.6% at 1080p (4933 against 4628 kbit/s), which the buffer and the packaging allow (Section 4.3.3). Every segment lasts between 5.97 and 6.01 seconds, except the last one of each rung. The two other videos measured, both synthetic test patterns, give the same agreement in all six rungs.

Table 6.4: Declared and delivered bit rate of a production recording, in kbit/s (47 segments per rung)

		PEAK		AVERAGE	
RUNG	**SIZE**	**DECLARED**	**DELIVERED**	**DECLARED**	**DELIVERED**
144p	256×144	201	201	154	154
240p	426×240	408	408	249	249
360p	638×360	688	688	420	420
480p	850×480	1178	1178	716	716
720p	1276×720	2443	2443	1578	1578
1080p	1912×1080	4933	4933	3562	3562

6.5 Transcoding

6.5.1 Time by File Size

Figure 6.7 shows how long a video took from the end of its upload to READY, for the four file sizes, and Table 6.5 gives the values. The 36-second clip has 640 by 360 pixels; the other three are 1080p test patterns at 30 frames per second. The two shorter files take the single path, and the two longer ones the chunked path. The sizes in MB and GB are the decimal names of the test files; limits and memory are given in binary units (MiB, GiB) throughout, and AWS bills memory by the GB-hour, which is a GiB-hour.

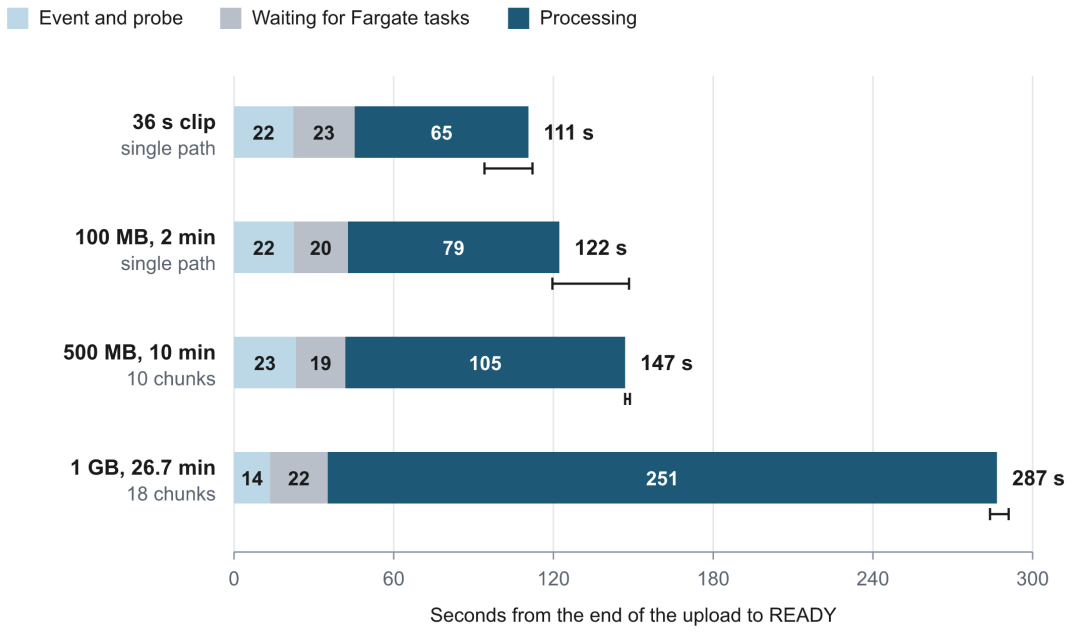


Figure 6.7: *Time from the end of the upload to READY, by file size; the bar is the median run of three and the line below it the range*

Table 6.5: Transcoding time by file size, three runs each

SOURCE	PATH	TO READY, MEDIAN	RANGE	vCPU-s PER SECOND OF SOURCE
36 s clip, 640×360	Single	111 s	94–112 s	4.67
100 MB, 2 min, 1080p	Single	122 s	120–148 s	5.09
500 MB, 10 min, 1080p	10 chunks	147 s	147–149 s	5.80
1 GB, 26.7 min, 1080p	18 chunks	287 s	284–291 s	4.58

Three observations follow. First, 29 to 57 seconds of every run pass before any encoding starts: 7 to 25 seconds from the upload to the end of the probe, then 19 to

35 seconds while Batch starts Fargate tasks. This is the price of having no transcoding capacity running while idle, and it is the same for a clip and for a long video. Second, the chunked path makes time grow far more slowly than the source: the 10-minute file is ready 25 seconds after the 2-minute one, and the 26.7-minute file needs about twice as long as the 10-minute one for nearly three times the content. Third, the work per second of source is nearly constant, between 4.6 and 5.8 vCPU-seconds, whichever path a video takes. The samples differ in two ways that the ratio does not show: the 36-second clip is a 360p source and gets three rungs instead of six, and screening was off for the 1 GB file and the four-hour run (Section 6.5.2), which shortens their analysis job. The range is therefore an estimate for planning, not a constant. The 1080p sources are test patterns; one real recording checks the ratio on harder content. The game capture of Section 6.4.2, 4 minutes 40 seconds at 1912 by 1080 pixels, went through production on 3 November with the earlier task sizes (encode 2 vCPUs, analysis 1 vCPU) and screening on. It needed 1538 vCPU-seconds, 5.50 per second of source, at the top of the measured range; its time from the start of the workflow to READY, 12.7 minutes, is not comparable with Table 6.5, because the encode job now gets four times the vCPUs. This ratio is what makes the cost of Section 6.7 predictable.

6.5.2 A Four-Hour Video

A four-hour 1080p video of 5.26 GiB, a synthetic moving test pattern at 30 frames per second with a tone as its sound, was ready 38.3 minutes after its upload completed. Being larger than the 5 GiB of a single POST, the file also went through the multipart path in production: 169 parts of 32 MiB, uploaded in 132.6 seconds and completed by the API. Figure 6.8 shows where the time went. The probe split it into 48 chunks of five minutes. The audio job encoded the sound of the whole video in 10.8 minutes, and the chunks started only when it finished, because each chunk copies its window of that audio. The 48 chunks then ran twelve at a time, six on Spot and six that overflowed to On-Demand, for 26.5 minutes; a chunk took between 4 and 8 minutes. The video reached READY 24 seconds after the last chunk, of which stitching and publishing took under 3 seconds. The run used 18.7 vCPU-hours, 4.67 vCPU-seconds per second of source, the same ratio as the shorter files. For the 1 GB file and this run, screening was switched off so that test content was not sent to Rekognition; its cost is added by calculation in Section 6.7.

The timeline shows what limits a long video. The audio job is on the critical path for more than a quarter of the time, and the chunks are bounded by the twelve slots that the 48 vCPUs of the two compute environments allow. Neither is a limit of the design: the

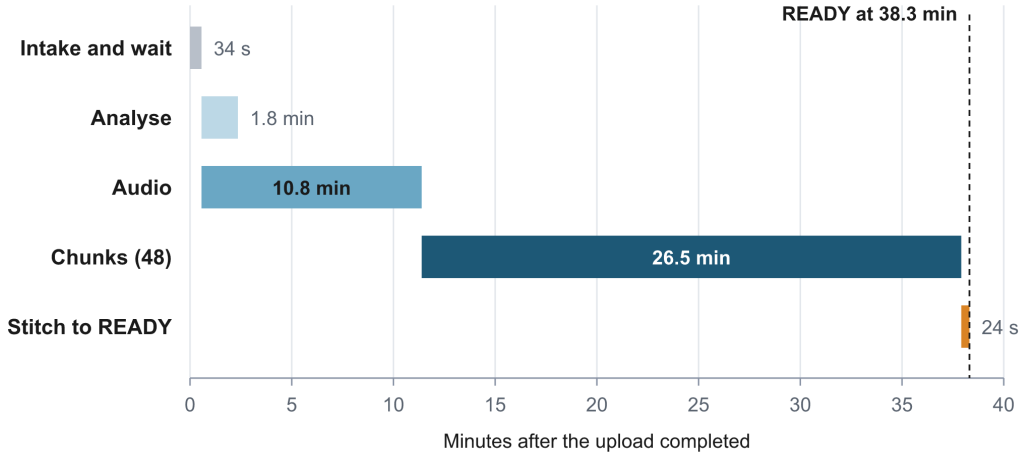


Figure 6.8: *Timeline of a four-hour video on the chunked path*

audio can be encoded in parallel with the chunks, and the slots grow with the capacity granted. The test pattern is also easier to encode than a recording, so a real four-hour recording would need more time per chunk. Chapter 7 returns to these limits.

6.5.3 Damaged Uploads

The runs above only used files that could be transcoded. To test NFR-08, two damaged files were uploaded to production through the same script (Table D.8). The first is half a mebibyte of random bytes named as an MP4 file. The probe step could not read it and failed at once; the workflow retries a function only for service errors, so it did not try again, and the video was marked ERROR 34.9 seconds after its upload. The second is the recording of Section 6.4.2 cut after 140 MB, with its index moved to the front of the file, so that the probe still reads the full 279.9 seconds while the stream ends after about two minutes.

These runs found three defects, each fixed and deployed on 12 November before the last run. First, the error reason stored on the video, which its owner can read, and sent in the alert quoted the presigned address of the source together with the session token of the probe function; the reason now keeps only the message of ffprobe, and the query of any address is removed before it is stored or sent. Second, one run of the truncated file failed at once on an internal error of the Batch API when the encode job was submitted, because that error was not in the retry list; it is now retried like the other errors of the Batch API. Third, the next run of the same file was published as READY: ffmpeg stops without an error where the damaged part begins, so every rung held 19 segments, about 113 seconds, while the video was listed at 279.9 seconds. Packaging now compares each

rung with the duration the probe read and, when it falls short by more than one segment or 1%, fails with the code that Batch does not retry. With that release the truncated file failed after one attempt with exit code 3, before any segment was uploaded, and the video was marked ERROR.

6.6 Behaviour under Load

6.6.1 One Hundred Simultaneous Uploads

One hundred uploads of the 36-second clip were started at the same moment, twenty from each of five accounts. Figure 6.9 shows every video that reached READY, sorted by how long it took. Of the 100 uploads, 92 reached READY and none failed during processing. Eight were refused with 429 when the upload was created, and the client gave up after three retries, so no video existed for them. Three more received 429 on the completion call, after their file had already reached Amazon S3; the event of S3 started their processing all the same, and the execution history shows all three ready. The measurement tool stopped following them at the refusal, so the figure and the times below cover the other 89. Screening was on during this run, and Rekognition answered all 736 frames of the 92 videos, retrying the 68 calls it throttled.

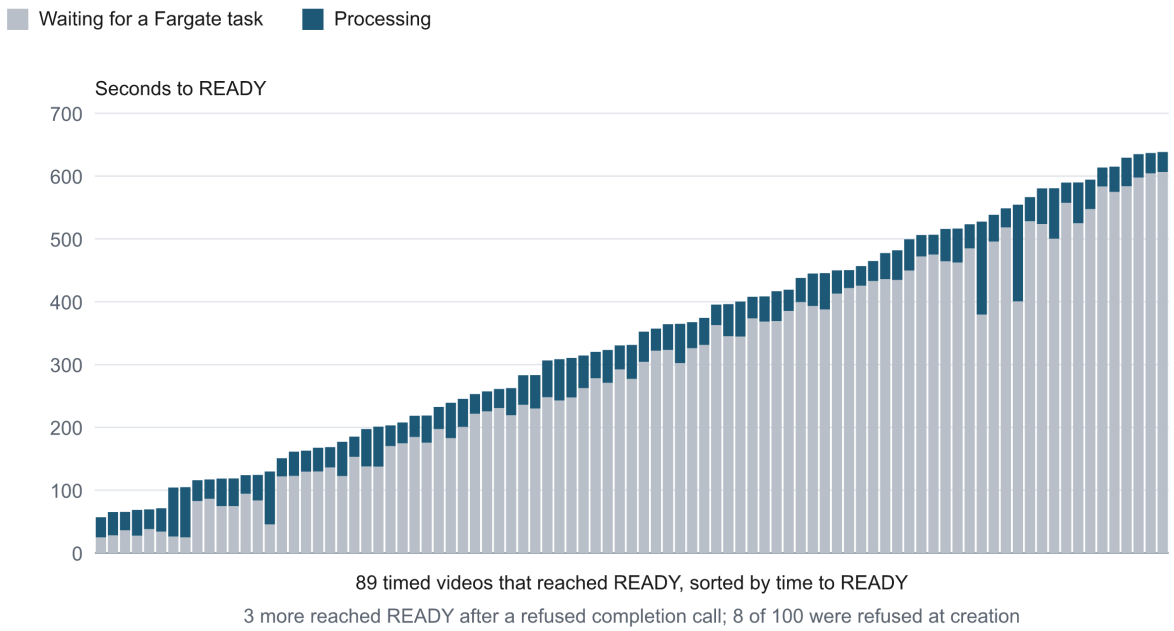


Figure 6.9: *One hundred simultaneous uploads: time to READY of each video, split into waiting and processing*

The figure shows the queue doing its work. Each video needs an analyse task of

two vCPUs and an encode task of four, so the 48 vCPUs of the two environments process about eight videos at a time; the rest wait in the Batch queue instead of failing. Processing itself stayed short, about 30 seconds per task, while the waiting grew steadily to ten minutes for the last video. The median video was ready after 331 seconds and the last after 638 seconds. Both environments were used: 96 tasks ran on Spot and 82 overflowed to On-Demand.

The 11 refusals came from the API, not from the pipeline. The function log shows that all 80 instances were busy in the first minute, Lambda throttled 56 invocations, and no invocation failed. A new instance needs about 1.1 seconds to start, and its first requests waited up to 8.8 seconds on the free database cluster, which slows down above 100 operations per second; the 3.5 seconds that the three retries of the client wait in total were not always enough.

6.6.2 API

Table 6.6 gives the latency of read requests, home feed, video and search, sent through the Worker from Vietnam. A warm instance answered in 156 ms at the median and 263 ms at the 95th percentile, network and Worker included. Every burst of up to 80 simultaneous requests was answered without an error. The latency of a burst is dominated by start-up: 79 of the 193 invocations of the run started a new instance, each taking about 1.1 seconds to start before it handled its request. The phases ran one after another, and the instances of one burst were still warm for the next, so each burst only needed new instances beyond those already running: 9 for the burst of 10, then 10, 20 and 40, which add up to the 79 that Lambda reported. The log gives this total, not a count per burst, but it explains the order of the medians. Nearly every request of the burst of 10 waited for a start, while half of the burst of 20 found a warm instance, so its median lies at about the fastest request that waited; in the larger bursts the new instances also queue longer on the database.

Table 6.6: Latency of read requests through the Worker, in milliseconds

PHASE	REQUESTS	ANSWERED 200	MEDIAN	95TH PERCENTILE
Sequential, warm	40	40	156	263
Burst of 10	10	10	2730	2916
Burst of 20	20	20	1336	2682
Burst of 40	40	40	2547	3382
Burst of 80	80	80	4395	5663

6.7 Cost

The cost model of Section 2.2.1 assumes a month of 300 uploads of five minutes each and 5.5 vCPU-seconds of work per second of source, the upper part of the range measured in Table 6.5. That is 137.5 vCPU-hours a month. Every task uses 2 GiB of memory per vCPU, so at the Fargate On-Demand list prices of 0.05056 USD per vCPU-hour and 0.00553 USD per GB-hour, one vCPU-hour costs 0.06162 USD. The alternative is an always-on c6i.2xlarge instance with eight vCPUs and 16 GiB, at 0.392 USD an hour for 730 hours. Screening sends one frame every five seconds to Rekognition, 60 frames for a five-minute video, at 0.0013 USD a frame [27]. Table 6.7 gives the result.

Table 6.7: Monthly cost of processing 300 five-minute videos, list prices in Singapore

ITEM	FARGATE, PER USE	EC2, ALWAYS ON
Compute: 137.5 vCPU-hours of work	8.47 USD	286.16 USD
Screening: 18 000 frames	23.40 USD	23.40 USD
Total	**31.87 USD**	**309.56 USD**

Paying per use costs a tenth of the always-on server, and the reason is utilisation: the work of the month fills only 2.4% of the hours that the server is paid for. Three points qualify the comparison. The Fargate figure uses On-Demand prices, so it is an upper bound: most tasks run on Spot, which is cheaper. The always-on server would have one fixed capacity of eight vCPUs, so a burst like that of Section 6.6.1 would wait longer on it, not shorter. Storage and delivery are left out, because they cost the same in both designs. With compute this cheap, screening becomes the largest item; its cost grows with the number of frames sampled, which is why the sampling interval widens for long videos (Section 4.8.1).

A smaller always-on server narrows the gap without closing it. At the prices of Table D.12, a c6i.xlarge with four vCPUs costs 143.08 USD a month for compute and a t3.medium with two burstable vCPUs 38.54 USD, against 8.47 USD per use. Both would still be idle most of the month, and with a half or a quarter of the vCPUs each video would take two to four times as long, so a burst would queue longer still. The serverless design also pays for parts that a server does not need: Step Functions state transitions, the invocations of the workflow functions, SQS messages and the work bucket. At 300 videos a month they stay well under 0.50 USD at list prices, so they do not change the comparison.

6.8 Evidence for the Non-Functional Requirements

Table 6.8 sets each non-functional requirement of Table 3.5 against the evidence of this chapter and of Chapter 5.

Table 6.8: Non-functional requirements and their evidence

NFR	EVIDENCE	RESULT
NFR-01 Playback start	Once cached, the requests before the first frame take 435 ms at the 95th percentile (Figure 6.6). Unthrottled, playback started after 2.27 s at the median, with no stall; two of the five runs took longer than 3 s, 3.10 and 3.23 s (Table D.3)	Met at the median (2.27 s); two runs of five above 3 s
NFR-02 Processing time	A 10-minute video was ready in 147 to 149 s (Table 6.5); a four-hour video in 38.3 minutes (Figure 6.8)	Met
NFR-03 Burst load	Of 100 simultaneous uploads, 92 reached READY, including 3 whose completion call was refused; the 8 refused at creation received 429 (Section 6.6.1)	Met
NFR-04 Idle cost	With nothing processing, both compute environments want 0 vCPUs and no task runs; the API, the workflow and the two scheduled functions (an hourly roll-up and a daily reconciler) are billed per invocation	Met
NFR-05 API latency	156 ms at the median when warm; 80 of 80 simultaneous requests answered (Table 6.6)	Met
NFR-06 Content security	A request to the CDN without a policy is refused with 403; the tests of Section 5.7 refuse the key for another video, version, origin or a missing token	Met
NFR-07 API security	A request to the function that skips the Worker, and a preflight from another origin, are refused with 403; sign-in is rate-limited	Met
NFR-08 Reliability	Retries and the 24-hour limit are part of the workflow (Section 4.2.2). Two damaged files uploaded to production ended in ERROR without useless retries, after fixes for a truncated file published as ready and an unretried Batch API error (Section 6.5.3)	Met after two fixes
NFR-09 Reproducibility	Staging and production are built by the same Terraform modules (Section 5.4); only the values of secrets are written outside the code	Met
NFR-10 Maintainability	The security jobs are required checks of the main branch (Section 5.5)	Met

Chapter 7

Conclusion and Future Work

This chapter sets the results against the objectives of Section 1.2, then states the strengths and the limitations of the delivered system and the work that would extend it.

7.1 Summary of Achievements

The project set out to build a video sharing platform whose transcoding runs on serverless containers driven by events, and to measure what that choice costs and delivers. Table 7.1 sets each of the five objectives against its result and the part of the report that holds the evidence. All five were reached on the production system.

Table 7.1: Objectives of the project against their results

OBJECTIVE	RESULT	EVIDENCE
1. Web platform	Users register with a verified e-mail address or Google, upload files of up to 20 GiB straight to Amazon S3 and watch every video at up to six quality levels. Each video is public, unlisted, private, protected by a password or reserved for members. Channels, subscriptions, comments, playlists, memberships, a search that ignores diacritics and a staff console complete the platform.	Sections 4.6, 4.3.5, 6.2
2. Event-driven pipeline	The creation of the source object alone starts a Step Functions execution, named after the video and its upload version so that a duplicate event never starts a second one. The execution ends with an HLS package encrypted with AES-128 and served through CloudFront with a signed policy. In a burst of 100 uploads, every upload whose file reached Amazon S3 became READY.	Sections 4.2, 4.3, 6.5
3. On-demand containers	Every FFmpeg job runs in a Fargate task that exists only while the job runs; with nothing to process, both compute environments hold zero vCPUs. Spot capacity is used first and On-Demand as overflow, which a burst of 100 uploads used. A four-hour video was ready in 38.3 minutes.	Sections 4.2.3, 6.5.2, 6.6.1
4. Infrastructure as Code and CI	Eleven Terraform modules build three environments on AWS and Cloudflare. Two workflows lint, type-check, test, scan for secrets and vulnerable dependencies, and run FFmpeg in the image on every change; 305 unit tests, 13 image checks and 27 end-to-end tests pass.	Sections 5.4, 5.5, 5.7
5. Empirical evaluation	Playback start, playback under limited bandwidth, declared against delivered bit rate, transcoding time from a 36-second clip to a four-hour video, 100 simultaneous uploads, API load and monthly cost were measured on production. Nine of the ten non-functional requirements are met and the tenth at its median.	Chapter 6, Table 6.8

7.2 Strengths of the System

The processing tier costs nothing while idle and scales with the work. A transcoding container exists only while its job runs, so the month of Section 6.7 costs 31.87 USD against 309.56 USD for an always-on server, and a burst of uploads is absorbed by the queue instead of failing: in the burst of Section 6.6.1, no video failed once its file had

reached Amazon S3.

The work grows with the content, not with the path. Every measured video needed between 4.6 and 5.8 vCPU-seconds per second of source, from a 36-second clip to a four-hour video, so both the cost and the capacity needed for a given catalogue can be predicted from its length. The range is an estimate rather than a constant: the clip had three rungs instead of six, and screening was off for the two largest runs. Three of the four sources were test patterns, which are easy to encode; the one real 1080p recording measured needed 5.50 vCPU-seconds per second, at the top of this range, and content that is harder still could sit above it.

The output is honest to the player. Every segment of a rung starts with a key frame at the same instant, and the declared bandwidth of every rung equals the measured peak of its segments (Section 6.4.3). The player therefore switches without a gap and chooses a rung it can sustain: on a link of 500 kbit/s it settled on the rung that fits and played without a stall.

The application tier never handles video bytes. The API issues grants, records metadata and answers queries, while video moves directly between the browser, Amazon S3 and CloudFront. The load on the API does not depend on the size of the files, and the processing containers hold no database credentials, so one function owns every processing state and a fault in a container cannot corrupt a record.

7.3 Limitations

Playback. The player relies on Media Source Extensions, because every request for a playlist or segment must carry the signed policy as query parameters and the native HLS player of a browser cannot add them. A browser without Media Source Extensions, such as Safari on older iPhones, therefore cannot play. Signed Cookies would avoid this, but browsers block third-party cookies inside an embedded player, which the platform supports. AES-128 keeps the files useless without a grant, but it is not digital rights management: a viewer who may play a video can also save it.

Transcoding. The ladder has six fixed rungs whatever the content. On a long video the audio job sits on the critical path, 10.8 of the 38.3 minutes of the four-hour run, and the chunks are bounded by the twelve slots that the capacity of the two compute environments allows. That run used a synthetic test pattern, which is easier to encode than a recording. Every video also waits 19 to 35 seconds for Fargate tasks to start, about a quarter of the time of a short clip (Section 6.5.1).

Load. The free MongoDB cluster slows down above 100 operations per second.

During the burst of 100 uploads, the first requests of new API instances waited up to 8.8 seconds on it, longer than the three retries of the client, so 8 uploads were refused at creation although the pipeline had capacity to queue them. Start-up on an unthrottled connection met its target of three seconds at the median (2.27 s) but not in two of five runs, which took 3.10 and 3.23 s.

Moderation. Screening samples one frame every five seconds and at most 120 frames, so content shown between two samples can pass. On a long video the gap widens: a four-hour video is sampled once every two minutes. The sound is not screened at all, and the thresholds of 60 and 90 per cent follow the provider's guidance rather than a calibration on this platform's content. The console measures the error rates from human decisions, but too few decisions exist yet for a rate to mean anything, so this report makes no claim about the accuracy of the screening.

Operation. Deployment is run by the operator rather than by the pipeline, so a change reaches production only as often as someone runs the steps, and a step left out is noticed only by the checks that follow it. The whole platform also runs in a single AWS region. CloudFront serves the web application from the edge, but an outage of that region stops uploads, processing and playback alike, because every playback grant and key comes from the API in that region.

7.4 Future Work

The first extension is a bitrate ladder chosen per video. Encoding short probes of the source at several operating points and keeping those that give the best quality for their bit rate, measured with a perceptual metric such as Video Multi-Method Assessment Fusion (VMAF), lowers delivery bandwidth at the same perceived quality; streaming services report savings of this kind from per-title and per-shot encoding [3, 20].

The second is a shorter critical path for long videos. Encoding the audio in windows that overlap the chunks, with the same packet handling at the boundaries that the chunks already use, would remove the wait for the audio job, and a larger capacity grant would raise the number of chunk slots without any change to the design.

The third is a fallback for browsers without Media Source Extensions. The API could return playlists in which every address carries its own signature, which the native player of the browser can follow without adding parameters, while other browsers keep the current path.

The fourth concerns load. A longer back-off with jitter on the client, or a client-side queue for uploads, would let a refused upload wait instead of failing, and a paid database

tier would remove the limit of 100 operations per second.

The fifth is calibrating the screening from evidence. The console already records the reason codes of reversed decisions and an unbiased audit sample; after a few hundred decisions, the thresholds and the label policy can be adjusted and the effect measured again. Further ahead, a deployment workflow that assumes a role in the account would make releases routine, and serverless encoding on graphics processing units (GPU) [28] and a codec such as High Efficiency Video Coding (HEVC) or AV1 next to H.264 would raise throughput and reduce bandwidth.

7.5 Task Distribution

Table 7.2 shows how the work was divided between the two members of the team. Each member was responsible for the parts listed and for the chapters of the report that describe them.

Table 7.2: Task distribution between the members of the team

MEMBER	**STUDENT ID**	**RESPONSIBILITIES**
Dinh Quoc Cuong	523H0008	Overall architecture; the transcoding workflow, the container image and the chunked path; the infrastructure in Terraform on AWS and Cloudflare; the CI pipeline and the security scans; the measurements and the evaluation; Chapters 4 to 6 of the report
Vo Huynh Minh Duc	523H0014	The web application (catalogue, watch page and player, upload, studio, channels, staff console, two languages); the account, community and library parts of the API; testing and the diagrams; Chapters 1 to 3 and 7 of the report

References

[1] R. Pantos and W. May, "RFC 8216: HTTP live streaming." https://datatracker.ietf.org/doc/html/rfc8216, 2017. Internet Engineering Task Force.

[2] Netflix Technology Blog, "Rebuilding netflix video processing pipeline with microservices." https://netflixtechblog.com/rebuilding-netflix-video-processing-pipeline-with-microservices-4e5e6310e359, 2024. Accessed: 2026-08-10.

[3] Netflix Technology Blog, "Per-title encode optimization." https://netflixtechblog.com/per-title-encode-optimization-7e99442b62a2, 2015. Accessed: 2026-08-10.

[4] Amazon Web Services, "Video on demand on AWS: Implementation guide." https://docs.aws.amazon.com/solutions/latest/video-on-demand-on-aws/, 2026. Accessed: 2026-08-10.

[5] FFmpeg Team, "FFmpeg documentation: HLS muxer." https://ffmpeg.org/ffmpeg-formats.html#hls-2, 2026. Accessed: 2026-08-10.

[6] W. A. Moina-Rivera, M. Garcia-Pineda, J. M. Claver, and J. Gutierrez-Aguado, "Event-driven serverless pipelines for video coding and quality metrics," *Journal of Grid Computing*, vol. 21, no. 2, p. 20, 2023.

[7] M. Zhang, Y. Zhu, C. Zhang, and J. Liu, "Video processing with serverless computing: A measurement study," in *Proceedings of the 29th ACM Workshop on Network and Operating Systems Support for Digital Audio and Video (NOSSDAV)*, 2019.

[8] Google, "YouTube recommended upload encoding settings." https://support.google.com/youtube/answer/1722171, 2026. Accessed: 2026-11-04.

[9] Vimeo, Inc., "Video and audio compression guidelines." https://help.vimeo.com/hc/en-us/articles/12426043233169-Video-and-audio-compression-guidelines, 2026. Accessed: 2026-11-04.

[10] Netflix Technology Blog, "The making of VES: the cosmos microservice for netflix video encoding." https://netflixtechblog.com/the-making-of-ves-the-cosmos-microservice-for-netflix-video-encoding-946b9b3cd300, 2024. Accessed: 2026-08-10.

[11] Amazon Web Services, "Compute environments on Fargate." https://docs.aws.amazon.com/batch/latest/userguide/fargate-compute-environments.html, 2026. Accessed: 2026-11-04.

[12] Amazon Web Services, "Lambda quotas." https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html, 2026. Accessed: 2026-11-04.

[13] M. Golec, G. K. Walia, M. Kumar, F. Cuadrado, S. S. Gill, and S. Uhlig, "Cold start latency in serverless computing: A systematic review, taxonomy, and future directions," *ACM Computing Surveys*, vol. 57, no. 3, 2025. Published version of arXiv:2310.08437.

[14] Amazon Web Services, "Serverless applications lens: AWS well-architected framework." https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/welcome.html, 2022. Accessed: 2026-11-04.

[15] Amazon Web Services, "What is Step Functions?" https://docs.aws.amazon.com/step-functions/latest/dg/welcome.html, 2026. Accessed: 2026-11-04.

[16] Amazon Web Services, "Run AWS Batch workloads with Step Functions." https://docs.aws.amazon.com/step-functions/latest/dg/connect-batch.html, 2026. Accessed: 2026-11-04.

[17] Amazon Web Services, "Array jobs." https://docs.aws.amazon.com/batch/latest/userguide/array_jobs.html, 2026. Accessed: 2026-11-04.

[18] Apple Inc., "HTTP Live Streaming (HLS) authoring specification for Apple devices." https://developer.apple.com/documentation/http-live-streaming/hls-authoring-specification-for-apple-devices, 2026. Accessed: 2026-11-04.

[19] video-dev, "HLS.js: JavaScript HLS client using media source extensions (originally created by dailymotion)." https://github.com/video-dev/hls.js, 2026. Accessed: 2026-08-10.

[20] Netflix Technology Blog, "Dynamic optimizer: A perceptual video encoding optimization framework." https://netflixtechblog.com/dynamic-optimizer-a

-perceptual-video-encoding-optimization-framework-e19f1e3a277f, 2018. Accessed: 2026-08-10.

[21] MongoDB, Inc., "MongoDB Search overview." https://www.mongodb.com/docs/atlas/atlas-search/, 2026. Accessed: 2026-11-04.

[22] Amazon Web Services, "Serve private content with signed URLs and signed cookies: Amazon cloudfront developer guide." https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/PrivateContent.html, 2026. Accessed: 2026-08-10.

[23] P. A. Grassi *et al.*, "Digital identity guidelines: Authentication and lifecycle management," NIST Special Publication 800-63B, National Institute of Standards and Technology, 2017. Includes updates as of 2020-03-02. https://doi.org/10.6028/NIST.SP.800-63b. Accessed: 2026-10-16.

[24] Amazon Web Services, "Moderating content, Amazon Rekognition developer guide." https://docs.aws.amazon.com/rekognition/latest/dg/moderation.html, 2026. Accessed: 2026-10-15.

[25] Amazon Web Services, "Calling AWS services and prices using the AWS price list." https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/price-changes.html, 2026. Accessed: 2026-11-05.

[26] ITU-T, "Recommendation P.1203: Parametric bitstream-based quality assessment of progressive download and adaptive audiovisual streaming services over reliable transport." https://www.itu.int/rec/T-REC-P.1203, 2017. Accessed: 2026-11-05.

[27] Amazon Web Services, "Amazon Rekognition pricing." https://aws.amazon.com/rekognition/pricing/, 2026. Accessed: 2026-11-05.

[28] A. Salcedo-Navarro, R. Peña-Ortiz, J. M. Claver, M. Garcia-Pineda, and J. Gutierrez-Aguado, "Towards GPU-enabled serverless cloud edge platforms for accelerating HEVC video coding," *Cluster Computing*, vol. 28, p. 68, 2025.

Appendix A

Complete REST API Reference

This appendix lists every route of the API, grouped by the feature that registers it; Table 4.4 in Section 4.5 gives the same groups in brief. Every path below is served under the prefix `/v1`, reached through the Cloudflare Worker, and every request and response body is JSON. An error always has the shape `{error:{code,message}}`, where the code is a stable identifier such as `EMAIL_NOT_VERIFIED`, `VIDEO_UNDER_REVIEW` or `RATE_LIMITED` that the interface reacts to, and the message is text for the viewer. Listings are paged with an opaque cursor.

The access column uses five levels. *Public* routes answer without a session, under the access rule of Section 4.3.5 wherever a video is involved. *Signed in* routes need a bearer token, and where a route changes a resource, the API also checks that the requester owns it. *Playback token* routes need the short token issued with a playback grant, so that an embedded player can use them without a session. A permission name, such as `moderation:review`, means a staff role that holds it (Table 4.5). Routes marked *limited* have their own rate limit on top of the global one of 300 requests a minute.

Table A.1: Routes of the API

ENDPOINT	METHOD	ACCESS	DESCRIPTION
Health			
`/health`, `/v1/health`	GET	Public	Liveness of the function; only the second needs the gateway secret
Accounts			
`/auth/register`	POST	Public, limited	Create an account and send the verification link
`/auth/login`	POST	Public, limited	Sign in with e-mail and password

ENDPOINT	METHOD	ACCESS	DESCRIPTION
`/auth/google`	POST	Public, limited	Sign in or register with a Google identity token
`/auth/link-google`	POST	Signed in	Link a Google identity to the account
`/auth/verify-email`	POST	Public, limited	Spend a verification link
`/auth/resend-verification`	POST	Signed in, limited	Send a new verification link
`/auth/forgot-password`	POST	Public, limited	Send a password reset link
`/auth/reset-password`	POST	Public, limited	Set a new password with a reset link
`/auth/change-password`	POST	Signed in, limited	Change the password with the current one
`/auth/logout`	POST	Signed in	Revoke every session of the account
`/auth/me`	GET	Signed in	Current account and role
Uploads			
`/uploads`	POST	Signed in, limited	Create a draft video and a POST or multipart upload grant; needs a verified e-mail address
`/uploads/:id/parts`	POST	Signed in	Sign the URLs of further parts of a multipart upload
`/uploads/:id/complete`	POST	Signed in	Complete the upload from the part list S3 holds
`/uploads/:id/abort`	POST	Signed in	Abort the upload and delete the draft
Videos and playback			
`/videos`	GET	Public	Public, ready and approved videos, newest first
`/videos/:id`	GET	Public	Details of a video the requester may see
`/videos/:id/unlock`	POST	Public, limited	Exchange the password of a video for an unlock token
`/videos/:id/playback`	GET	Public	Master playlist address, signed query policy and playback token
`/videos/:id/key`	GET	Playback token	The 16-byte AES key of the upload version
`/videos/:id/view`	POST	Playback token, limited	Count a view once 30 seconds have passed since the playback token was issued

ENDPOINT	METHOD	ACCESS	DESCRIPTION
`/videos/:id`	PATCH	Signed in	Edit title, description, tags, category, visibility, password, audience, tier and violence warning
`/videos/:id`	DELETE	Signed in	Delete the video, its comments and its files
Community			
`/videos/:id/reaction`	GET, PUT	Signed in	Read or set the like or dislike of the requester
`/videos/:id/comments`	GET	Public	Comments of a video, pinned first
`/videos/:id/comments`	POST	Signed in, limited	Post a comment or a reply
`/comments/:id/replies`	GET	Public	Replies to a comment
`/comments/:id`	DELETE	Signed in	Delete a comment of the requester or on the requester's video
`/comments/:id/pin`	PUT, DELETE	Signed in	Pin or unpin a comment on the requester's video
`/comments/:id/report`	POST	Signed in, limited	Report a comment with a listed reason
Channels and memberships			
`/channels/:ref`	GET	Public	Channel by identifier or handle
`/channels/:ref/videos`	GET	Public	Public videos of a channel
`/channels/:ref/subscription`	PUT, DELETE	Signed in, limited	Subscribe or unsubscribe
`/channels/:ref/membership`	PUT, DELETE	Signed in	Join a membership tier or leave it
`/me/subscriptions`	GET	Signed in	Channels the requester follows
`/me/tiers`	POST	Signed in	Create a membership tier
`/me/tiers/:id`	PATCH, DELETE	Signed in	Edit or retire a tier
`/me/members`	GET	Signed in	Members of the requester's channel
`/me/profile`	PATCH	Signed in	Edit display name, handle, description and the e-mail preference
Library			
`/videos/:id/heartbeat`	POST	Playback token	Record watch time and the resume position

ENDPOINT	METHOD	ACCESS	DESCRIPTION
`/me/progress/:videoId`	GET	Signed in	Resume position of a video
`/me/history`	GET, DELETE	Signed in	Watch history, or clear it
`/me/history/:videoId`	DELETE	Signed in	Remove one video from the history
`/me/feed`	GET	Signed in	New videos of subscribed channels
`/me/playlists`	GET, POST	Signed in, limited	List or create playlists
`/playlists/:id`	GET	Public	A public playlist or one of the requester
`/playlists/:id`	PATCH, DELETE	Signed in	Rename, change visibility or delete
`/playlists/:id/items`	POST	Signed in	Add a video
`/playlists/:id/items/:videoId`	DELETE	Signed in	Remove a video
`/playlists/:id/order`	PUT	Signed in	Reorder with the full list
Studio			
`/me/videos`	GET	Signed in	Every video of the requester with its processing state
`/images/uploads`	POST	Signed in, limited	Grant for an avatar, banner or thumbnail image
`/me/avatar`, `/me/banner`	PUT	Signed in	Screen and set the avatar or banner
`/videos/:id/thumbnail`	PUT, DELETE	Signed in	Set or remove a custom thumbnail
`/videos/:id/captions/:lang`	PUT, DELETE	Signed in	Upload or remove a WebVTT caption track
`/videos/:id/chapters`	PUT	Signed in	Save the chapter list
`/videos/:id/schedule`	PUT, DELETE	Signed in	Set, move or cancel the publication time
`/videos/:id/analytics`	GET	Signed in	Daily views and watch time of one video
`/me/analytics`	GET	Signed in	Totals of the channel
Discovery and notifications			
`/search`	GET	Public, limited	Full-text search with filters
`/search/suggest`	GET	Public, limited	Title suggestions after two characters
`/videos/:id/related`	GET	Public	Related videos
`/me/notifications`	GET	Signed in	Inbox with the unread count
`/me/notifications/read`	POST	Signed in	Mark notifications as read

ENDPOINT	METHOD	ACCESS	DESCRIPTION
Moderation and administration			
`/videos/:id/report`	POST	Signed in, limited	Report a video with a listed reason
`/videos/:id/appeal`	GET, POST	Signed in	Read or file the owner's appeal
`/admin/stats`	GET	`moderation:review` or `metrics:read`	Counts shown on the console tabs
`/admin/queue`	GET	`moderation:review`	Videos held or reported, or with `tab=removed` the videos the machine removed
`/admin/videos/:id/reports`	GET	`moderation:review`	Reports of one video
`/admin/videos/:id/decision`	POST	`moderation:review`	Approve or remove with a reason code
`/admin/comments`	GET	`moderation:review`	Reported comments
`/admin/comments/:id/decision`	POST	`moderation:review`	Remove a comment or dismiss its reports
`/admin/appeals`	GET	`moderation:appeals`	Open appeals
`/admin/appeals/:id/decision`	POST	`moderation:appeals`	Uphold or reject an appeal
`/admin/audit/sample`	POST	`moderation:sample`	Draw an audit sample of machine-approved videos
`/admin/audit`	GET	`moderation:audit`	Pending audit samples
`/admin/audit/:id`	POST	`moderation:audit`	Label a sample pass or fail
`/admin/metrics`	GET	`metrics:read`	Error rates with their Wilson intervals
`/admin/events`	GET	`events:read`	Decision log
`/admin/role-changes`	GET	`events:read`	Log of role changes
`/admin/users`	GET	`users:manage`	Search accounts
`/admin/users/:id/role`	PUT	`users:manage`	Change the role of an account

Appendix B

Environment Variables Reference

Terraform sets every variable below when it creates the functions and the job definitions, so none is typed by hand in a deployed environment. Values that are secret are not environment variables at all: the API and the workflow functions read them from AWS Secrets Manager at start-up, and only the identifier of the secret is passed in the environment. Table B.1 lists the variables by the part that reads them, and Table B.2 the fields of the secret of the API.

Table B.1: Environment variables by component

VARIABLE	REQUIRED	DESCRIPTION
API function		
`APP_ENV`	Yes	*local*, *test*, *staging* or *prod*
`API_SECRET_ARN`	Yes	Secret that holds the fields of Table B.2
`DB_NAME`	Yes	MongoDB database of the environment
`WEB_ORIGINS`	Yes	Comma-separated origins allowed by CORS
`WEB_BASE_URL`	Yes	Public address of the web application, used in e-mail links
`API_BASE_URL`	Yes	Public address of the API without `/v1`, written into key addresses
`CDN_BASE_URL`	Yes	Public address of the media distribution
`RAW_BUCKET`, `PROCESSED_BUCKET`, `KEYS_BUCKET`	Yes	Buckets of uploads, of the HLS packages and of the keys

VARIABLE	REQUIRED	DESCRIPTION
TABLE_RATE_LIMITS, TABLE_VIEW_MARKS, TABLE_WATCH_PROGRESS, TABLE_HEARTBEATS, TABLE_NOTIFICATIONS	Yes	The five DynamoDB tables of Table 4.3
REQUIRE_ORIGIN_SECRET	No	Defaults to true; may be false only in *local* and *test*
MONGO_POOL_SIZE	No	Connections per instance, 1 to 10; defaults to 3
UPLOAD_MAX_BYTES	No	Upload ceiling; an invalid value falls back to 20 GiB and never raises it
SCHEDULE_GROUP, SCHEDULE_TARGET_ARN, SCHEDULE_ROLE_ARN	No	EventBridge Scheduler group, target and role of scheduled publishing; unset disables scheduling
Workflow functions		
STATE_MACHINE_ARN	Yes	*starter*: the state machine it starts
WORK_BUCKET	Yes	*probe*, *status*, *stitch*: bucket of plans and chunk results
CHUNK_PARALLEL_SLOTS	Yes	*probe*: chunks that run at once, used to size the chunks
FFPROBE_PATH	Yes	*probe*: path of *ffprobe* in the Lambda layer
MONGODB_SECRET_ARN, DB_NAME	Yes	*status*: connection string secret and database
KEYS_BUCKET	Yes	*status*, *stitch*: bucket of the AES keys
EVENT_BUS_NAME	Yes	*status*: bus on which the published event is sent
ALERT_TOPIC_ARN	Yes	*status*: topic of failure alerts
PROCESSED_BUCKET, API_BASE_URL	Yes	*stitch*: bucket of the packages and base of the key address
API_SECRET_ARN, DB_NAME	Yes	*notifier*, *rollup*, *reconciler*: database access through the API secret
TABLE_NOTIFICATIONS, TABLE_RATE_LIMITS	Yes	*notifier*: inbox table and the daily mail counter
DAILY_MAIL_CAP	No	*notifier*: e-mails per day across subscribers; defaults to 400
WEB_BASE_URL	Yes	*notifier*: address used in e-mail links
TABLE_HEARTBEATS	Yes	*rollup*: table of viewing heartbeats
RAW_BUCKET	Yes	*reconciler*: bucket whose abandoned drafts are removed
Transcoder jobs		
VIDEO_ID, UPLOAD_VERSION	Yes	Video and upload version the job works on
RAW_BUCKET, RAW_KEY	Yes	Location of the source
PROCESSED_BUCKET, WORK_BUCKET, KEYS_BUCKET	Yes	Output, plans and chunk results, and the key of the upload version

VARIABLE	REQUIRED	DESCRIPTION
EVENT_BUS_NAME	Yes	Bus that receives progress events
API_BASE_URL	Yes	*encode*, *chunk*: base of the key address written into playlists
JOB_TIMEOUT_SECONDS	Yes	*encode*: budget the encode guard projects against
MODERATION_ENABLED	No	*analyse*: screening runs unless this is false
AWS_BATCH_JOB_ARRAY_INDEX, AWS_BATCH_JOB_ATTEMPT	Set by Batch	*chunk*: index of the chunk and number of the attempt
Web application, at build time		
VITE_API_BASE_URL	Yes	Address of the API the build talks to
VITE_GOOGLE_CLIENT_ID	No	Google client identifier; without it the Google button is hidden
Terraform variables of an environment		
transcoder_image_tag	Yes	Commit of the transcoder image; no default in production
attach_hosts	No	Puts the hosts on the distributions; false only while hosts move
moderation_enabled	No	False only during measurement runs
alert_emails	No	Addresses subscribed to the alert topic
cdn_transition_reader_arns, web_transition_reader_arns	No	Distributions that may still read the buckets while hosts move

Table B.2: Fields of the API secret in AWS Secrets Manager

FIELD	REQUIRED	DESCRIPTION
mongodbUri	Yes	Connection string of the Atlas cluster
jwtSecret	Yes	Key of at least 32 bytes that signs access, playback and unlock tokens
originSecret	Yes	Value the Worker sends as X-Origin-Verify, at least 32 bytes
cloudfrontKeyPairId, cloudfrontPrivateKey	No	Key pair that signs playback policies; without it listings carry no images and nothing plays
googleClientId	No	Audience checked on Google identity tokens
smtpUser, smtpPassword	No	Mail account for verification, reset and notification e-mail

At minimum, the API needs the variables marked as required and the three required fields of its secret. Playback needs the CloudFront key pair, Google sign-in needs the client identifier on both the API and the web build, and e-mail needs the mail account. The pipeline runs without any of the optional values; screening is on unless it is switched off explicitly.

Appendix C

Installation and Execution Guide

C.1 Prerequisites

Running the platform needs an AWS account, a domain whose DNS is hosted on Cloudflare and a MongoDB Atlas cluster. Building it needs Node.js 22, Docker, Terraform 1.14 and the AWS command-line tools, with credentials of the AWS account in the default profile and a Cloudflare token exported as `CLOUDFLARE_API_TOKEN`. The installation of these tools is not described here.

C.2 Installation Steps

The repository is one set of npm workspaces, so one command installs every part, and the build bundles the API, the functions, the transcoder and the web application.

```
npm ci
npm run build
npm run layer:ffprobe -w @zelos/functions   # ffprobe for the probe layer,
    needs Docker
docker build -f transcoder/Dockerfile -t zelos-transcoder .
```

C.3 Running Tests

`npm run check` runs the linter with the security rules, the type checks and the 305 unit tests. The smoke test of the image builds its *smoke* stage and runs FFmpeg on generated sources. The end-to-end suite runs against a deployed environment and signs in with test accounts read from a local file named by `E2E_ACCOUNTS`.

```
npm run check
docker build -f transcoder/Dockerfile --target smoke -t zelos-smoke . &&
    docker run --rm zelos-smoke
E2E_BASE_URL=<web origin> E2E_API_URL=<api origin> E2E_ACCOUNTS=<file> \
  npx playwright test -c e2e/playwright.config.ts
```

C.4 Production Deployment

The shared environment, which holds the network and the image registry, is applied once. The image is then pushed with its commit as tag, and the environment is applied with that tag.

```
terraform -chdir=infra/envs/shared init && terraform -chdir=infra/envs/shared
    apply
docker tag zelos-transcoder <registry>/zelos-transcoder:<commit> && docker
    push <registry>/zelos-transcoder:<commit>
terraform -chdir=infra/envs/prod init
terraform -chdir=infra/envs/prod apply -var transcoder_image_tag=<commit>
```

After the first apply, the fields of Table B.2 are written into the secret of the API and the connection string into the secret of the workflow. `scripts/ensure-indexes.sh prod` creates the database and search indexes, and runs again after every apply that changes the API code. The Cloudflare Worker is deployed with `npx wrangler deploy -env prod` after its target host and origin secret are set, and `scripts/deploy-web.sh prod` uploads the web application and refreshes its distribution. The deployment is checked by playing a video through the CDN host and by running the end-to-end suite against the production origins.

Appendix D

Measurement Data

This appendix gives the raw values behind the results of Chapter 6. Each measurement script writes a result file to `docs/results/runs`, and two scripts in `docs/diagrams/svg` read those files: one builds the data of the charts and the other the tables below, so a number in a chart and the same number here come from one record. Table D.1 maps each table and figure of Chapter 6 to its result file, its script and its table in this appendix.

Table D.1: Sources of the tables and figures of Chapter 6

IN CHAPTER 6	RESULT FILE	SCRIPT	RAW DATA
Figure 6.6	`2026-11-04-v2-ttff.json`	`bench/ttff.mjs`	Table D.2
Table 6.3	`2026-11-04-v2-qoe-unthrottled.json`, `2026-11-04-v2-qoe-fast3g.json`, `2026-11-04-v2-qoe-slow3g.json`	`qoe/collect.js`	Table D.3
Bandwidth drop, Section 6.4.2	`2026-11-12-v2-qoe-drop.json`	`qoe/collect.js`	Table D.4
Table 6.4	`2026-11-04-v2-bitrate.json`	`bench/bitrate.mjs`	Table D.5
Figure 6.7, Table 6.5	`2026-11-04-v2-s2b-resized.json`	`bench/transcode.mjs`	Table D.6
Real recording, Section 6.5.1	`2026-11-03-v2-real-recording.json`	Execution history, read back	The file itself
Figure 6.8	`2026-11-04-v2-s4-long.json`	`bench/transcode.mjs`	Table D.7
Damaged uploads, Section 6.5.3	`2026-11-11-v2-fault.json`	`bench/fault.mjs`	Table D.8
Figure 6.9	`2026-11-04-v2-s3c-burst.json`	`bench/transcode.mjs`	Tables D.9 and D.11
Table 6.6	`2026-11-04-v2-api-final.json`	`bench/api-load.mjs`	Tables D.10 and D.11
Table 6.7	`2026-11-04-v2-prices.json`, `2026-11-05-v2-rekognition-price.json`	`bench/cost.mjs`, AWS Price List Query API	Table D.12

The scripts are under `scripts`. Every run was made on the production environment, with the configuration of Chapter 5.

Percentiles use the nearest rank: the p-th percentile of n sorted values is the value at position $\lfloor p\,n \rfloor$, counted from zero and capped at the last one. With 20 values or fewer, the 95th percentile is therefore the largest value, which is why the columns 95TH PCT. and MAX agree in Table D.2 and in the bursts of 10 and 20 requests of Table D.10.

D.1 Playback

Table D.2 breaks the requests of Figure 6.6 into their steps. The first request found every cache cold; the other 19 followed it one after another.

Table D.2: Requests before the first frame of the 720p rendition, in milliseconds; the first request and 19 that followed it

STEP	FIRST	MEAN	MEDIAN	95TH PCT.	MIN	MAX
Playback grant (API)	479.4	122.3	121.6	137.5	108.2	137.5
Master playlist (CDN)	376.4	8.2	6.9	18.0	5.5	18.0
Rendition playlist (CDN)	128.3	10.8	8.1	56.9	5.7	56.9
Key (API)	175.4	150.0	144.2	216.6	125.2	216.6
First segment (CDN)	265.0	28.1	22.7	81.4	19.7	81.4
Total	1424.9	319.5	307.5	435.2	288.4	435.2

Table D.3 gives every playback run behind Table 6.3. The rung is read from the size of the video element's picture, so it is the rung on screen rather than the one being downloaded; no run changed rung while it played. The bit rate is that of the rung on screen, computed from the bytes of the segments of that rung which the run downloaded. Without throttling, the player buffers the whole video, so the value equals the average of the rung over the whole video (716 kbit/s at 480p, Table 6.4). Under Fast 3G and Slow 3G it downloads only the first one to two minutes, which are harder to compress than the rest, so the value lies above that average: 809 against 716 kbit/s at 480p, and 258 against 249 kbit/s at 240p.

Table D.3: Playback runs of 60 seconds under three network profiles

PROFILE	RUN	START-UP, S	STALLS	STALLED, S	KBIT/S	RUNG SHOWN
Unthrottled	1	3.23	0	0.0	716	850×480
	2	3.10	0	0.0	716	850×480
	3	2.09	0	0.0	716	850×480
	4	1.98	0	0.0	716	850×480
	5	2.27	0	0.0	716	850×480
Fast 3G	1	6.07	0	0.0	809	850×480
	2	5.74	0	0.0	809	850×480
	3	5.70	0	0.0	809	850×480
	4	5.87	0	0.0	809	850×480
	5	8.89	0	0.0	795	850×480
Slow 3G	1	9.13	0	0.0	258	426×240
	2	9.01	0	0.0	258	426×240
	3	9.29	0	0.0	258	426×240
	4	8.88	0	0.0	258	426×240
	5	8.90	0	0.0	258	426×240

Table D.4 gives the runs with a bandwidth drop. The drop is applied twenty seconds after playback is requested. A lower rung counts as requested at the first segment request below the highest rung requested so far, and as on screen at the first change of the picture to a rung below the highest shown so far.

Table D.4: Playback runs of 90 seconds that start on Fast 3G and drop to Slow 3G after 20 seconds: the rung shown before the drop, and the seconds from the drop until a lower rung was asked for and shown

RUN	START-UP	BEFORE	ASKED	RUNG	SHOWN	RUNG	STALLS
1	5.94	480p	8.6	240p	20.2	240p	0
2	5.83	480p	8.5	240p	20.2	240p	0
3	6.22	480p	8.9	240p	20.4	240p	0
4	5.94	480p	8.5	360p	20.6	360p	0
5	6.05	480p	8.7	240p	20.3	240p	0

Table D.5 compares the playlists of the three measured videos with their segments. The first video is the recording of Table 6.4; the other two are synthetic test patterns.

Table D.5: Declared (decl.) and delivered (deliv.) bit rate of three videos, in kbit/s; count of segments, and length of a segment without the last one

			PEAK		AVERAGE		
VIDEO	**RUNG**	**COUNT**	**DECL.**	**DELIV.**	**DECL.**	**DELIV.**	**LENGTH, S**
Recording	144p	47	201	201	154	154	5.97–6.01
	240p	47	408	408	249	249	5.97–6.01
	360p	47	688	688	420	420	5.97–6.01
	480p	47	1178	1178	716	716	5.97–6.01
	720p	47	2443	2443	1578	1578	5.97–6.01
	1080p	47	4933	4933	3562	3562	5.97–6.01
Test pattern A	144p	31	181	181	59	59	6.00–6.00
	240p	31	253	253	74	74	6.00–6.00
	360p	31	334	334	89	89	6.00–6.00
	480p	31	406	406	122	122	6.00–6.00
	720p	31	569	569	189	189	6.00–6.00
	1080p	31	750	750	274	274	6.00–6.00
Test pattern B	144p	20	61	61	59	59	6.00–6.00
	240p	20	74	74	73	73	6.00–6.00
	360p	20	91	91	88	88	6.00–6.00
	480p	20	122	122	122	122	6.00–6.00
	720p	20	190	190	188	188	6.00–6.00
	1080p	20	278	278	274	274	6.00–6.00

D.2 Transcoding

Table D.6 lists the twelve runs behind Figure 6.7. Intake is the time from the end of the upload to the end of the probe, and the start column is the time from the end of the upload until the first Fargate task starts, so it includes the intake. Analysis runs in parallel with encoding, so the columns do not add up to the total.

Table D.6: Transcoding runs by file size, in seconds from the end of the upload; intake ends with the probe, and start is when the first task starts; encoding is the single job or the chunk array

SOURCE	RUN	CHUNKS	INTAKE	START	ANALYSE	ENCODE	STITCH	READY	vCPU-S
36 s clip	1	–	24.9	55.2	24.9	30.4	–	112.1	171
	2	–	22.2	55.6	24.1	30.0	–	94.1	168
	3	–	22.2	45.3	24.4	28.0	–	110.6	161
100 MB	1	–	22.4	42.8	32.9	68.1	–	122.2	611
	2	–	21.7	57.0	35.1	89.8	–	148.4	789
	3	–	22.1	48.4	39.5	62.8	–	119.6	581
500 MB	1	10	11.2	33.8	76.3	111.2	1.5	148.7	3757
	2	10	23.2	41.8	76.5	101.8	0.9	146.9	3120
	3	10	22.2	41.1	92.2	102.4	0.7	146.9	3477
1 GB	1	18	22.8	45.7	90.8	241.1	1.7	291.0	7853
	2	18	13.5	35.2	66.3	245.1	1.5	286.6	7328
	3	18	6.6	29.2	74.7	251.6	0.9	284.0	7226

Table D.7: Steps of the four-hour video, 5.259 GiB, 48 chunks of 4 vCPUs

STEP OR QUANTITY	VALUE
Upload of the source	132.6 s
Intake: workflow start and probe	12.8 s
End of upload to the first task	33.9 s
Analyse, in parallel with the audio	107.3 s
Audio of the whole video	649.2 s
Chunk array, 48 chunks	1591.6 s
Stitch	2.3 s
Publish	0.3 s
End of upload to ready	2298.9 s
Work of all tasks	67285 vCPU-s
Tasks on Spot and On-Demand	27 and 23

Table D.8 lists the uploads of damaged files of Section 6.5.3. The image is the transcoder release that ran; attempts are those of the failed step, and for a Batch step the exit codes of its attempts. The first run of the truncated file failed on an error of the Batch API, not of the file, and the second shows the defect that the release 9f43f64 fixed.

Table D.8: Uploads of damaged files on production; time in seconds from the end of the upload to the end of the workflow

FILE	IMAGE	FAILED STEP	ATTEMPTS	RESULT	TIME
Random bytes	54da21d	Probe	1	ERROR	34.9
Truncated, run 1	54da21d	Encode	1	ERROR	26.2
Truncated, run 2	54da21d	–	–	READY	141.1
Truncated, after the fix	9f43f64	Encode	1 (exit 3)	ERROR	154.2

D.3 Load

Table D.9 summarises the 89 videos of Figure 6.9 that reached READY with a timing record; the three that became ready after a refused completion call have none, because the measurement tool stopped following them, and the `follow_up` field of the result file records their executions. Table D.11 gives the log of the API function during that run and during the API run of Table D.10.

Table D.9: One hundred simultaneous uploads: 92 ready (89 timed, 3 of the 3 with a refused completion call) and 8 refused at creation; times of the timed videos in seconds

TIME PER VIDEO	MIN	MEDIAN	95TH PCT.	MAX
End of upload to the first task	24.9	292.3	583.5	606.6
First task to ready	28.4	41.0	79.8	153.9
End of upload to ready	57.0	331.2	615.1	638.4
Tasks run on Spot and On-Demand: 96 and 82				

Table D.10: Latency of read requests through the Worker by phase, in milliseconds

PHASE	SENT	200	MEAN	MEDIAN	95TH PCT.	MAX	WALL
Sequential, warm	40	40	166	156	263	279	–
Burst of 10	10	10	2347	2730	2916	2916	2922
Burst of 20	20	20	1540	1336	2682	2682	2686
Burst of 40	40	40	2217	2547	3382	3390	3396
Burst of 80	80	80	4280	4395	5663	5682	5708

Table D.11: Log of the API function during the API run and the burst of uploads; times in milliseconds

QUANTITY	API RUN	BURST OF UPLOADS
Invocations	193	–
Most instances at once	–	80
New instances	79	80
Start-up of an instance, mean	1101	1106
Start-up of an instance, max	1248	1210
Handler, median	1394	512
Handler, 95th percentile	3938	7803
Handler, max	4077	8846
Throttled invocations	–	56
Failed invocations	–	0

D.4 Prices

Table D.12 gives the list prices read for the cost comparison of Table 6.7.

Table D.12: List prices in Singapore, in USD

ITEM	PRICE	UNIT
Fargate, vCPU	0.05056	per vCPU-hour
Fargate, memory	0.00553	per GB-hour
EC2 t3.medium, 2 vCPUs, 4 GiB	0.0528	per hour
EC2 t3.xlarge, 4 vCPUs, 16 GiB	0.2112	per hour
EC2 c6i.xlarge, 4 vCPUs, 8 GiB	0.196	per hour
EC2 c6i.2xlarge, 8 vCPUs, 16 GiB	0.392	per hour
Lambda, requests	0.20	per million requests
Lambda, compute	0.0000166667	per GB-second
S3 Standard	0.024	per GB-month
Rekognition, moderation	0.0013	per image